MINES Saint-Étienne

Une école de l'IMT

ARSENE

FRANCE 2030

Mécanismes de sécurité d'un processeur RISC-V face aux attaques par injection de fautes

PhD Defense
Théophile Gousselot
Mines Saint-Étienne
June 18, 2025

INSPIRING INNOVATION SINCE 1816

MINES Saint-Étienne

Une école de l'IMT

ARSENE

FRANCE 2030

Mécanismes de sécurité d'un processeur RISC-V face aux attaques par injection de fautes

Sous l'encadrement de :

- **Jean-Baptiste Rigaud**
- **Jean-Max Dutertre**
- **Olivier Potin**

PhD Defense

Théophile Gousselot

Mines Saint-Étienne

June 18, 2025

INSPIRING INNOVATION SINCE 1816

MINES Saint-Étienne
Une école de l'IMT

Context

Context
Arsene project

ARchitectures SEcurisées pour le Numérique Embarqué (ARSENE)

➜ Secure embedded systems

Context
Arsene project

ARchitectures SEcurisées pour le Numérique Embarqué (ARSENE)

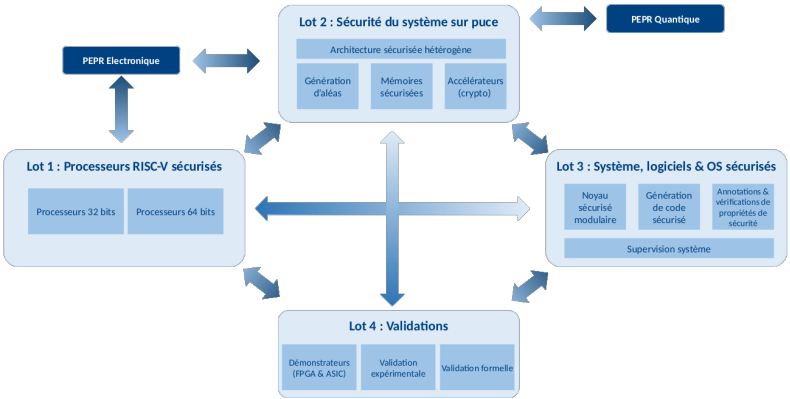
➜ Secure embedded systems

... against physical attacks.

Context
Arsene project

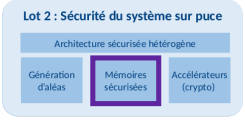
ARchitectures SEcurisées pour le Numérique Embarqué (ARSENE)

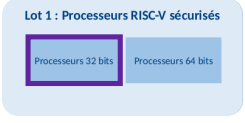
➜ Secure embedded systems



ARchitectures SEcurisées pour le Numérique Embarqué (ARSENE)

➜ Secure embedded systems

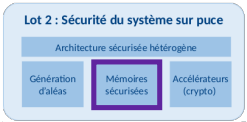




ARchitectures SEcurisées pour le Numérique Embarqué (ARSENE)

➜ Secure embedded systems

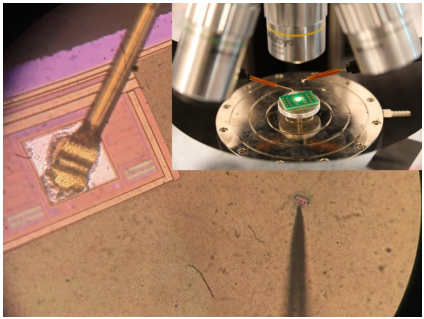
Texplained



Manuscript, Part 3

➜ Linear Code Extraction
➜ Microprobing

✔ Assess RISC-V microarchitecture
✔ 3 Hardware Countermeasures
➜ Code Confidentiality

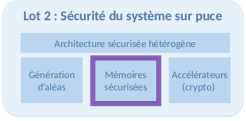


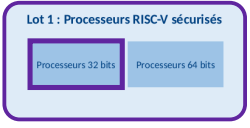
Microprobing

Context
Contributions

ARchitectures SEcurisées pour le Numérique Embarqué (ARSENE)

➜ Secure embedded systems



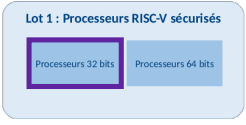


Context
Contributions

ARchitectures SEcurisées pour le Numérique Embarqué (ARSENE)

➜ Secure embedded systems

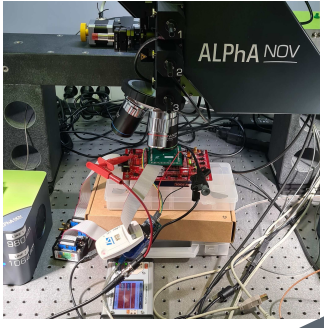
➜ Fault Injection Attacks



Manuscript, Part 2

✔ Execution Integrity

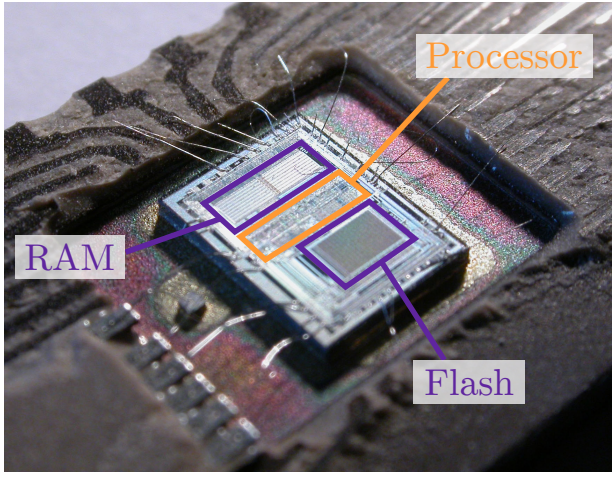
Electromagnetic fault injection



Laser beam

Technical Introduction

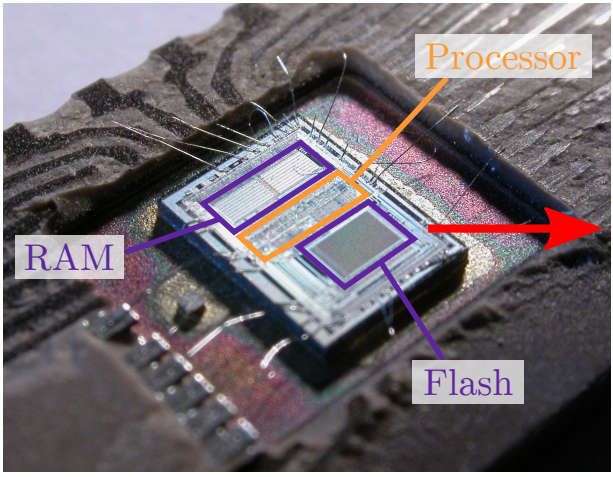
Technical Introduction
Device to protect



Microcontroller

Threat model

- Memory instruction extraction



Microcontroller

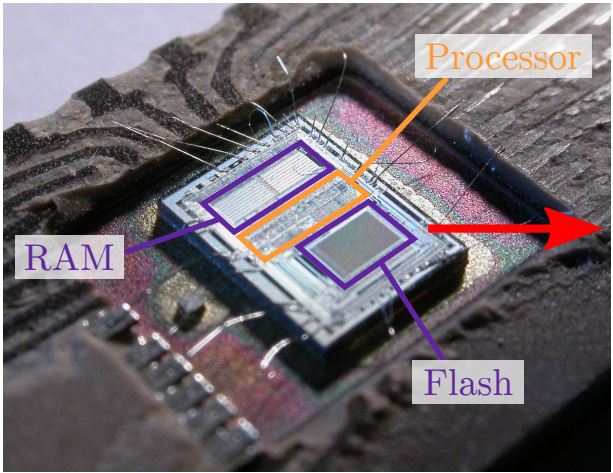
Technical Introduction
Threat model

Threat model

- Memory instruction extraction

Use-cases

➜ Identify vulnerabilities



Microcontroller

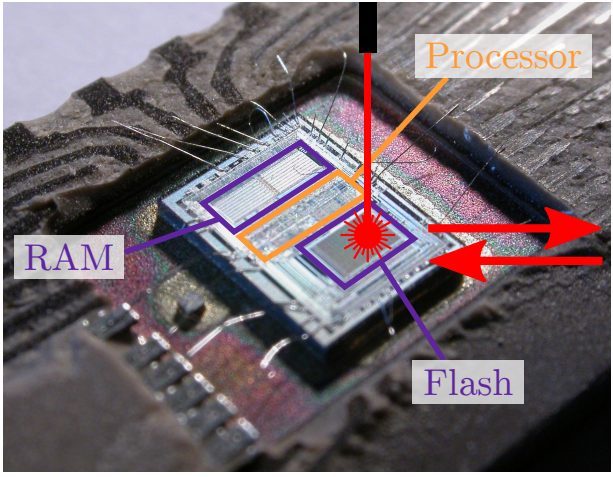
Technical Introduction
Threat model

Threat model

- Memory instruction extraction
- Memory instruction corruption
- FIA on instruction memory

Use-cases

➜ Identify vulnerabilities



Microcontroller

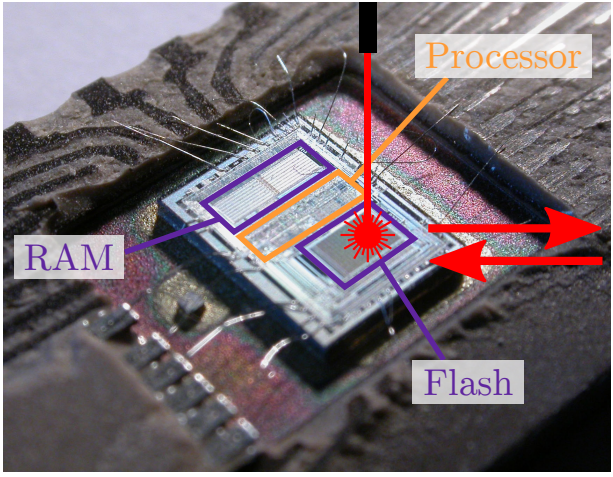
Technical Introduction
Threat model

Threat model

- Memory instruction extraction
- Memory instruction corruption
- FIA on instruction memory

Use-cases

- ➜ Identify vulnerabilities
- ➜ Bypass PIN, hash, ... verification



Microcontroller

Technical Introduction
Threat model

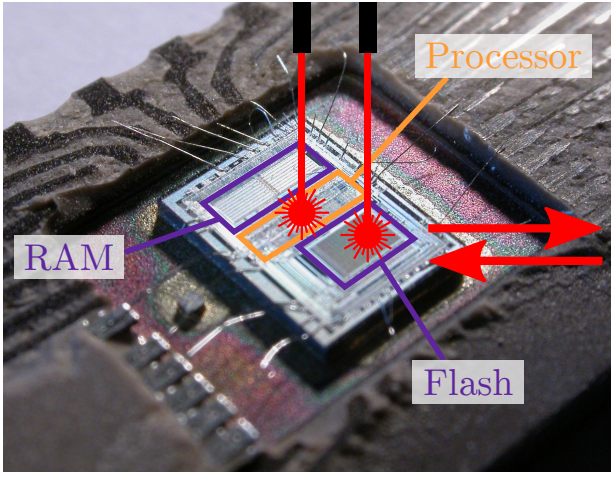
Threat model

- Memory instruction extraction
- Memory instruction corruption
- FIA on instruction memory
- FIA on processor control logic

Use-cases

➜ Identify vulnerabilities

➜ Bypass PIN, hash, ... verification



Microcontroller

Technical Introduction
Security properties

Integrity of:

- ➜ Control Flow (CFI)
- ➜ Instructions
- ➜ Control Signals
- ➜ ~~Data~~

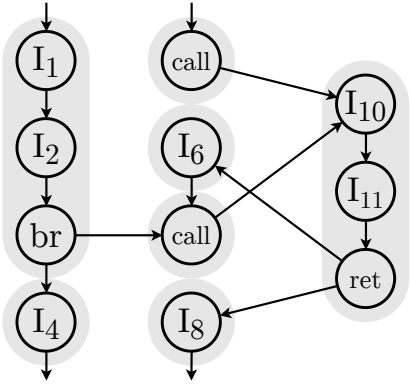
Confidentiality of:

- ➜ Instructions
- ➜ ~~Data~~

Technical Introduction

Control Flow and Instruction Integrity

CFG

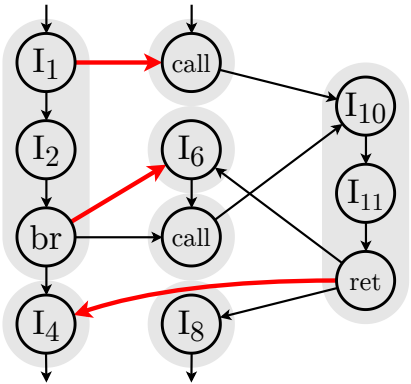


Control Flow Graph

Technical Introduction

Control Flow and Instruction Integrity

CFG



Control Flow Graph

Control Flow Integrity

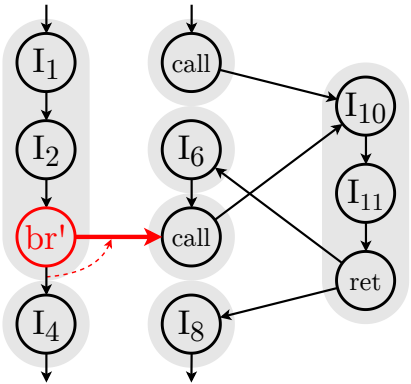
Tamper with:

- Program Counter (PC)
- Data
- Microarchitecture
- Instruction

Technical Introduction

Control Flow and Instruction Integrity

CFG

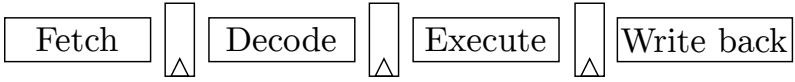


Control Flow Graph

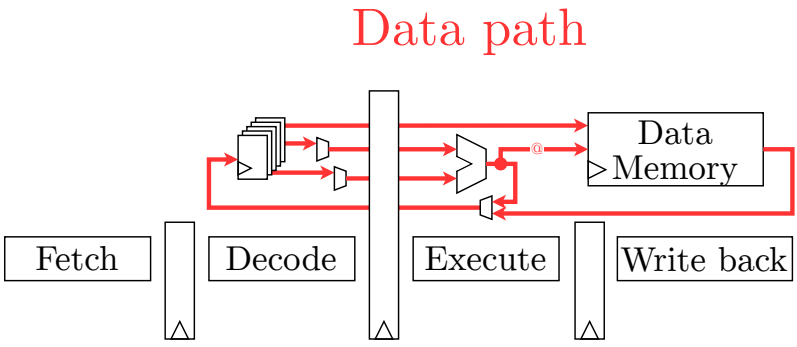
Control Flow Integrity

Instruction Integrity

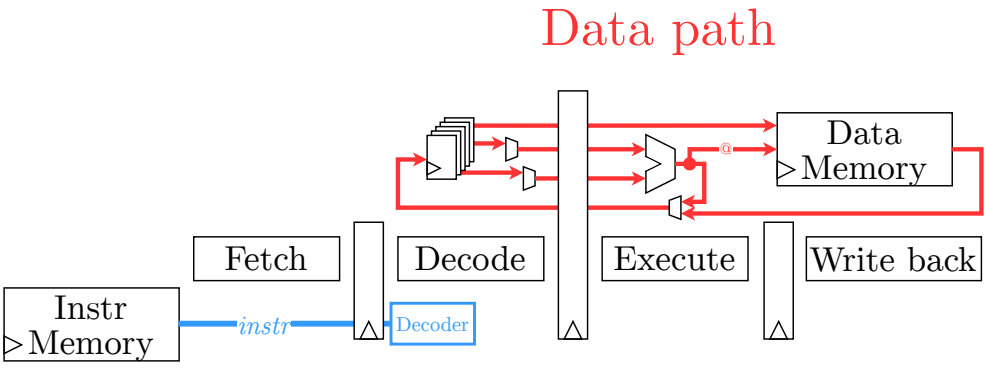
Technical Introduction
Bibliography: Execution integrity



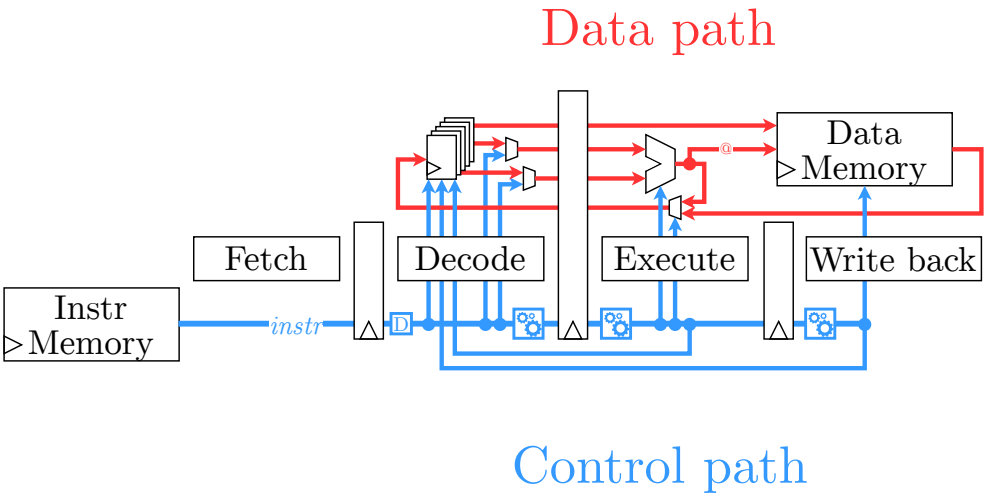
Technical Introduction
Bibliography: Execution integrity



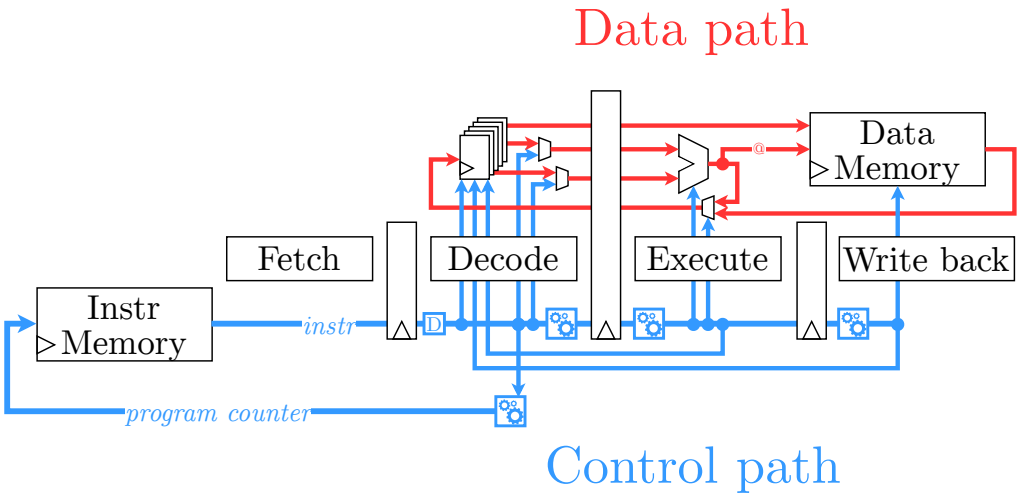
Technical Introduction
Bibliography: Execution integrity



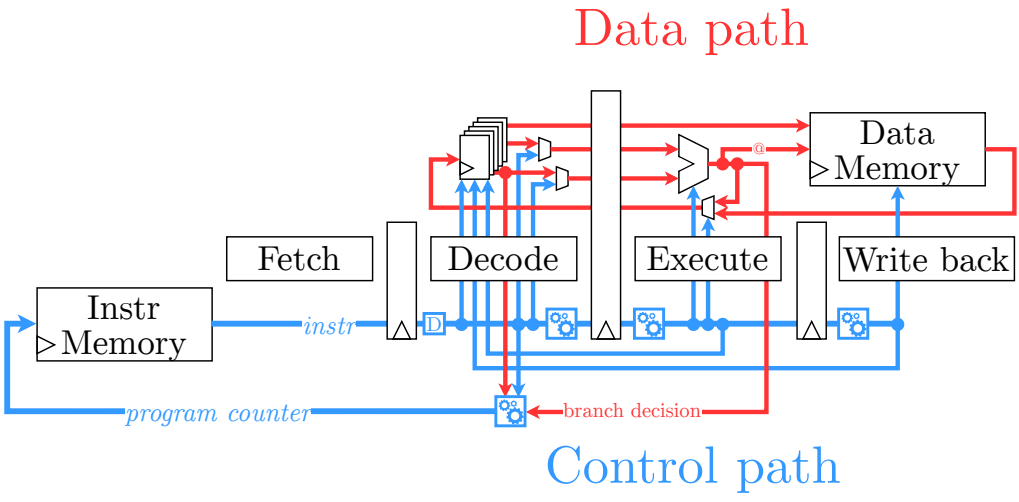
Technical Introduction
Bibliography: Execution integrity



Technical Introduction
Bibliography: Execution integrity

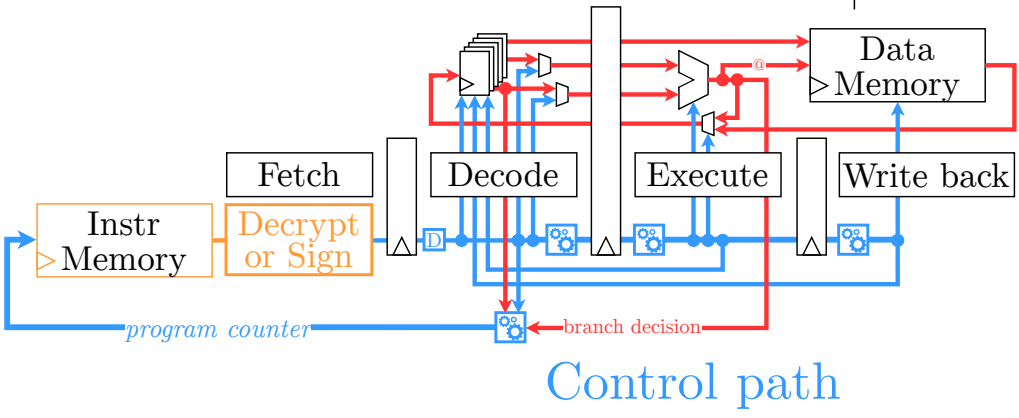


Technical Introduction
Bibliography: Execution integrity

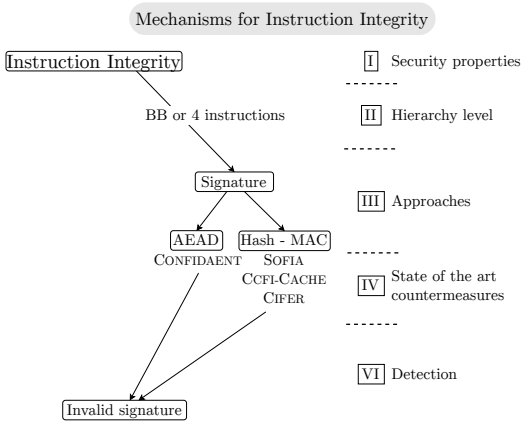


Technical Introduction
Bibliography: Execution integrity

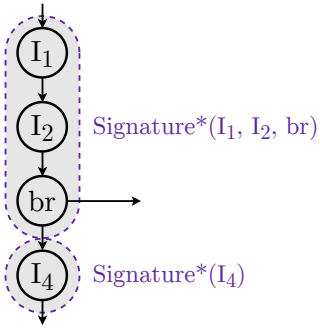
- Gpsa-Csm [7]
- Sofia [8]
- Ccfi-Cache [9]
- Scfp [10]
- Confidaent [11]
- Hapei [12]

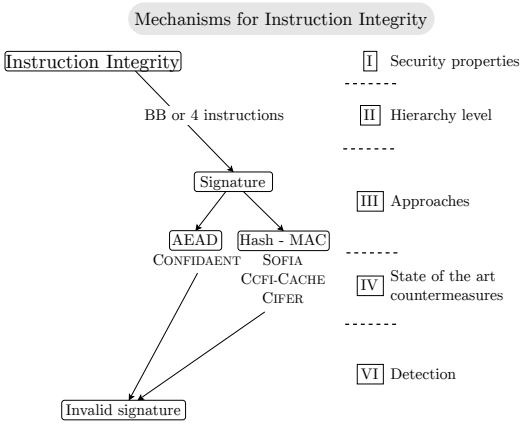


Technical Introduction
Bibliography: Execution integrity



Technical Introduction
Bibliography: Execution integrity



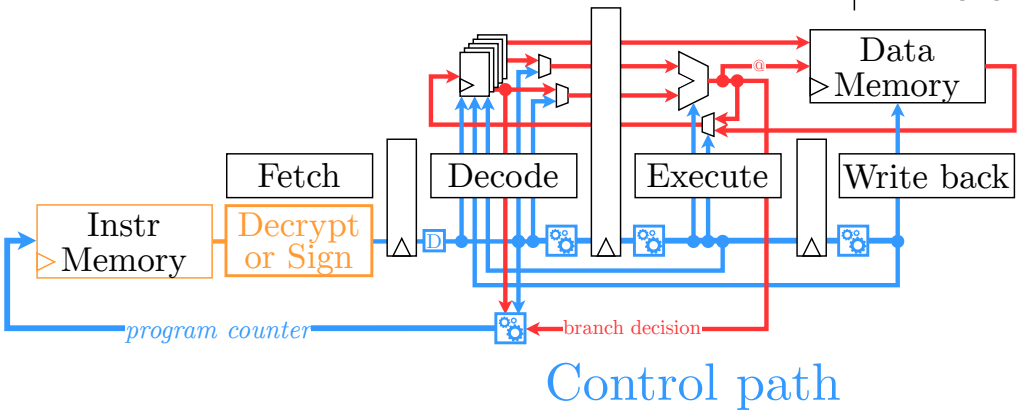


*signature: Hash, Tag, Error Detecting Code

Technical Introduction
Bibliography: Execution integrity

- GPSA-CSM [7]
- SOFIA [8]
- CCFI-CACHE [9]
- SCFP [10]
- CONFIDAENT [11]
- HAPEI [12]

➜ Instruction integrity

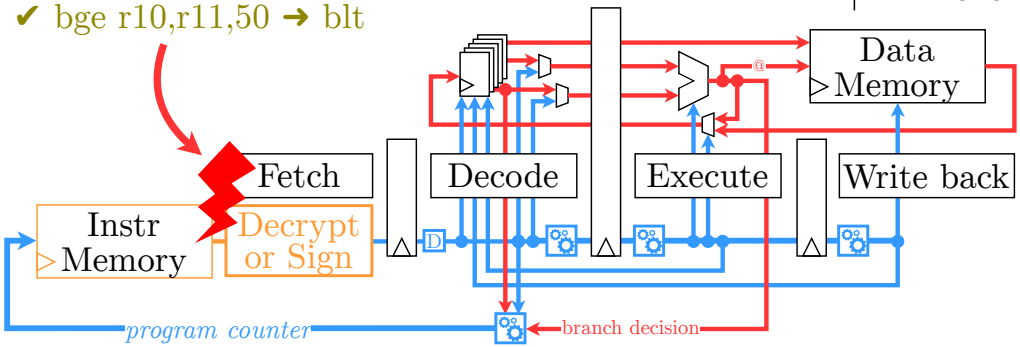


Technical Introduction
Bibliography: Execution integrity

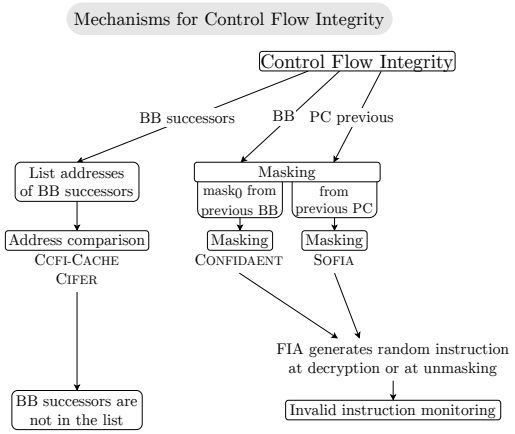
- GPSA-CSM [7]
- SOFIA [8]
- CCFI-CACHE [9]
- SCFP [10]
- CONFIDAENT [11]
- HAPEI [12]

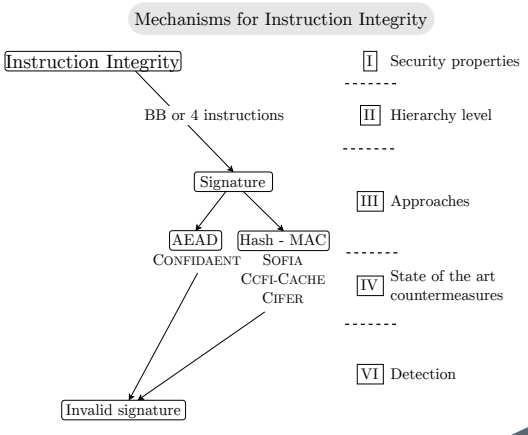
➜ Instruction integrity

✔ bge r10,r11,50 ➜ blt

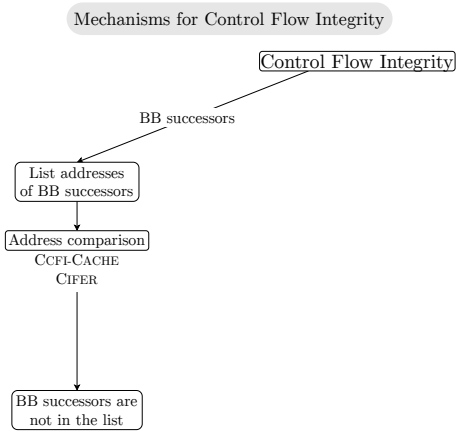


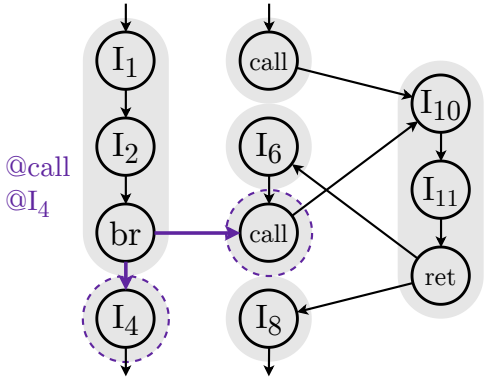
Technical Introduction
Bibliography: Execution integrity





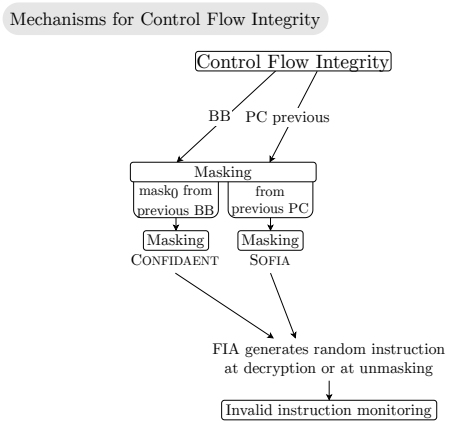
Technical Introduction
Bibliography: Execution integrity

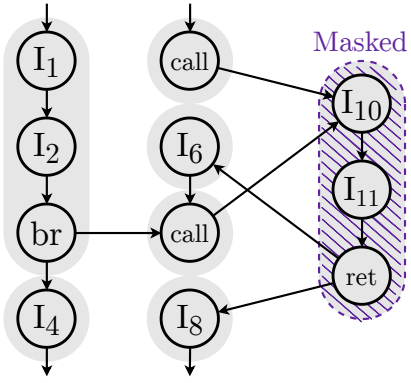




Control Flow Graph

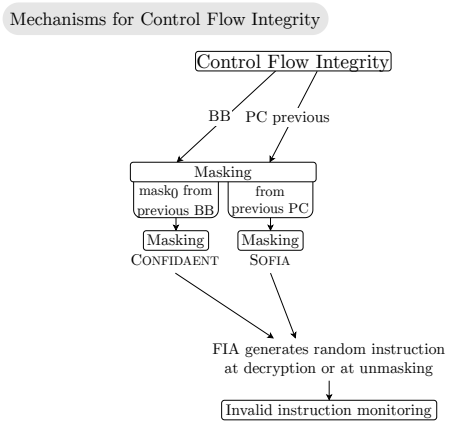
Technical Introduction
Bibliography: Execution integrity

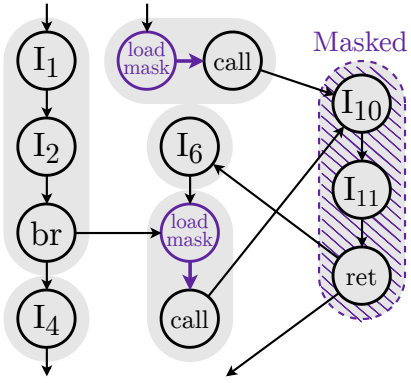




Control Flow Graph

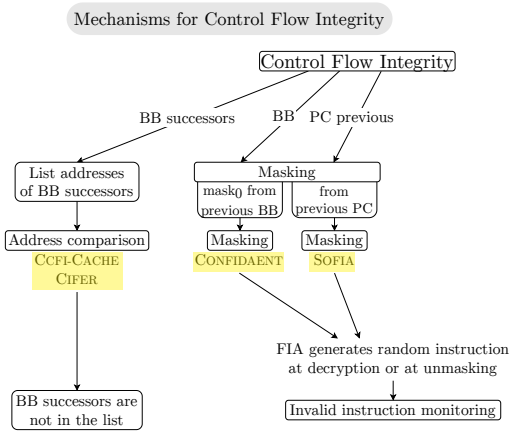
Technical Introduction
Bibliography: Execution integrity

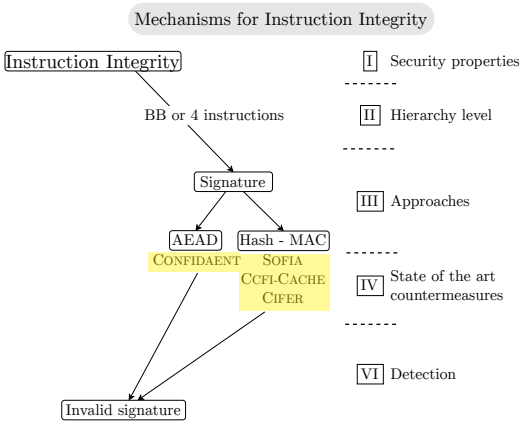




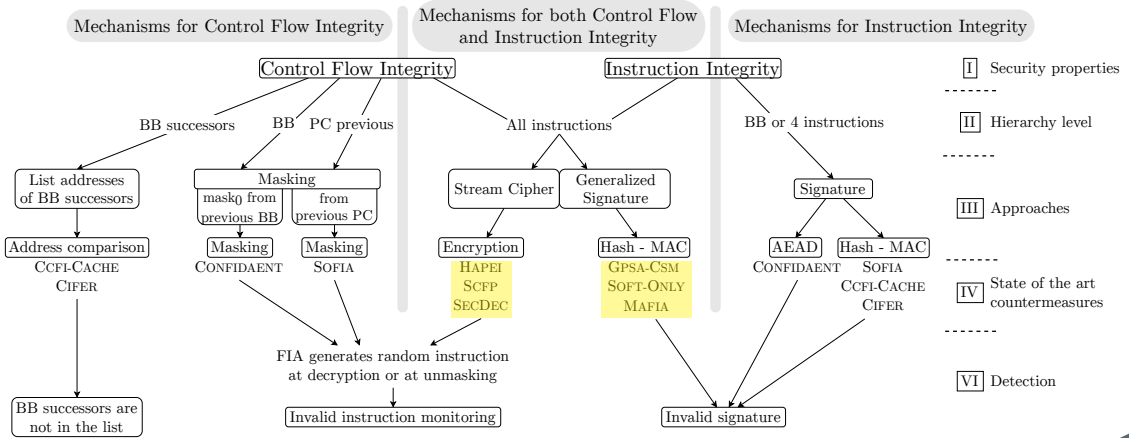
Control Flow Graph

Technical Introduction
Bibliography: Execution integrity





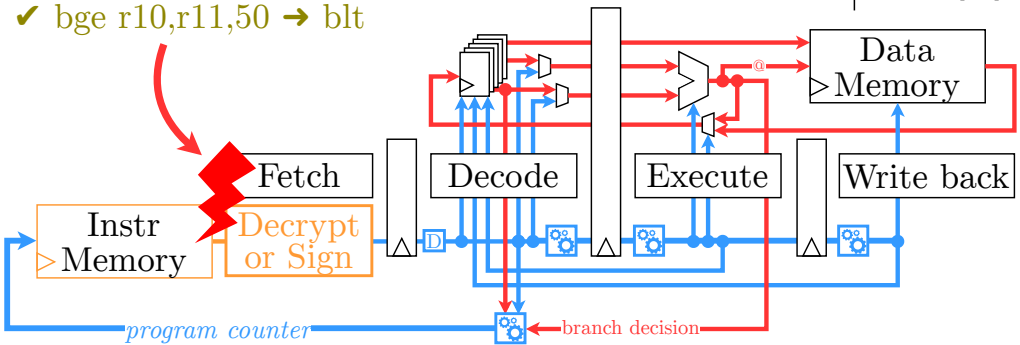
Technical Introduction
Bibliography: Execution integrity



Technical Introduction
Bibliography: Execution integrity

- ➜ Instruction integrity
- ➜ Control flow integrity
  - ✔ bge r10,r11,50 ➜ blt

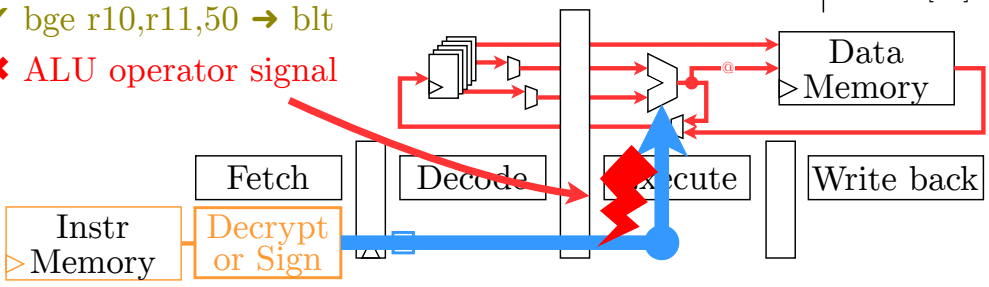
Gpsa-Csm [7]
Sofia [8]
Ccfi-Cache [9]
Scfp [10]
Confidaent [11]
Hapei [12]

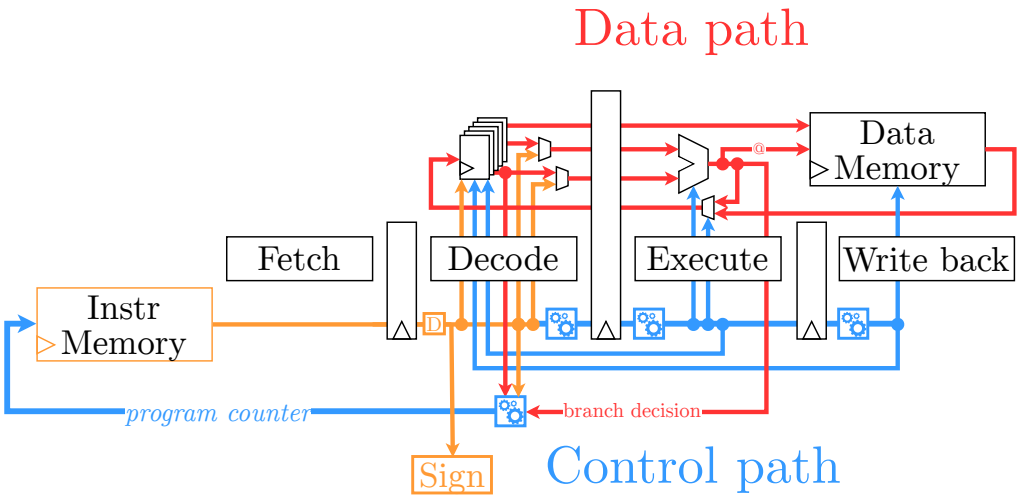


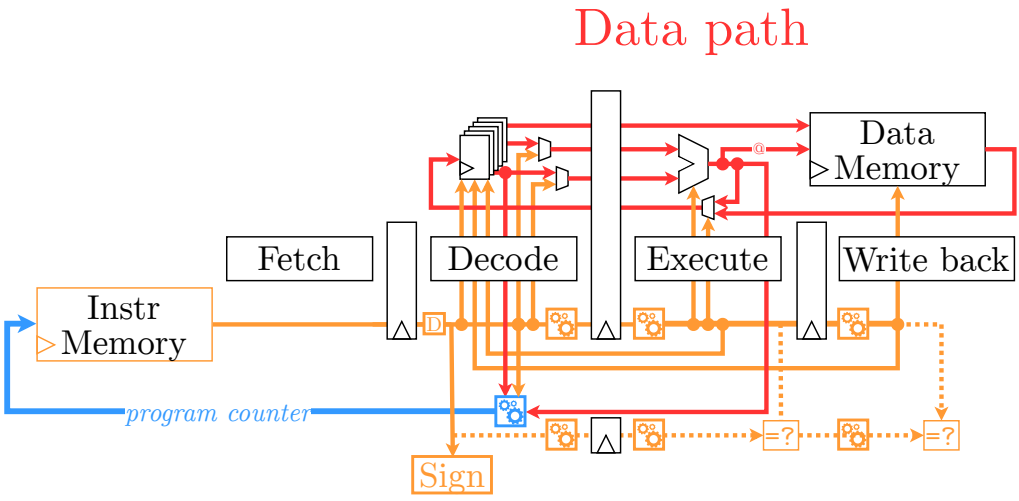
Technical Introduction
Bibliography: Execution integrity

- ➜ Instruction integrity
- ➜ Control flow integrity
  - ✔ bge r10,r11,50 ➜ blt
  - ✖ ALU operator signal

GPSA-CSM [7]
SOFIA [8]
CCFI-CACHE [9]
SCFP [10]
CONFIDAENT [11]
HAPEI [12]




Data path
Data Memory
Fetch
Decode
Execute
Write back
Instr Memory
D
program counter
branch decision
Sign
Control path


Data path
Data Memory
Fetch
Decode
Execute
Write back
Instr Memory
D
program counter
=?
=?
Sign

Technical Introduction

State-of-the-art execution integrity

*against physical fault injection attacks

Name	Instruction Confidentiality	Control Flow Integrity	Instruction Integrity	Control Signal Integrity	Fine-grained CFI	No clock cycle penalty	Memory overhead
Instr. Enc.[15]	✓		✓			✓	+
Confidaent [11]	✓	✓	✓				+
Sofia [8]	✓	✓	✓		✓		+
Ccfi-Cache [9]		✓	✓		✓	✓	+
Cifer [13]		✓	✓	(✓)		✓	+
Soft-Only [16]		✓	✓				++
SecDec [17]		✓	✓	(✓)			+
Hapei [12]	✓	✓	✓		✓		+
Scfp [10]	✓	✓	✓				+
Gpsa-Csm [7]		✓	✓				+
Mafia [14]		✓	✓	✓			+
This work	✓	✓	✓	✓	✓	✓	++

Technical Introduction

State-of-the-art execution integrity

*against physical fault injection attacks

Name	Control Flow Integrity	Instruction Integrity	Control Signal Integrity				
Instr. Enc.[15]	✓		✓			✓	+
Confidaent [11]	✓	✓	✓				+
Sofia [8]	✓	✓	✓		✓		+
Ccfi-Cache [9]		✓	✓		✓	✓	+
Cifer [13]		✓	✓	(✓)		✓	+
Soft-Only [16]		✓	✓				
SecDec [17]		✓	✓	(✓)			+
Hapei [12]	✓	✓	✓		✓		+
Scfp [10]	✓	✓	✓				+
Gpsa-Csm [7]		✓	✓				+
Mafia [14]		✓	✓	✓			+
This work	✓	✓	✓	✓	✓	✓	++

Only signals in decode

Control signal redundancy

Technical Introduction

State-of-the-art execution integrity

*against physical fault injection attacks

Name	Instruction Confidentiality	Control Flow Integrity	Instruction Integrity	Control Signal Integrity	Fine-grained CFI	No clock cycle penalty	Memory overhead
Instr. Enc.[15]	✓		✓			✓	+
Confidaent [11]	✓	✓	✓				+
Sofia [8]	✓	✓	✓		✓		+
Ccfi-Cache [9]		✓	✓		✓	✓	+
Cifer [13]		✓	✓	(✓)		✓	+
Soft-Only [16]		✓	✓				++
SecDec [17]		✓	✓	(✓)			+
Hapei [12]	✓	✓	✓		✓		+
Scfp [10]	✓	✓	✓				+
Gpsa-Csm [7]		✓	✓				+
Mafia [14]		✓	✓	✓			+
This work	✓	✓	✓	✓	✓	✓	++

MINES Saint-Étienne
Une école de l'IMT

Proposed Scheme

Proposed Scheme
Goal

Security properties

- **Integrity** of Control Flow, Instructions and Control Signals (from all stages)
- **Confidentiality** of Instructions

Proposed Scheme
Goal

Security properties

- **Integrity** of Control Flow, Instructions and Control Signals (from all stages)
- **Confidentiality** of Instructions

Can these properties be provided together?

(by an approach other than redundancy)

Proposed Scheme
Constraints

Security properties

- ➜ **Integrity** of Control Flow, Instructions and Control Signals (from all stages)
- ➜ **Confidentiality** of Instructions

Constraints

- ➜ No insertion of instructions
- ✔ Commercial off-the-shelf binaries

Proposed Scheme
Constraints

Security properties

- ➜ **Integrity** of Control Flow, Instructions and Control Signals (from all stages)
- ➜ **Confidentiality** of Instructions

Constraints

- ➜ No insertion of instructions
- ✔ Commercial off-the-shelf binaries

Assumptions (microcontroller)

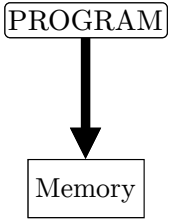
- ➜ No memory cache
- ➜ Indirect jump destinations are known

Proposed Scheme
Our mechanism

Chained Instruction Encryption with Absorption of Control Signals

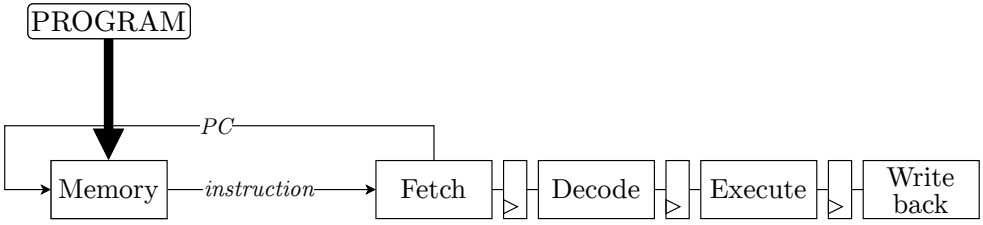
Proposed Scheme
Our mechanism

Chained Instruction Encryption with Absorption of Control Signals



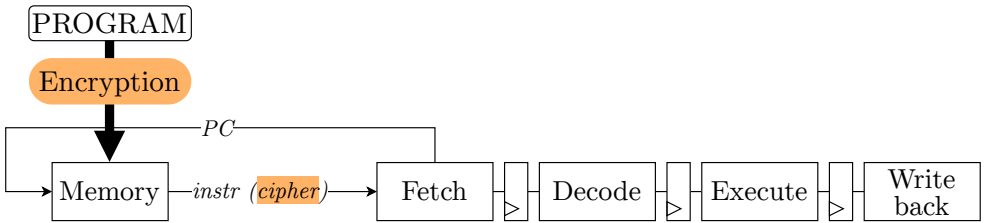
Proposed Scheme
Our mechanism

Chained Instruction Encryption with Absorption of Control Signals



Proposed Scheme
Our mechanism

Chained Instruction Encryption with Absorption of Control Signals

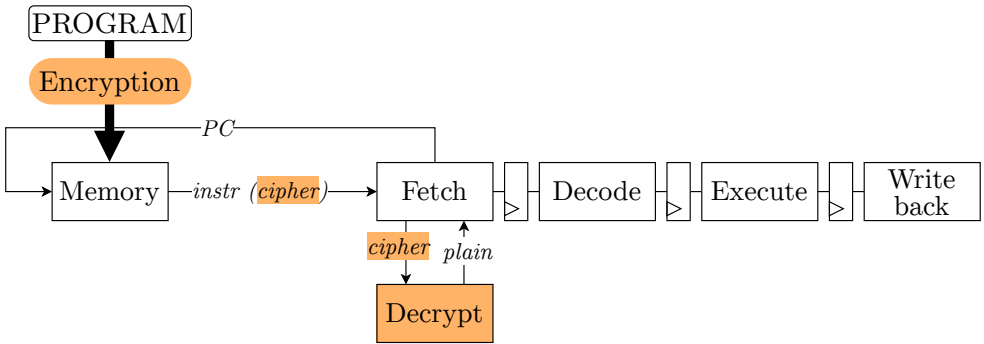


➜ Instruction confidentiality

1 Chained Encryption of Instructions (before programming memory)

Proposed Scheme
Our mechanism

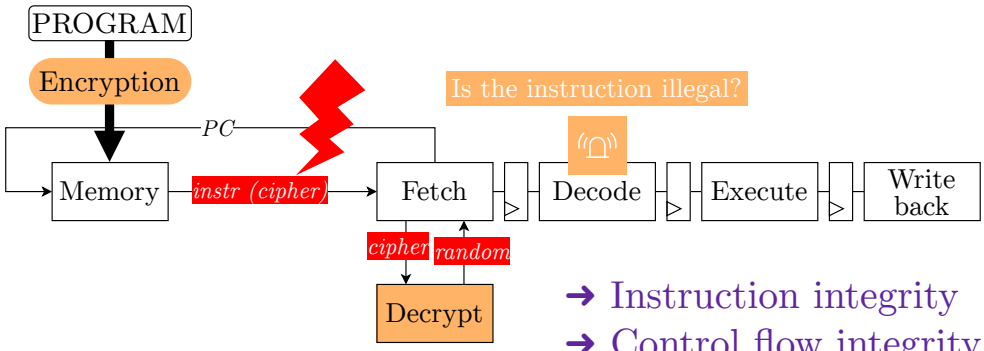
Chained Instruction Encryption with Absorption of Control Signals



1. Chained Encryption of Instructions (before programming memory)
2. On-the-fly Decryption

Proposed Scheme
Our mechanism

Chained Instruction Encryption with Absorption of Control Signals

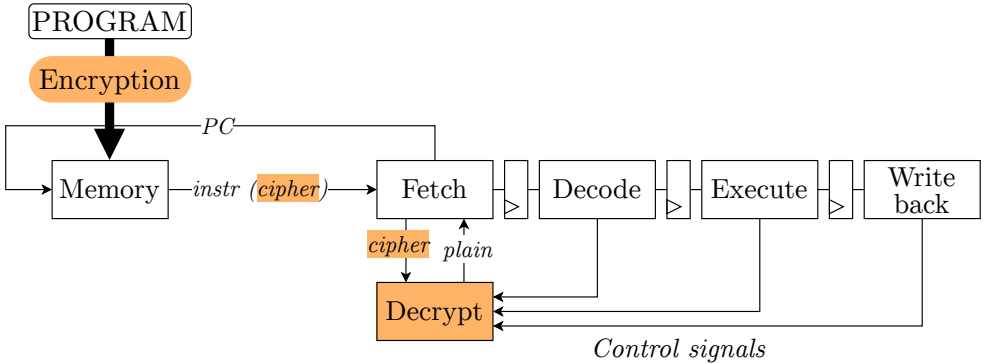


➜ Instruction integrity
➜ Control flow integrity

1. Chained Encryption of Instructions (before programming memory)
2. On-the-fly Decryption

Proposed Scheme
Our mechanism

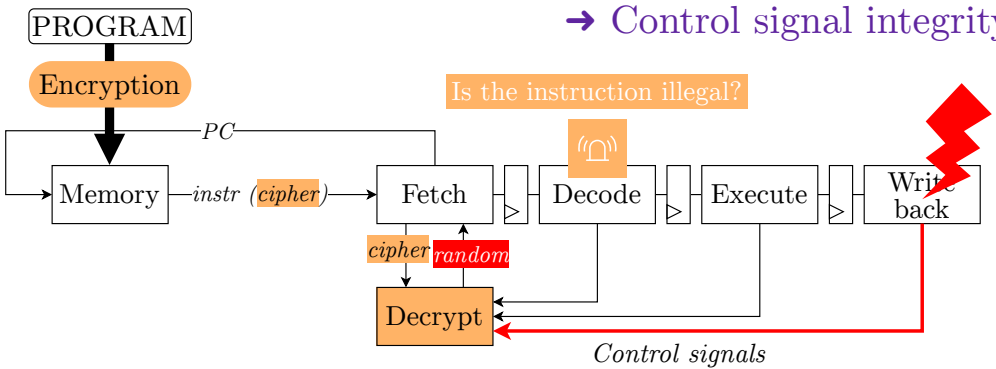
Chained Instruction Encryption with Absorption of Control Signals



1. Chained Encryption of Instructions (before programming memory)
2. On-the-fly Decryption

Proposed Scheme
Our mechanism

Chained Instruction Encryption with Absorption of Control Signals

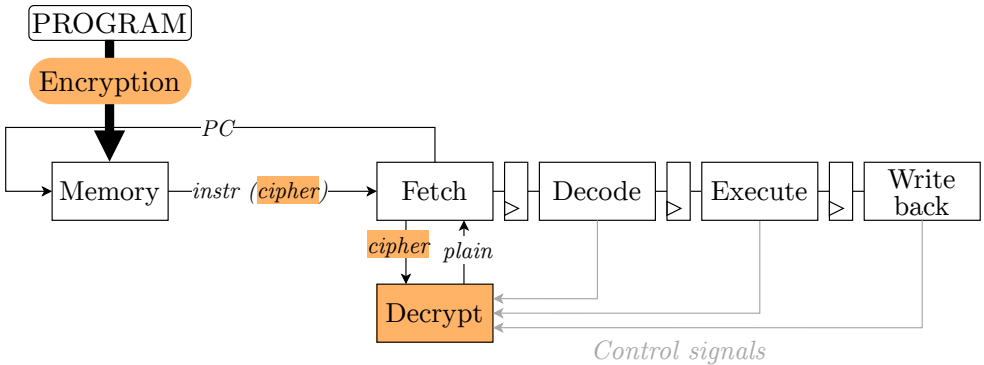


➜ Control signal integrity

1. Chained Encryption of Instructions (before programming memory)
2. On-the-fly Decryption

Proposed Scheme
Our mechanism

Chained Instruction Encryption with Absorption of Control Signals



1. Chained Encryption of Instructions (before programming memory)
2. On-the-fly Decryption

MINES Saint-Étienne
Une école de l'IMT

Chained Instruction Encryption

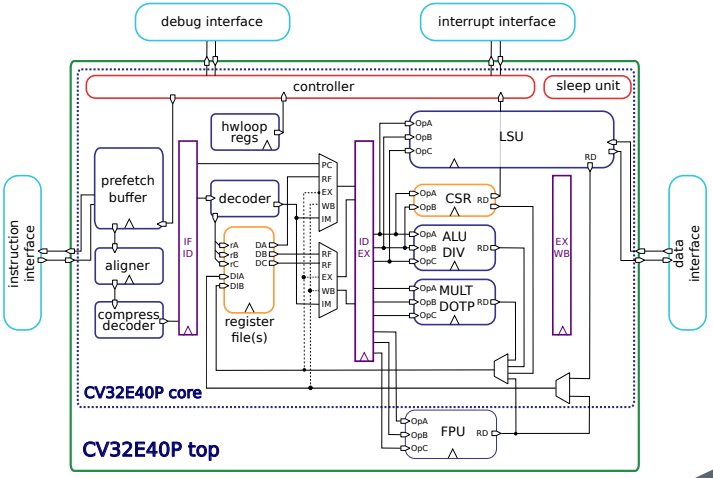
Chained Instruction Encryption

Implementation

RISC-V core: CV32E40P

CV32E40P: RISC-V core

- ➜ Embedded system
- 32-bit
- 4-stage
- In-order
- Forwarding



Chained Instruction Encryption

Implementation

Authenticated Encryption: Ascon

Ascon: cipher suite, which provides Authenticated Encryption with Associated Data

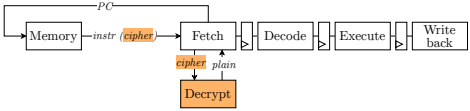
- Winner of CAESAR Authenticated Encryption - Lightweight Cryptography (2019)
- Winner of NIST - Lightweight Cryptography (2023) $\Rightarrow$ **standardize the Ascon family**
- Lightweight (better Throughput per Area than AES [1])
- Highly tested
- Large security margins

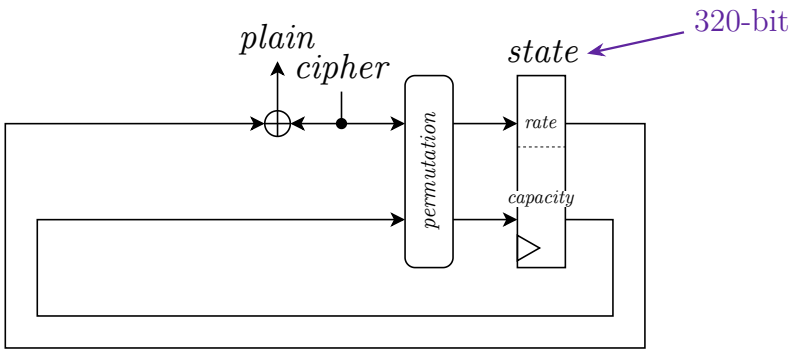
[1] *"Need for Low-latency Ciphers: A Comparative Study of NIST LWC Finalists"*, NIST LWC Workshop 2022, Tolga Yalcin - Google

Chained Instruction Encryption

Implementation

Hardware decryption module



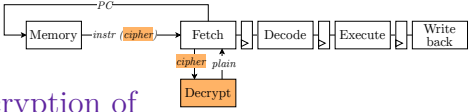


Hardware decryption

Chained Instruction Encryption

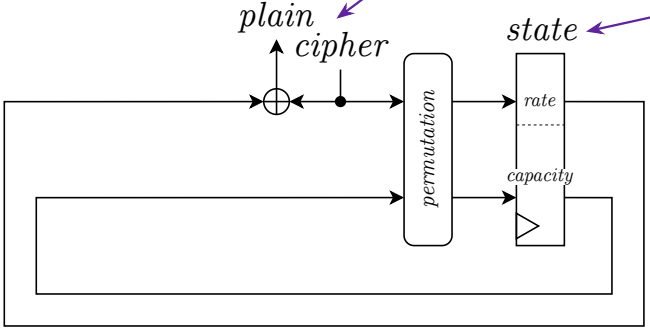
Implementation

Hardware decryption module



Decryption of instructions

320-bit

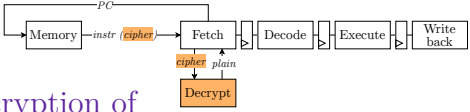


Hardware decryption

Chained Instruction Encryption

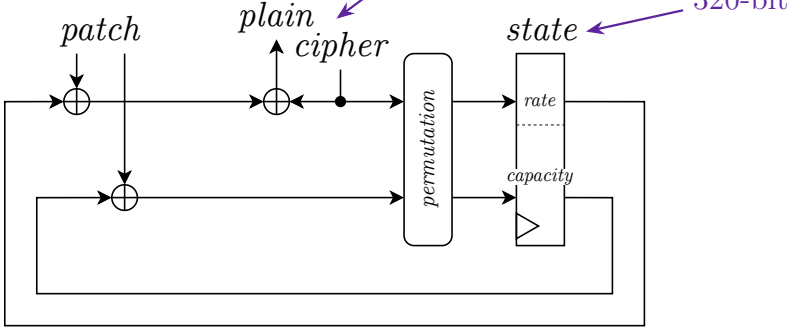
Implementation

Hardware decryption module



Decryption of instructions

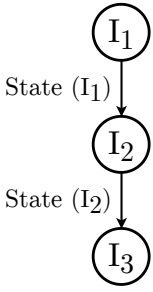
320-bit



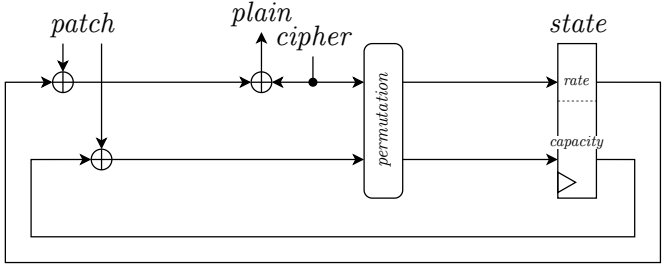
Hardware decryption

Chained Instruction Encryption

Need for patches

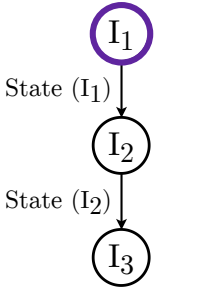


One predecessor

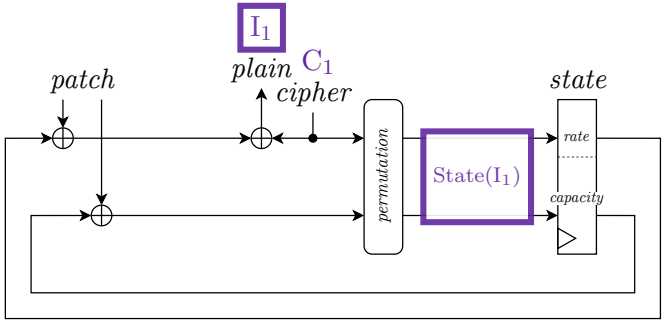


Chained Instruction Encryption

Need for patches

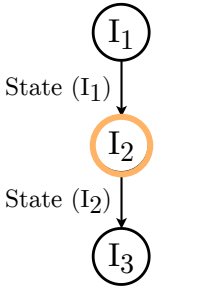


One predecessor

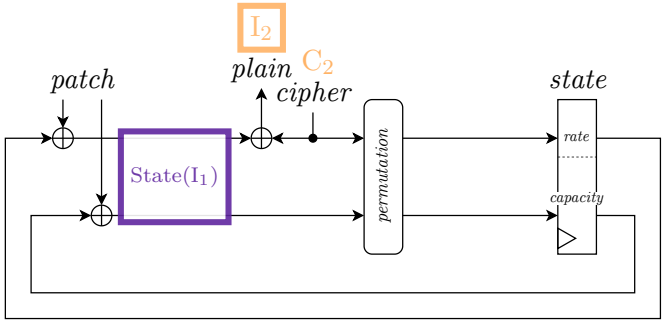


Chained Instruction Encryption

Need for patches

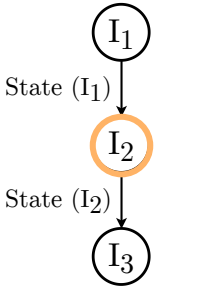


One predecessor

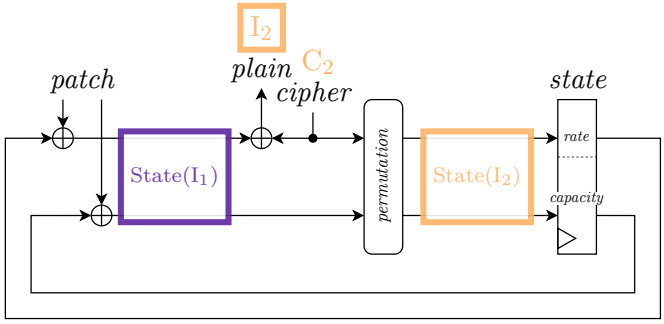


Chained Instruction Encryption

Need for patches

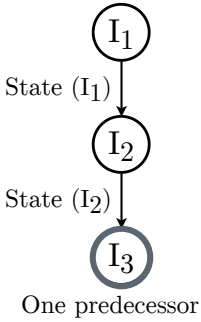


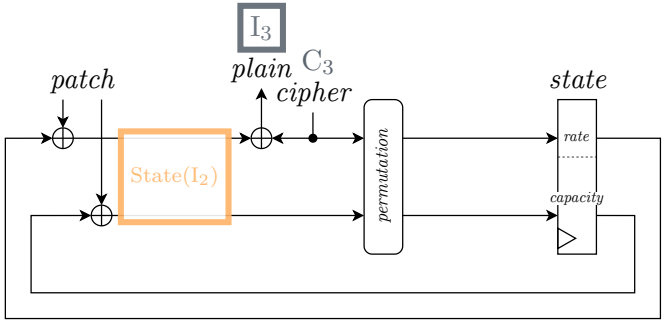
One predecessor



Chained Instruction Encryption

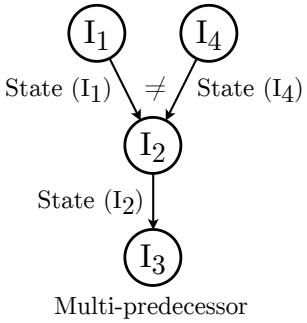
Need for patches



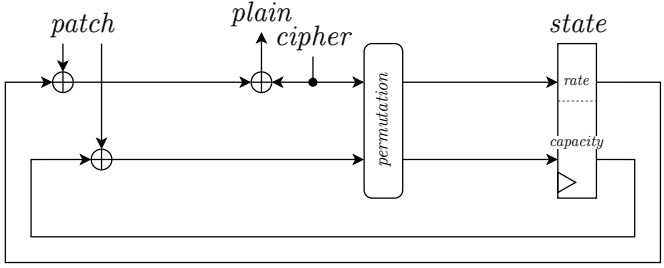


Chained Instruction Encryption

Need for patches

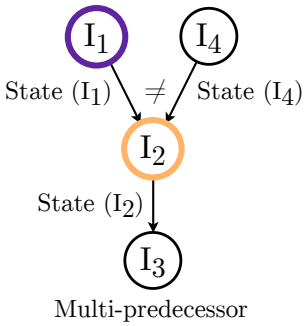


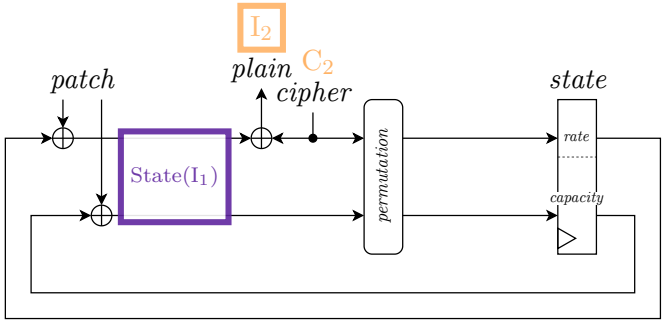
Multi-predecessor



Chained Instruction Encryption

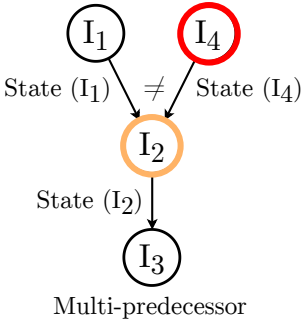
Need for patches



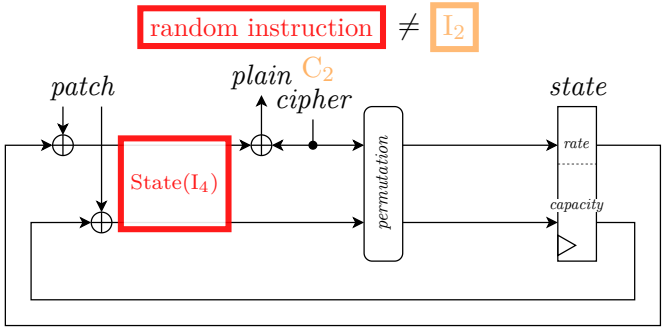


Chained Instruction Encryption

Need for patches

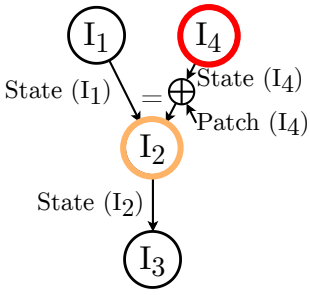


Multi-predecessor



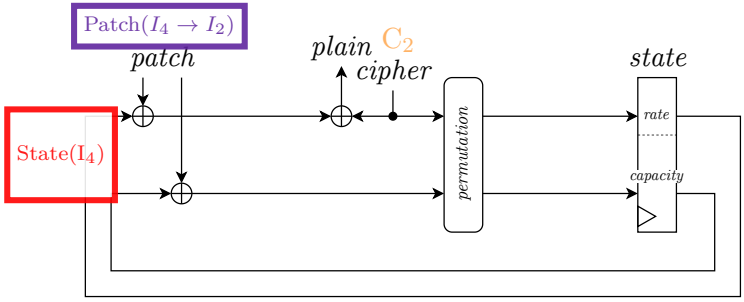
Chained Instruction Encryption

Need for patches



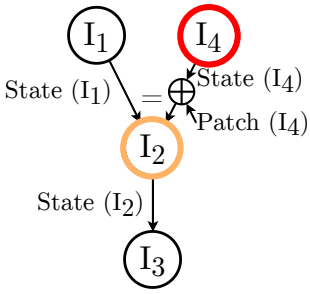
Patching multi-predecessor

Patch (I_4) = State (I_1) xor State (I_4)



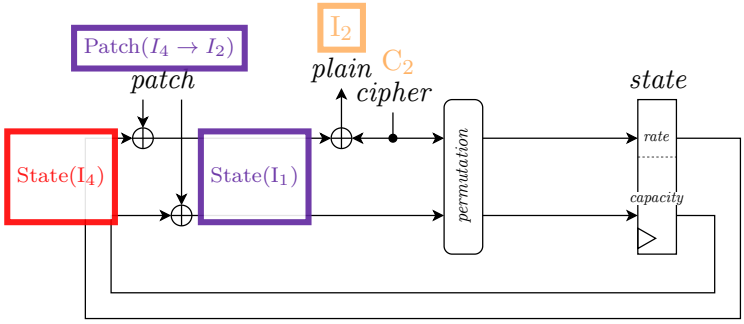
Chained Instruction Encryption

Need for patches



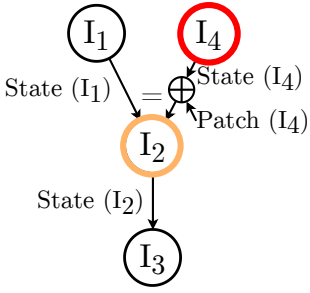
Patching multi-predecessor

Patch (I_4) = State (I_1) xor State (I_4)



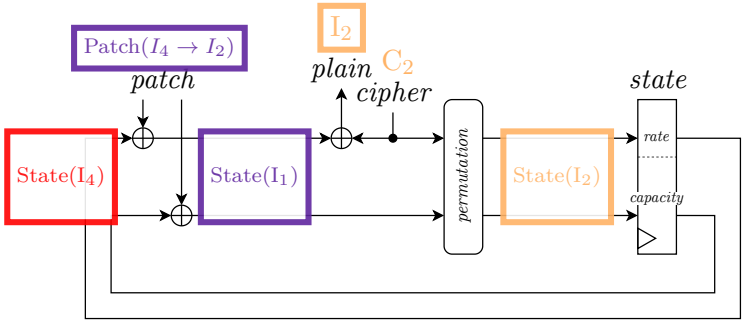
Chained Instruction Encryption

Need for patches



Patching multi-predecessor

Patch (I_4) = State (I_1) xor State (I_4)



Chained Instruction Encryption
Patch policy

- How instructions are ordered in the chain?
- Which CFG edges are patched?
- Where patches are stored?
- How patches are accessed?
- When a patch should be applied?

Chained Instruction Encryption
Patch policy

- How instructions are ordered in the chain?
- Which CFG edges are patched?
- Where patches are stored?
- How patches are accessed?
- When a patch should be applied?

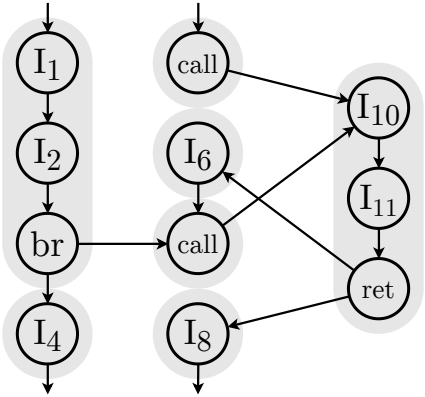
➜ Do not approximate the CFG

✔ **Fine-grained Control Flow Integrity**

Chained Instruction Encryption

Patch policy

How instructions are ordered in the chain?



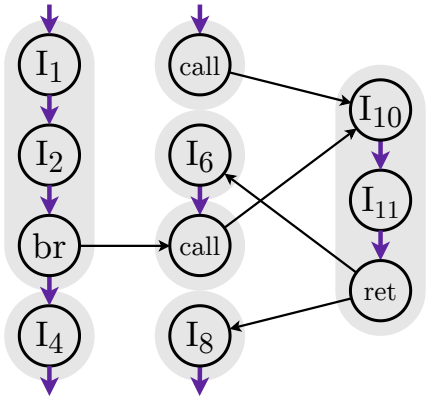
Control Flow Graph

Chained Instruction Encryption

Patch policy

How instructions are ordered in the chain?

"The instructions are encrypted from the state of the previous instruction in memory"



Control Flow Graph

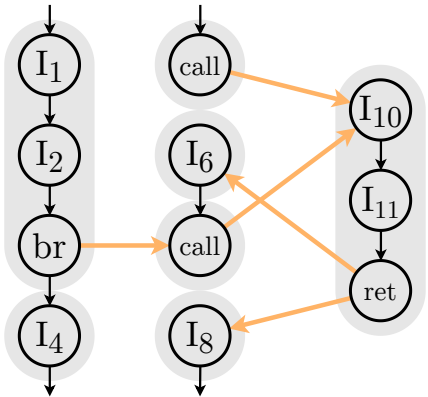
➜ 85.5 % of instructions are sequential (Embench)

Chain sequential instr.

Chained Instruction Encryption

Patch policy

Which CFG edges are patched?



Control Flow Graph

Patches
(for control transfers)

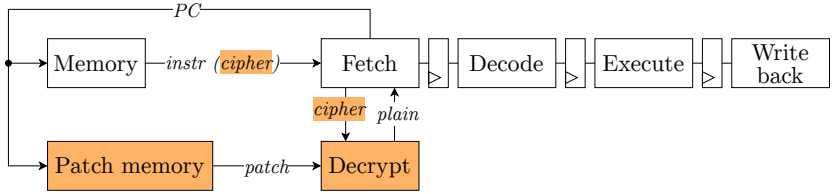
Chained Instruction Encryption

Patch policy

Where patches are stored?

- Patches stored in a dedicated patch memory
- Patches also addressed by the Program Counter
  - ➜ No need for custom instructions

⇒ **zero clock cycle overhead**



Chained Instruction Encryption

Patch policy

How patches are accessed?

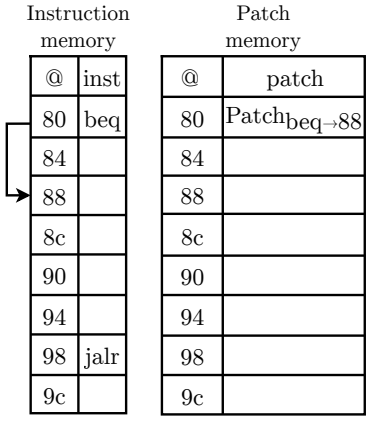
1 destination
⇒ 1 patch

Control transfer instruction	PC
Direct Jump	$@_{jal} = Patch_{jal \rightarrow dest}$
Conditional Branch Taken	$@_{br} = Patch_{br \rightarrow dest}$
Conditional Branch Not Taken	*no need for patch*
Indirect jump	$@_{dest} = Patch_{jalr \rightarrow dest}$

Chained Instruction Encryption

Patch policy

How patches are accessed?

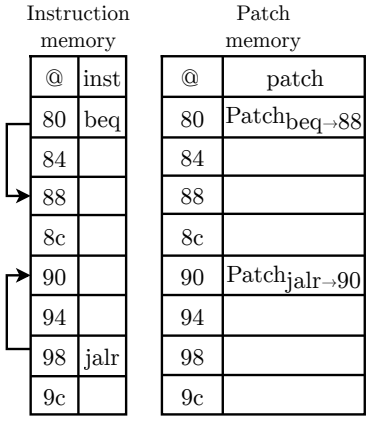


(a) Conflict at address 80

Chained Instruction Encryption

Patch policy

How patches are accessed?

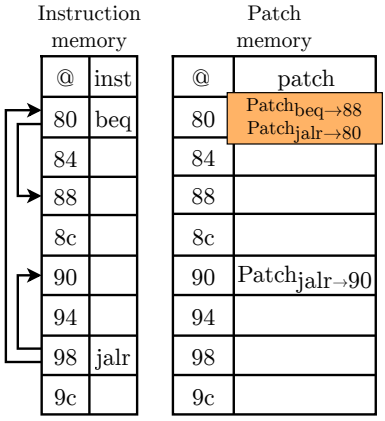


(a) Conflict at address 80

Chained Instruction Encryption

Patch policy

How patches are accessed?

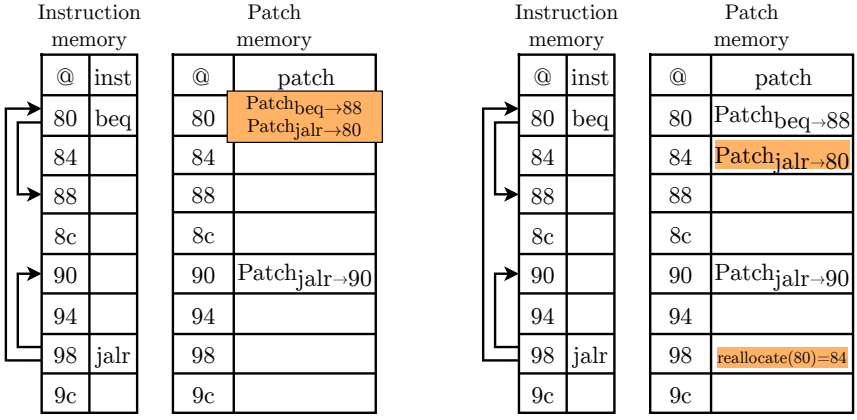


(a) Conflict at address 80

Chained Instruction Encryption

Patch policy

How patches are accessed?



(a) Conflict at address 80

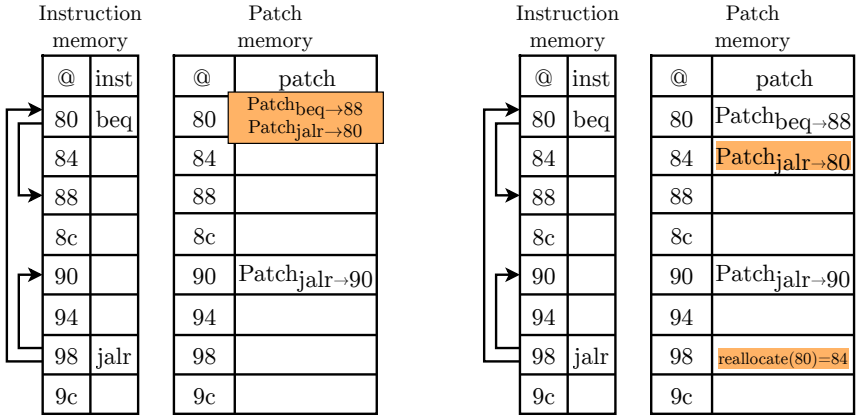
(b) Reallocation for jalr patches

Chained Instruction Encryption

Patch policy

How patches are accessed?

➜ Fine-grained CFI (1 patch/control transfer)



(a) Conflict at address 80

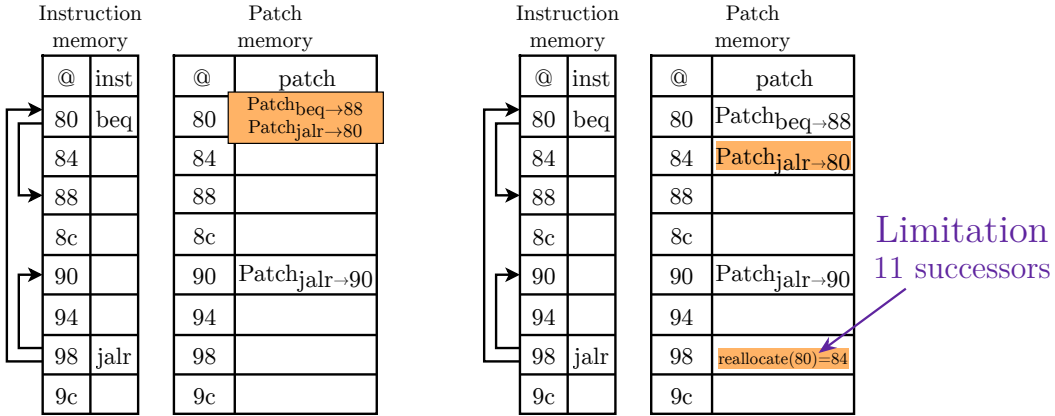
(b) Reallocation for jalr patches

Chained Instruction Encryption

Patch policy

How patches are accessed?

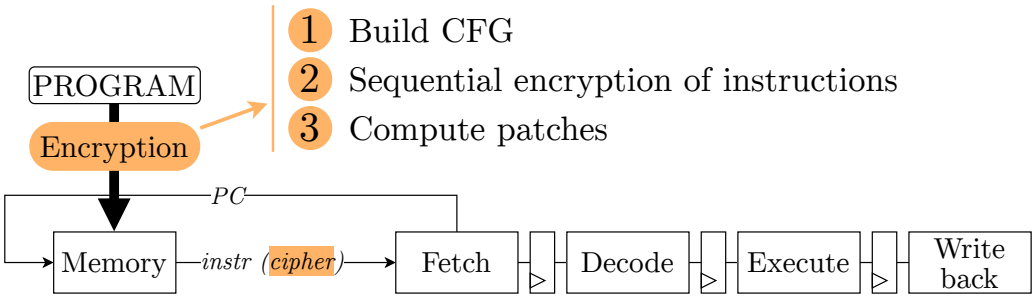
➜ Fine-grained CFI (1 patch/control transfer)



(a) Conflict at address 80

(b) Reallocation for jalr patches

Chained Instruction Encryption
Solution flow



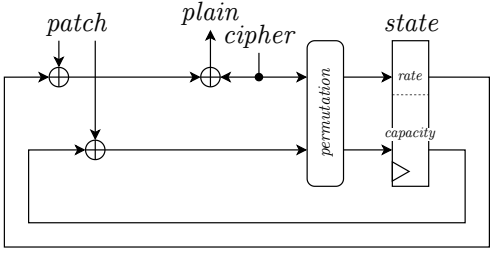
Chained Instruction Encryption
Solution flow

Software encryption

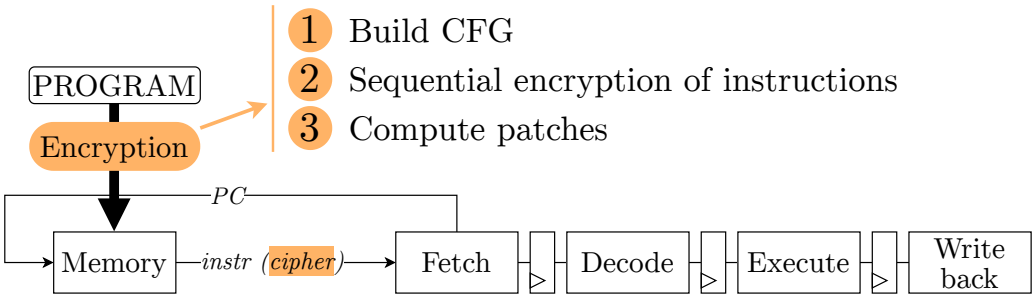
Algorithm 1 Sequential encryption of instructions.

1: state ← initialization(key, nonce)
2: **for** each instruction in code **do**
3: $\quad \text{state}_{\text{rate}} \leftarrow \text{state}_{\text{rate}} \oplus \text{instruction}$
4: $\quad \text{instruction\_cipher} \leftarrow \text{state}_{\text{rate}}$
5: $\quad \text{state} \leftarrow \text{permutation}(\text{state}, 6)$

Hardware decryption

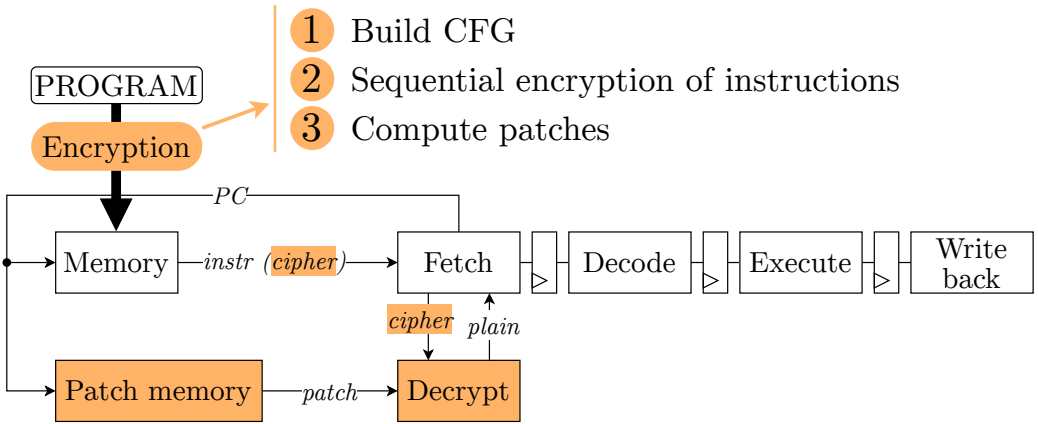


Chained Instruction Encryption
Solution flow



Chained Instruction Encryption

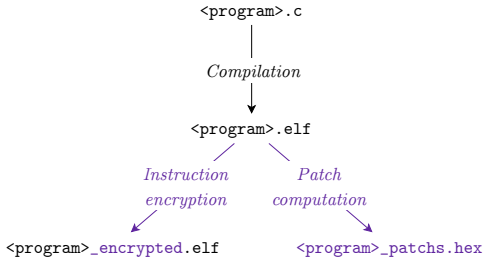
Solution flow



Chained Instruction Encryption

Validation and characterization

core-v-verif-fpga environnement



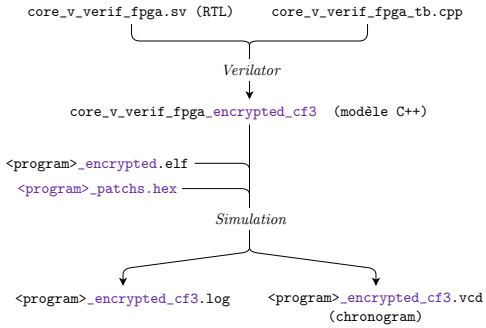
Makefile based:

➜ Compilation (*VerifyPin [18]*, *Embench [19]* - 22 programs) and software encryption flow

Chained Instruction Encryption

Validation and characterization

core-v-verif-fpga environnement



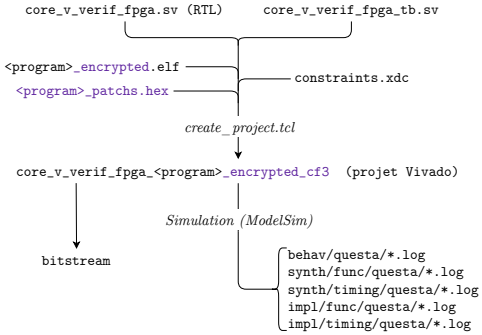
Makefile based:

- ➜ Compilation (*VerifyPin [18]*, *Embench [19]* - 22 programs) and software encryption flow
- ✔ **Cycle-accurate** simulations with assertion of PC and instruction values (*Verilator*)

Chained Instruction Encryption

Validation and characterization

core-v-verif-fpga environnement



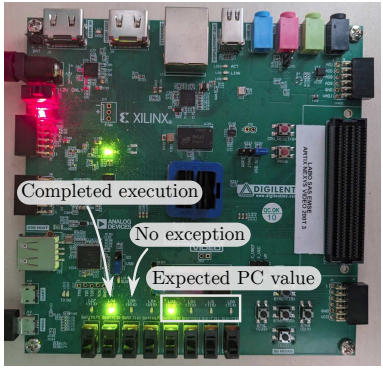
Makefile based:

- ➜ Compilation (*VerifyPin [18]*, *Embench [19]* - 22 programs) and software encryption flow
- ✔ **Cycle-accurate** simulations with assertion of PC and instruction values (*Verilator*)
- ✔ Execution and validation on **FPGA** board (*Nexys Video*, *Vivado 2022.1*)

Chained Instruction Encryption

Validation and characterization

core-v-verif-fpga environnement



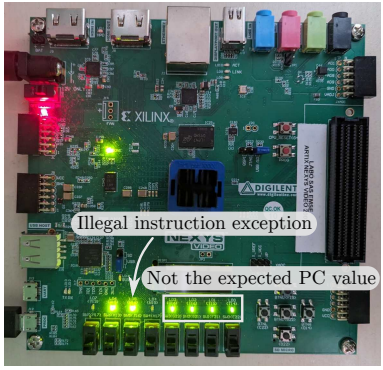
Makefile based:

- ➜ Compilation (*VerifyPin [18]*, *Embench [19]* - 22 programs) and software encryption flow
- ✔ **Cycle-accurate** simulations with assertion of PC and instruction values (*Verilator*)
- ✔ Execution and validation on **FPGA** board (*Nexys Video*, *Vivado 2022.1*)

Chained Instruction Encryption

Validation and characterization

core-v-verif-fpga environnement



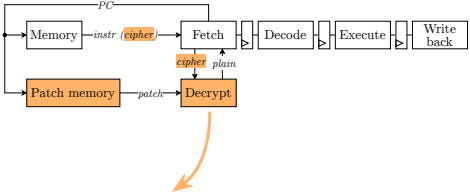
Makefile based:

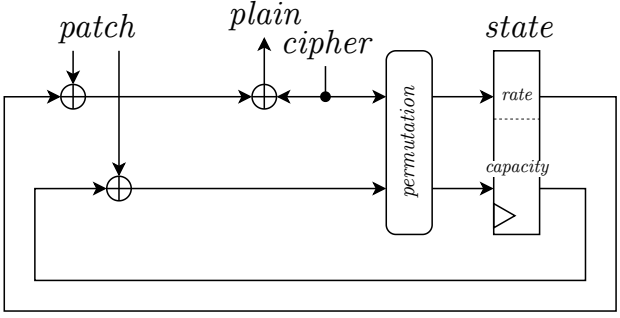
- ➜ Compilation (*VerifyPin [18]*, *Embench [19]* - 22 programs) and software encryption flow
- ✔ **Cycle-accurate** simulations with assertion of PC and instruction values (*Verilator*)
- ✔ Execution and validation on **FPGA** board (*Nexys Video*, *Vivado 2022.1*)
- ✔ **FIA**: emulated on FPGA (instruction memory only) and on simulation (instructions)

Chained Instruction Encryption

Validation and characterization

Cross domain clocking

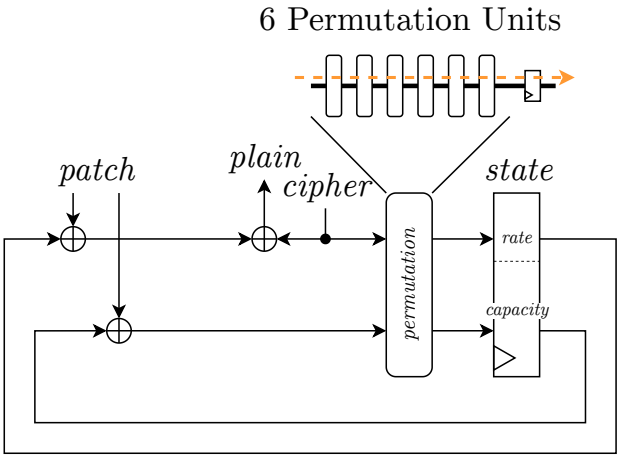




Chained Instruction Encryption

Validation and characterization

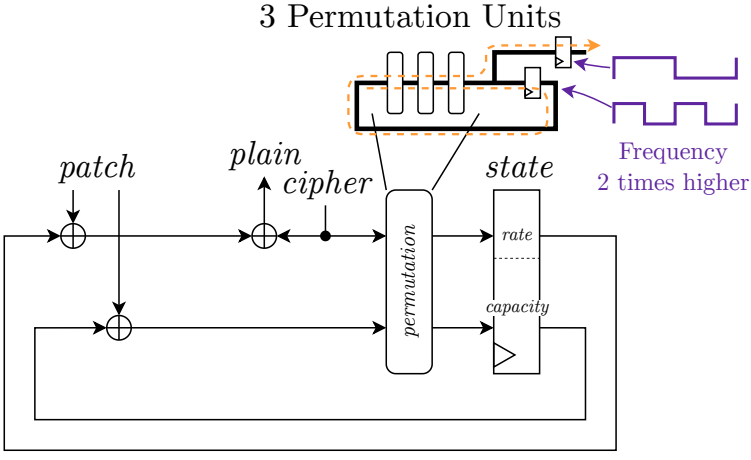
Cross domain clocking



Chained Instruction Encryption

Validation and characterization

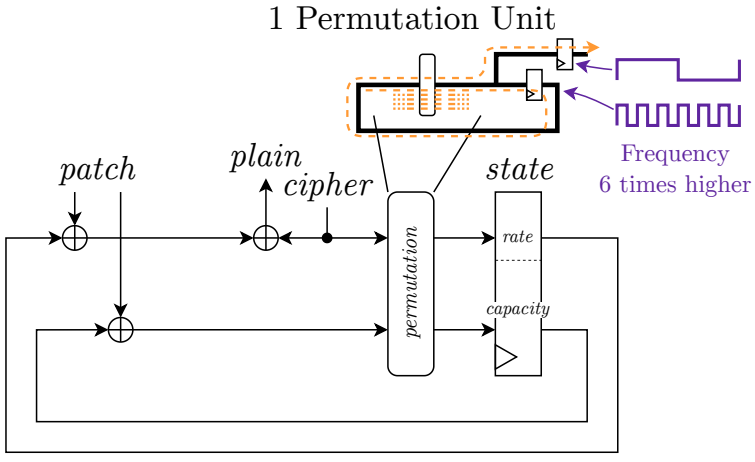
Cross domain clocking



Chained Instruction Encryption

Validation and characterization

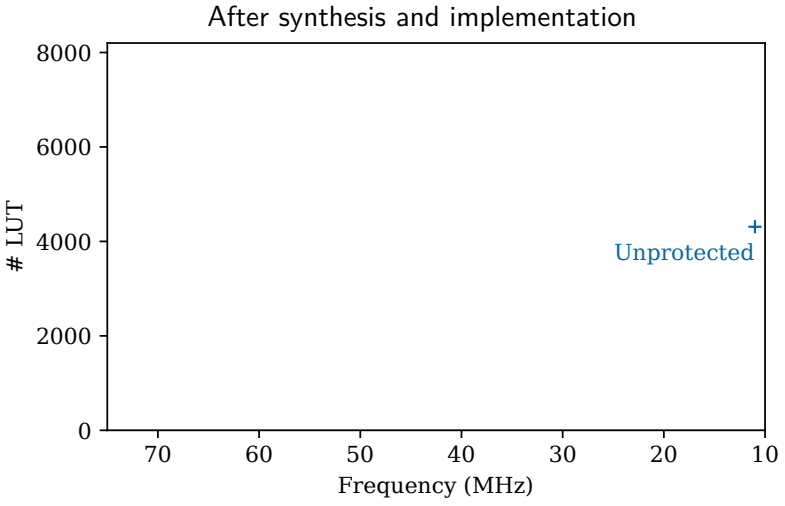
Cross domain clocking



Chained Instruction Encryption

Validation and characterization

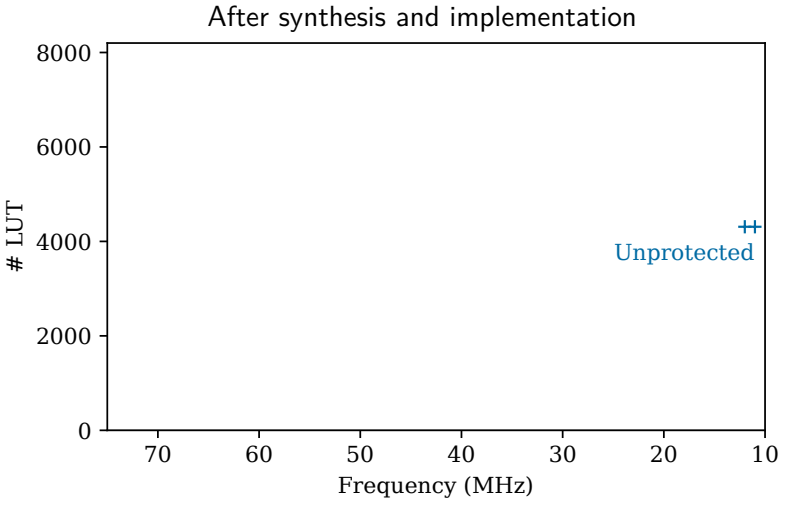
LUT and Frequency



Chained Instruction Encryption

Validation and characterization

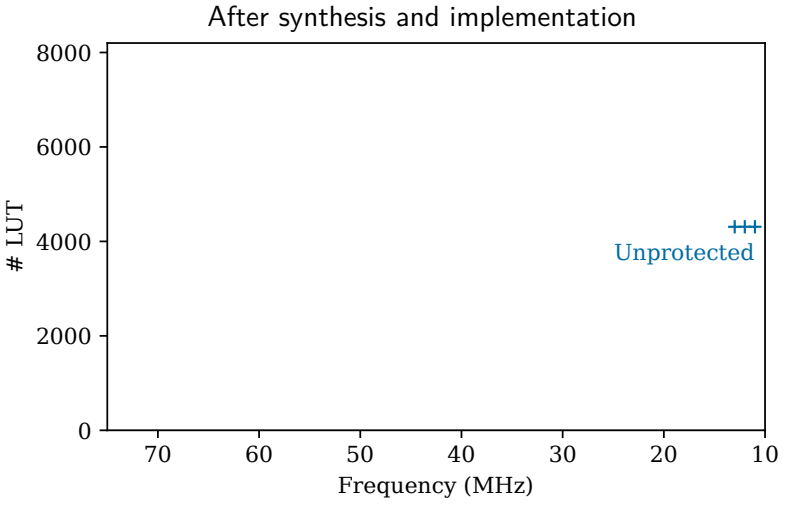
LUT and Frequency



Chained Instruction Encryption

Validation and characterization

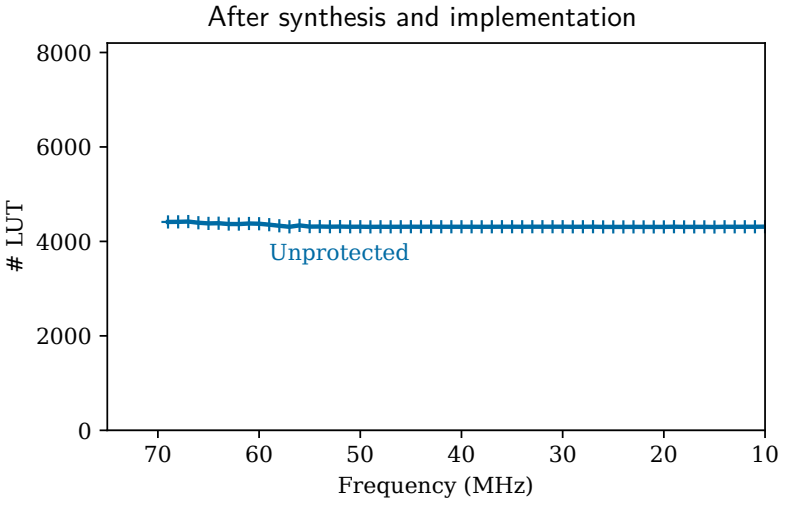
LUT and Frequency



Chained Instruction Encryption

Validation and characterization

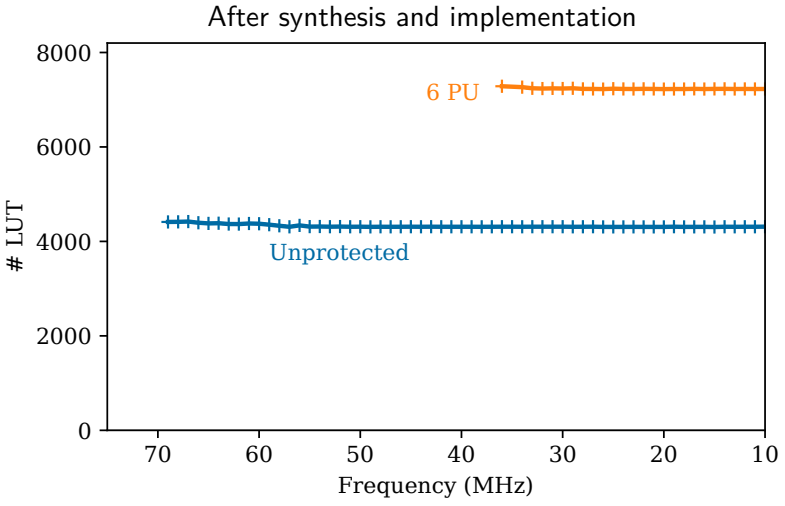
LUT and Frequency



Chained Instruction Encryption

Validation and characterization

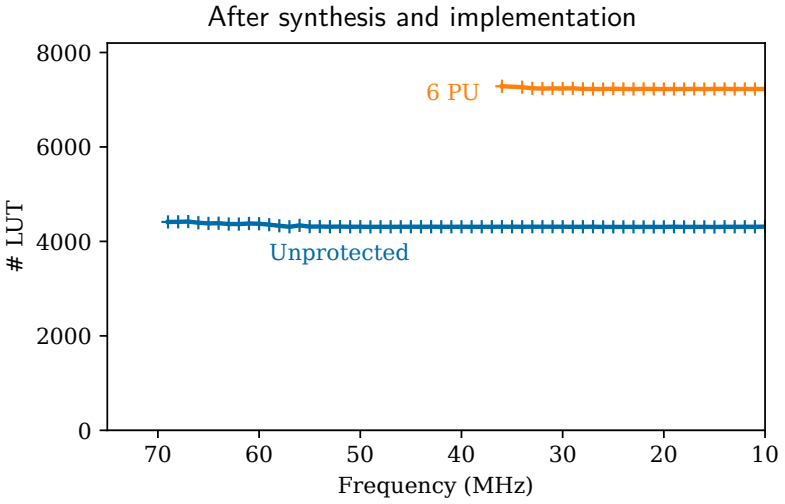
LUT and Frequency



Chained Instruction Encryption

Validation and characterization

LUT and Frequency

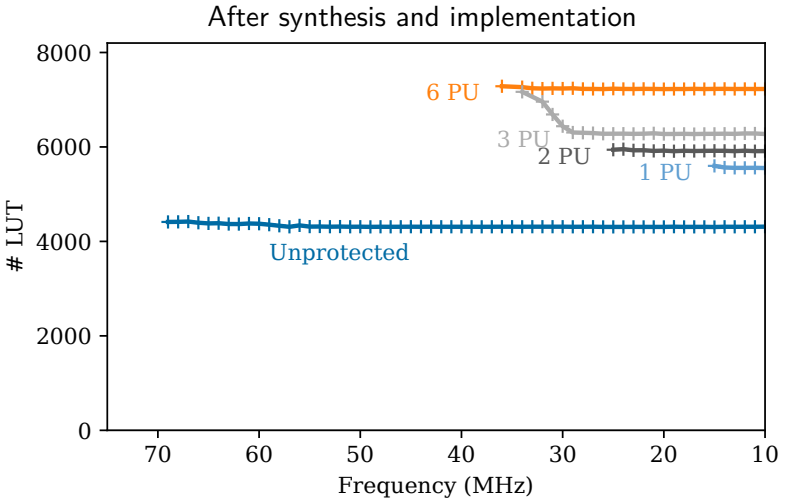


FPGA ≠ ASIC
➜ 70 % net delay

Chained Instruction Encryption

Validation and characterization

LUT and Frequency



FPGA ≠ ASIC
➜ 70 % net delay

Chained Instruction Encryption

Validation and characterization

Memory and Cycles

Name	Memory	Cycles
(Ascon 320bits)	$\times 10$	0%

Chained Instruction Encryption

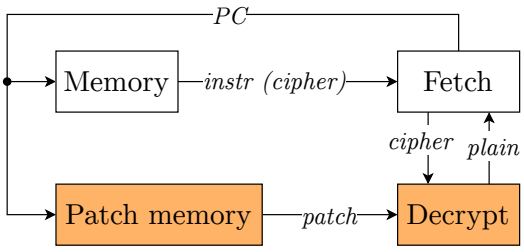
Validation and characterization

Memory and Cycles

Name	Memory	Cycles
(Ascon 320bits)	$\times 10$	0%

✔ **Patch policy ➜ Fine-grained CFI**

➜ 0 clock cycle overhead (COTS binaries)



Chained Instruction Encryption

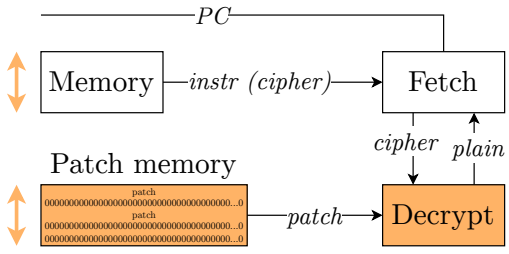
Validation and characterization

Memory and Cycles

Name	Memory	Cycles
(Ascon 320bits)	×10	0%

✔ **Patch policy ➜ Fine-grained CFI**

- ➜ 0 clock cycle overhead (COTS binaries)
- ✖ But, patch memory as high as instr. mem.
- ✖ But, empty at 75 %



Chained Instruction Encryption

Validation and characterization

Memory and Cycles

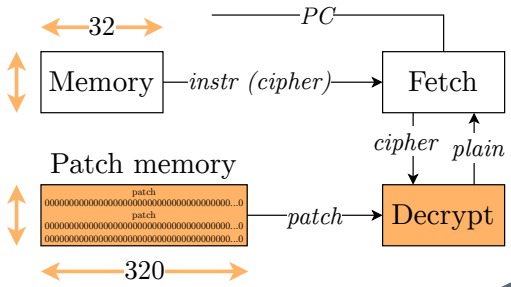
Name	Memory	Cycles
(Ascon 320bits)	×10	0%

✔ **Patch policy ➜ Fine-grained CFI**

- ➜ 0 clock cycle overhead (COTS binaries)
- ✖ But, patch memory as high as instr. mem.
- ✖ But, empty at 75 %

✔ **Ascon ➜** New standard for lightweight crypto.

- ✖ But, patches are 320 bits wide



Chained Instruction Encryption

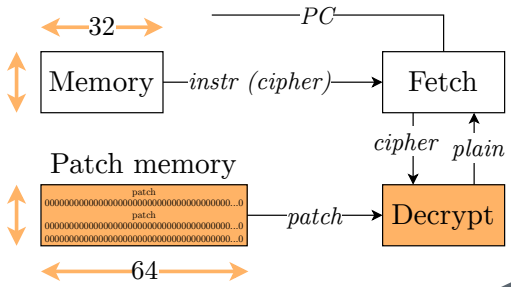
Validation and characterization

Memory and Cycles

Name	Memory	Cycles
(Ascon 320bits)	×10	0%
(Prince 64bits)	×2	0%

✔ **Patch policy ➜ Fine-grained CFI**
- ➜ 0 clock cycle overhead (COTS binaries)
- ✖ But, patch memory as high as instr. mem.
- ✖ But, empty at 75 %

✔ **Ascon ➜** New standard for lightweight crypto.
- ✖ But, patches are 320 bits wide
- ➜ Consider a 64 bits state (e.g., Prince)



Chained Instruction Encryption

Validation and characterization

State-of-the-art overhead

Name	Memory (%)	Cycles (%)	Frequency (%)	Utilization (%)
Instr. Enc.	2−38	129−293*	0	26 ALM
Confidaent	116−218*	151−276*		
Sofia	110−437	36−438	-23	12 LUT
Ccfi-Cache	118−160	2−63		11 LUT 9 FF
Cifer	16−68	0	0	35−55 Slice
SecDec	1−2	1−2		4−17 GE*
Hapei	123−507*			
Scfp	15−26	4−15		47 GE
Gpsa-Csm	88−118*	2−71*		6 GE
Mafia	7−55	3−44		7−24 GE
This work (Ascon 320bits)	1000	0	-39 − -75	29−67 LUT 64−80 FF

Chained Instruction Encryption

Pseudo-random instruction generation

FIA ➜ Incorrect state ➜ Decryption = random instruction ➜ Illegal instruction

Chained Instruction Encryption

Pseudo-random instruction generation

FIA ➜ Incorrect state ➜ Decryption = random instruction ➜ Illegal instruction

Original code is not executed anymore

Detection

Chained Instruction Encryption

Pseudo-random instruction generation

FIA ➜ Incorrect state ➜ Decryption = random instruction ➜ Illegal instruction

Original code is not executed anymore

Detection

✔ Rate of legal instructions (RV32IM): 4.6 %

Chained Instruction Encryption

Pseudo-random instruction generation

FIA ➜ Incorrect state ➜ Decryption = random instruction ➜ Illegal instruction

Original code is not executed anymore

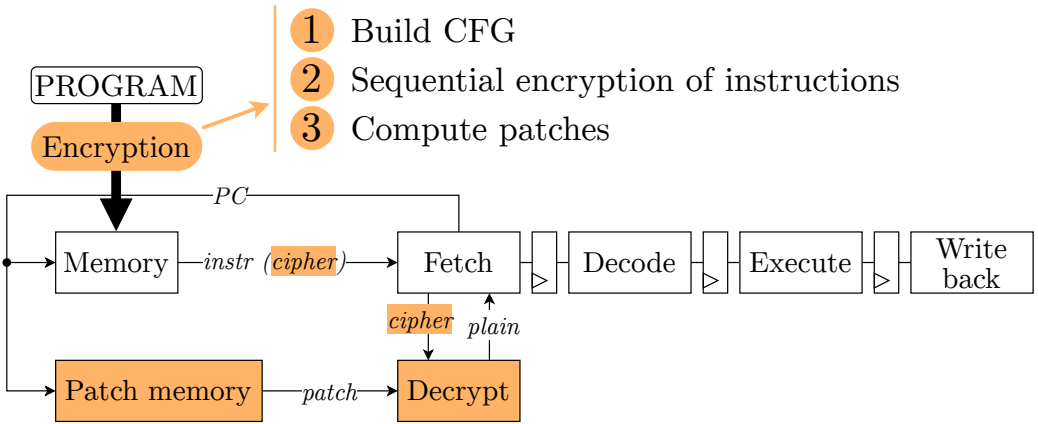
Detection

✔ Rate of legal instructions (RV32IM): 4.6 %

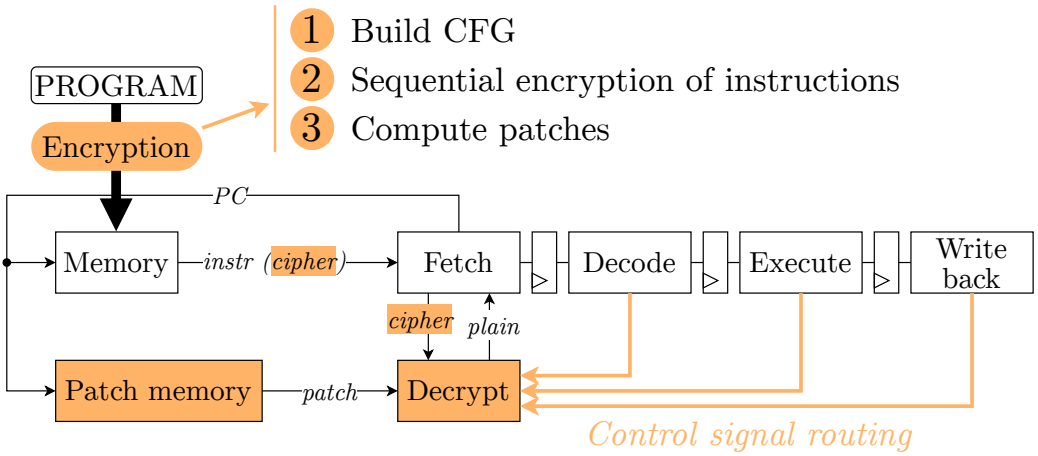
➜ Probability of not being detected after 3 pseudo-random instructions < 0.01 %

Absorption of Control Signals

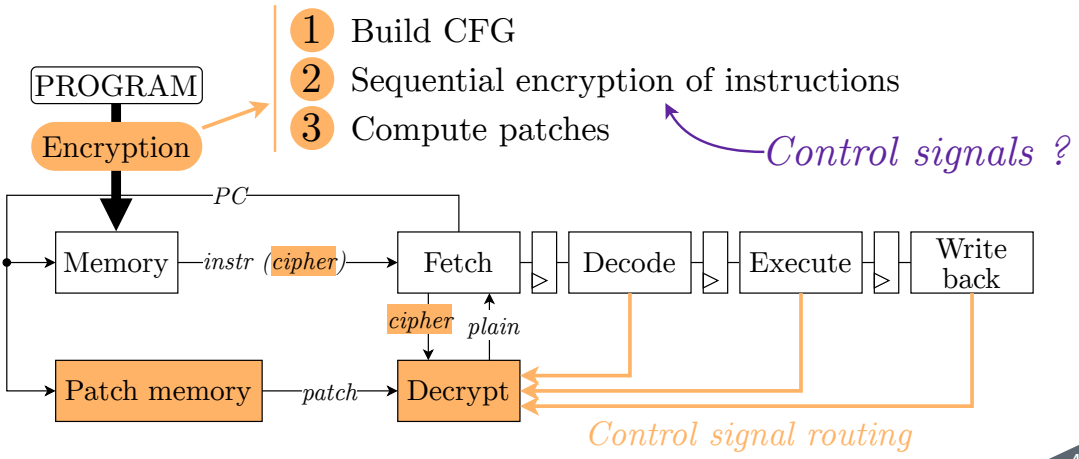
Absorption of Control Signals
Solution flow



Absorption of Control Signals
Solution flow



Absorption of Control Signals
Solution flow



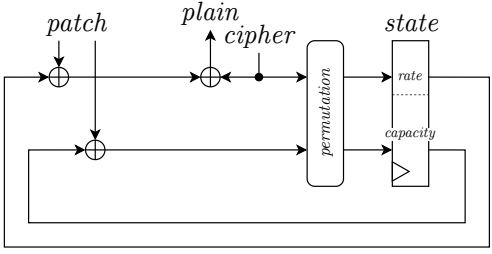
Absorption of Control Signals
Solution flow

Software encryption

Algorithm 1 Sequential encryption of instructions.

1: state ← initialization(key, nonce)
2: **for** each instruction in code **do**
3: $\quad \text{state}_{\text{rate}} \leftarrow \text{state}_{\text{rate}} \oplus \text{instruction}$
4: $\quad \text{instruction\_cipher} \leftarrow \text{state}_{\text{rate}}$
5: $\quad \text{state} \leftarrow \text{permutation}(\text{state}, 6)$

Hardware decryption



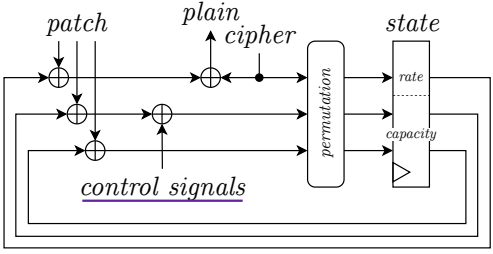
Absorption of Control Signals
Solution flow

Software encryption

Algorithm 1 Sequential encryption of instructions.

1: state ← initialization(key, nonce)
2: **for** each instruction in code **do**
3: $\quad \text{state}_{\text{rate}} \leftarrow \text{state}_{\text{rate}} \oplus \text{instruction}$
4: $\quad \text{instruction\_cipher} \leftarrow \text{state}_{\text{rate}}$
5: $\quad \text{state} \leftarrow \text{permutation}(\text{state}, 6)$

Hardware decryption



Control signal?

Absorption of Control Signals

Software model to infer control signal values

- ✔ Instructions (machine code)
- ✔ Microarchitecture netlist

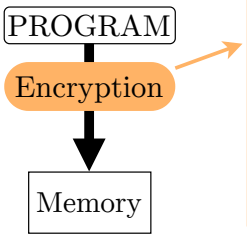
➜ Infer control signal values

Absorption of Control Signals

Software model to infer control signal values

- ✔ Instructions (machine code)
- ✔ Microarchitecture netlist

➜ Infer control signal values

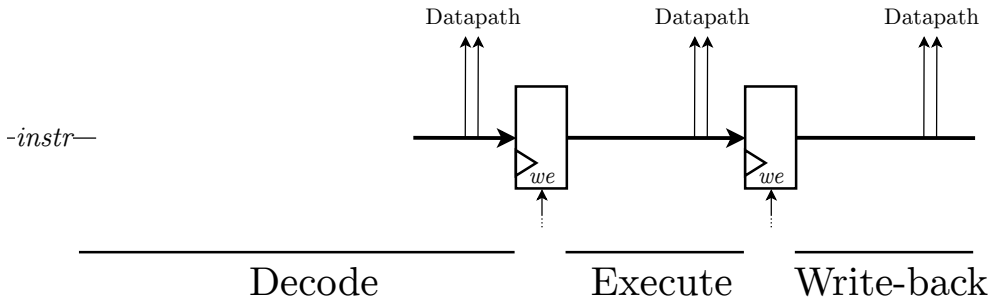


1. Analyze control signals
2. Build CFG
3. Sequential encryption of instructions
   + modelling of control signal propagation
4. Compute patches
   + adaptation

Absorption of Control Signals

Software model to infer control signal values

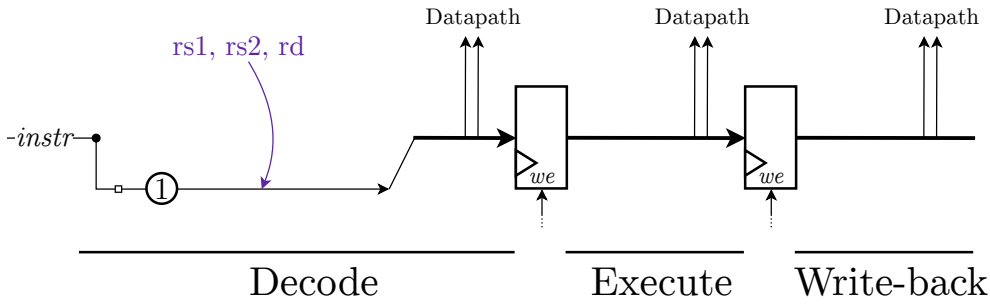
Control signals taxonomy



Absorption of Control Signals

Software model to infer control signal values

Control signals taxonomy

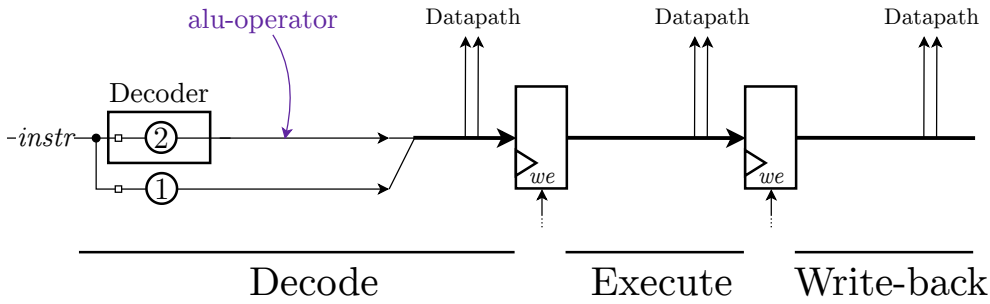


① Extracted

Absorption of Control Signals

Software model to infer control signal values

Control signals taxonomy

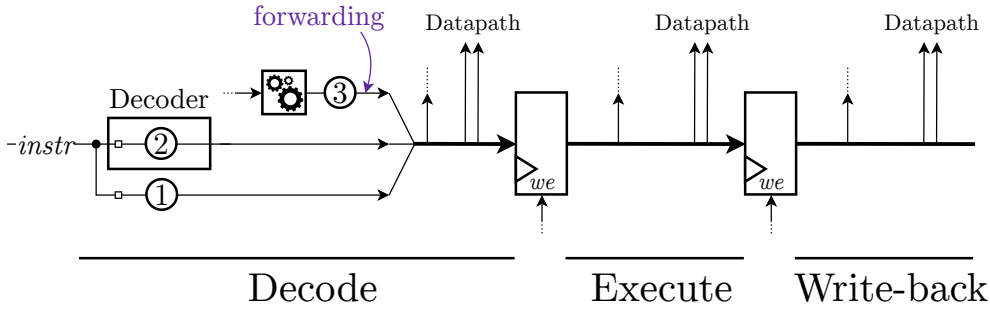


① Extracted ② Decoded

Absorption of Control Signals

Software model to infer control signal values

Control signals taxonomy

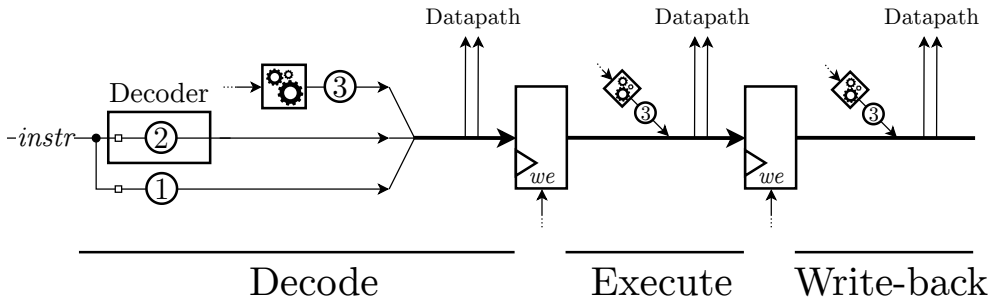


① Extracted ② Decoded ③ Combinational

Absorption of Control Signals

Software model to infer control signal values

Control signals taxonomy

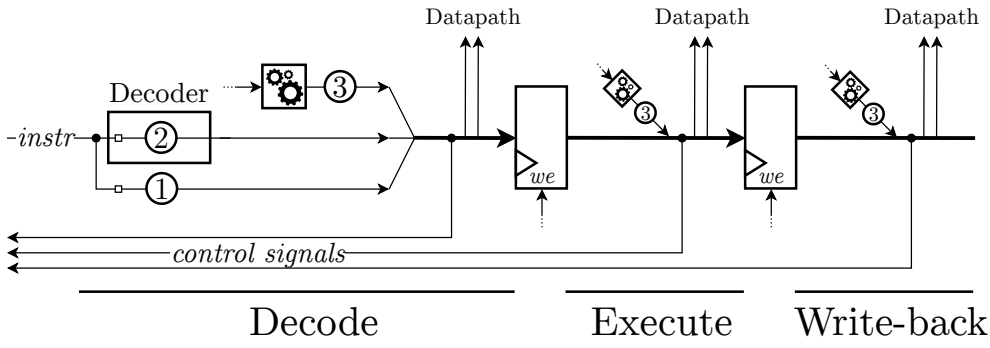


① Extracted ② Decoded ③ Combinational

Absorption of Control Signals

Software model to infer control signal values

Control signals taxonomy

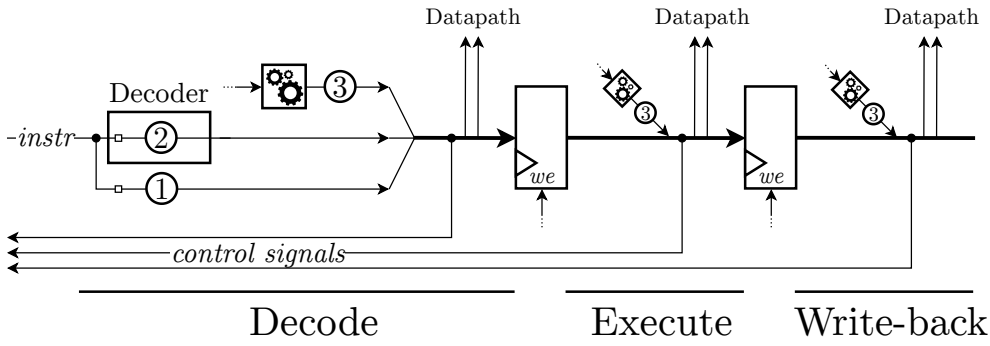


① Extracted ② Decoded ③ Combinational

Absorption of Control Signals

Software model to infer control signal values

Control signals taxonomy



① Extracted ② Decoded ③ Combinational ④ Combinational (with data)

Absorption of Control Signals

Software model to infer control signal values

Analyze 41 control signals of the CV32E40P

- Type (①, ②, ③)
- Stages
- Width
- Default value in every stage
- Reset value
- Condition of propagation

Absorption of Control Signals

Software model to infer control signal values

Analyze 41 control signals of the CV32E40P

- Type (①, ②, ③)
- Stages
- Width
- Default value in every stage
- Reset value
- Condition of propagation

➜ Combinational function for Extracted signals ①

➜ Combinational function for Combinational signals ③

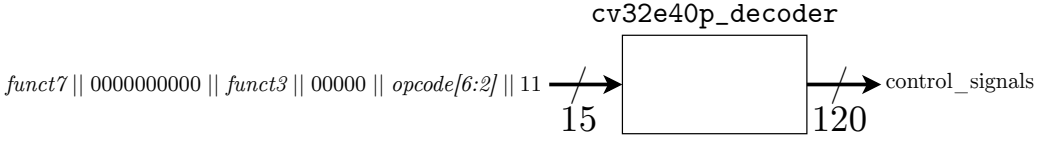
Absorption of Control Signals

Software model to infer control signal values

Analyze 41 control signals of the CV32E40P

- Type (①, ②, ③)
- Stages
- Width
- Default value in every stage
- Reset value
- Condition of propagation

➜ Combinational function for Extracted signals ①
➜ Combinational function for Combinational signals ③
➜ Position in the "decoder LookUpTable" for Decoded signals ②



Absorption of Control Signals

Software model to infer control signal values

Analyze 41 control signals of the CV32E40P

- Type (①, ②, ③)
- Stages
- Width
- Default value in every stage
- Reset value
- Condition of propagation

➜ Combinational function for Extracted signals ①
➜ Combinational function for Combinational signals ③
➜ Position in the "decoder LookUpTable" for Decoded signals ②

Origin	Decode	Execute	Write-back	examples
Extracted①	2 (12 bits)	1 (6 bits)	1 (6 bits)	$rs1$, $rs2$, rd
Decoded②	13 (24 bits)	8 (17 bits)	4 (6 bits)	$alu\_operator$
Combinational③	2 (64 bits)	7 (7 bits)	3 (3 bits)	$forwarding, imm$

Absorption of Control Signals

Software model to infer control signal values

During sequential encryption of instructions

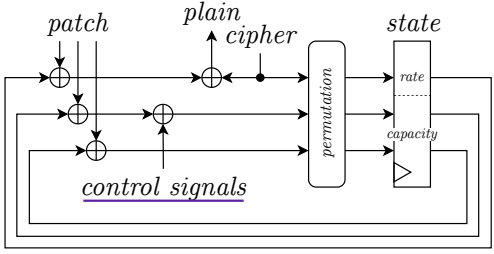
Software encryption

Algorithm 1 Sequential encryption of instructions.

```
1: state ← initialization(key, nonce)
2: for each instruction in code do
3:     state_rate ← state_rate ⊕ instruction
4:     instruction_cipher ← state_rate
5:     state ← permutation(state, 6)
```

Control signal?

Hardware decryption



Absorption of Control Signals

Software model to infer control signal values

During sequential encryption of instructions

Software encryption

Algorithm 1 Sequential encryption of instructions and propagation of control signals.

1: state ← initialization(key, nonce)
2: $CS_{modeled}[if, id, ex, wb] \leftarrow CS_{reset}$
3: **for** each instruction in code **do**
4: ▷ *Transmission between stages*
5: $CS_{modeled}[wb] \leftarrow CS_{modeled}[ex]$ / $CS_{modeled}[id]$ / $CS_{reset}(wb)$ ▷ *stall? multi-cycle?*
6: $CS_{modeled}[ex] \leftarrow CS_{modeled}[id]$ / $CS_{modeled}[if]$ / $CS_{reset}(ex)$ ▷ *stall? multi-cycle?*
7: $CS_{modeled}[id] \leftarrow CS_{modeled}[if]$ ▷ *multi-cycle?*
8:
9: ▷ *Evaluation*
10: $CS_{modeled}[if] \leftarrow$ Function(instruction) ▷ *Extracted signals ①*
11: $CS_{modeled}[if] \leftarrow$ LookUp Table(instruction) ▷ *Decoded signals ②*
12: $CS_{modeled}[id, ex, wb] \leftarrow$ Function($CS_{modeled}$) ▷ *Combinational signals ③*
13:
14: ▷ *Encryption and absorption of signals*
15: $CS \leftarrow selection(CS_{modeled})$
16: state ← state $\oplus$ $0^* || CS || 0^*$ ← *Control signal absorption*
17: $state_{rate} \leftarrow state_{rate} \oplus$ instruction
18: instruction_cipher ← $state_{rate}$
19: state ← permutation(state, 6)

Absorption of Control Signals

Software model to infer control signal values

During sequential encryption of instructions

Algorithm 1 Sequential encryption of instructions and propagation of control signals.

1: state ← initialization(key, nonce)
2: $CS_{modeled}[if, id, ex, wb] \leftarrow CS_{reset}$
3: **for** each instruction in code **do**
4: ▷ *Transmission between stages*
5: $CS_{modeled}[wb] \leftarrow CS_{modeled}[ex]$ / $CS_{modeled}[id]$ / $CS_{reset}(wb)$
6: $CS_{modeled}[ex] \leftarrow CS_{modeled}[id]$ / $CS_{modeled}[if]$ / $CS_{reset}(ex)$
7: $CS_{modeled}[id] \leftarrow CS_{modeled}[if]$
8:
9: ▷ *Evaluation*
10: $CS_{modeled}[if] \leftarrow$ Function(instruction)
11: $CS_{modeled}[if] \leftarrow$ LookUp Table(instruction)
12: $CS_{modeled}[id, ex, wb] \leftarrow$ Function($CS_{modeled}$)
13:
14: ▷ *Encryption and absorption of signals*
15: $CS \leftarrow selection(CS_{modeled})$
16: state ← state $\oplus$ $0^* \,||\, CS \,||\, 0^*$
17: $\text{state}_{\text{rate}} \leftarrow \text{state}_{\text{rate}} \oplus$ instruction
18: instruction_cipher ← $\text{state}_{\text{rate}}$
19: state ← permutation(state, 6)

IF	ID	EX	WB
828	18	03	
82C			
830			
834			
@	alu_operator		

Absorption of Control Signals

Software model to infer control signal values

During sequential encryption of instructions

Algorithm 1 Sequential encryption of instructions and propagation of control signals.

1: state ← initialization(key, nonce)
2: $CS_{modeled}[if, id, ex, wb] \leftarrow CS_{reset}$
(82C) 3: **for** each instruction in code **do**
4: ▷ *Transmission between stages*
5: $CS_{modeled}[wb] \leftarrow CS_{modeled}[ex]$ / $CS_{modeled}[id]$ / $CS_{reset}(wb)$
6: $CS_{modeled}[ex] \leftarrow CS_{modeled}[id]$ / $CS_{modeled}[if]$ / $CS_{reset}(ex)$
7: $CS_{modeled}[id] \leftarrow CS_{modeled}[if]$
8:
9: ▷ *Evaluation*
10: $CS_{modeled}[if] \leftarrow$ Function(instruction)
11: $CS_{modeled}[if] \leftarrow$ LookUp Table(instruction)
12: $CS_{modeled}[id, ex, wb] \leftarrow$ Function($CS_{modeled}$)
13:
14: ▷ *Encryption and absorption of signals*
15: $CS \leftarrow selection(CS_{modeled})$
16: state ← state $\oplus$ $0^* \,||\, CS \,||\, 0^*$
17: $\text{state}_{\text{rate}} \leftarrow \text{state}_{\text{rate}} \oplus$ instruction
18: instruction_cipher ← $\text{state}_{\text{rate}}$
19: state ← permutation(state, 6)

IF	ID	EX	WB
828	18	03	
82C			
830			
834			
@	alu_operator		

Absorption of Control Signals

Software model to infer control signal values

During sequential encryption of instructions

Algorithm 1 Sequential encryption of instructions and propagation of control signals.

1: state ← initialization(key, nonce)
2: $CS_{modeled}[if, id, ex, wb] \leftarrow CS_{reset}$
(82C) 3: **for** each instruction in code **do**
4: ▷ *Transmission between stages*
➔ 5: $CS_{modeled}[wb] \leftarrow CS_{modeled}[ex]$ / $CS_{modeled}[id]$ / $CS_{reset}(wb)$
➔ 6: $CS_{modeled}[ex] \leftarrow CS_{modeled}[id]$ / $CS_{modeled}[if]$ / $CS_{reset}(ex)$
➔ 7: $CS_{modeled}[id] \leftarrow CS_{modeled}[if]$
8:
9: ▷ *Evaluation*
10: $CS_{modeled}[if] \leftarrow$ Function(instruction)
11: $CS_{modeled}[if] \leftarrow$ LookUp Table(instruction)
12: $CS_{modeled}[id, ex, wb] \leftarrow$ Function($CS_{modeled}$)
13:
14: ▷ *Encryption and absorption of signals*
15: $CS \leftarrow selection(CS_{modeled})$
16: state ← state $\oplus$ $0^* \,||\, CS \,||\, 0^*$
17: $\text{state}_{\text{rate}} \leftarrow \text{state}_{\text{rate}} \oplus$ instruction
18: instruction_cipher ← $\text{state}_{\text{rate}}$
19: state ← permutation(state, 6)

IF	ID	EX	WB
828	18	03	
82C		18	
830			
834			
@	alu_operator		

Absorption of Control Signals

Software model to infer control signal values

During sequential encryption of instructions

Algorithm 1 Sequential encryption of instructions and propagation of control signals.

1: state ← initialization(key, nonce)
2: $CS_{modeled}[if, id, ex, wb] \leftarrow CS_{reset}$
(82C) 3: **for** each instruction in code **do**
4: ▷ *Transmission between stages*
5: $CS_{modeled}[wb] \leftarrow CS_{modeled}[ex]$ / $CS_{modeled}[id]$ / $CS_{reset}(wb)$
6: $CS_{modeled}[ex] \leftarrow CS_{modeled}[id]$ / $CS_{modeled}[if]$ / $CS_{reset}(ex)$
7: $CS_{modeled}[id] \leftarrow CS_{modeled}[if]$
8:
9: ▷ *Evaluation*
➜ 10: $CS_{modeled}[if] \leftarrow$ Function(instruction)
➜ 11: $CS_{modeled}[if] \leftarrow$ LookUp Table(instruction)
➜ 12: $CS_{modeled}[id, ex, wb] \leftarrow$ Function($CS_{modeled}$)
13:
14: ▷ *Encryption and absorption of signals*
15: $CS \leftarrow selection(CS_{modeled})$
16: state ← state $\oplus$ $0^* \,||\, CS \,||\, 0^*$
17: $state_{rate} \leftarrow state_{rate} \oplus$ instruction
18: instruction_cipher ← $state_{rate}$
19: state ← permutation(state, 6)

IF	ID	EX	WB
828	18	03	
82C	0C	18	
830			
834			
@	alu_operator		

Absorption of Control Signals

Software model to infer control signal values

During sequential encryption of instructions

Algorithm 1 Sequential encryption of instructions and propagation of control signals.

1: state ← initialization(key, nonce)
2: $CS_{modeled}[if, id, ex, wb] \leftarrow CS_{reset}$
(82C) 3: **for** each instruction in code **do**
4: ▷ *Transmission between stages*
5: $CS_{modeled}[wb] \leftarrow CS_{modeled}[ex]$ / $CS_{modeled}[id]$ / $CS_{reset}(wb)$
6: $CS_{modeled}[ex] \leftarrow CS_{modeled}[id]$ / $CS_{modeled}[if]$ / $CS_{reset}(ex)$
7: $CS_{modeled}[id] \leftarrow CS_{modeled}[if]$
8:
9: ▷ *Evaluation*
10: $CS_{modeled}[if] \leftarrow$ Function(instruction)
11: $CS_{modeled}[if] \leftarrow$ LookUp Table(instruction)
12: $CS_{modeled}[id, ex, wb] \leftarrow$ Function($CS_{modeled}$)
13:
14: ▷ *Encryption and absorption of signals*
➔ 15: $CS \leftarrow selection(CS_{modeled})$
➔ 16: state ← state $\oplus$ $0^* \,||\, CS \,||\, 0^*$
17: $state_{rate} \leftarrow state_{rate} \oplus$ instruction
18: instruction_cipher ← $state_{rate}$
19: state ← permutation(state, 6)

IF	ID	EX	WB
828	18	03	
82C	0C	18	
830			
834			
@	alu_operator		

Control signal absorption

Absorption of Control Signals

Software model to infer control signal values

During sequential encryption of instructions

Algorithm 1 Sequential encryption of instructions and propagation of control signals.

1: state ← initialization(key, nonce)
2: $CS_{modeled}[if, id, ex, wb] \leftarrow CS_{reset}$
(830) 3: **for** each instruction in code **do**
4: ▷ *Transmission between stages*
➔ 5: $CS_{modeled}[wb] \leftarrow CS_{modeled}[ex]$ / $CS_{modeled}[id]$ / $CS_{reset}(wb)$
➔ 6: $CS_{modeled}[ex] \leftarrow CS_{modeled}[id]$ / $CS_{modeled}[if]$ / $CS_{reset}(ex)$
➔ 7: $CS_{modeled}[id] \leftarrow CS_{modeled}[if]$
8:
9: ▷ *Evaluation*
10: $CS_{modeled}[if] \leftarrow$ Function(instruction)
11: $CS_{modeled}[if] \leftarrow$ LookUp Table(instruction)
12: $CS_{modeled}[id, ex, wb] \leftarrow$ Function($CS_{modeled}$)
13:
14: ▷ *Encryption and absorption of signals*
15: $CS \leftarrow selection(CS_{modeled})$
16: state ← state $\oplus$ $0^* \,||\, CS \,||\, 0^*$
17: $\text{state}_{\text{rate}} \leftarrow \text{state}_{\text{rate}} \oplus$ instruction
18: instruction_cipher ← $\text{state}_{\text{rate}}$
19: state ← permutation(state, 6)

IF	ID	EX	WB
828	18	03	
82C	0C	18	
830		0C	
834			
@	alu_operator		

Absorption of Control Signals

Software model to infer control signal values

During sequential encryption of instructions

Algorithm 1 Sequential encryption of instructions and propagation of control signals.

1: state ← initialization(key, nonce)
2: $CS_{modeled}[if, id, ex, wb] \leftarrow CS_{reset}$
(830) 3: **for** each instruction in code **do**
4: ▷ *Transmission between stages*
5: $CS_{modeled}[wb] \leftarrow CS_{modeled}[ex]$ / $CS_{modeled}[id]$ / $CS_{reset}(wb)$
6: $CS_{modeled}[ex] \leftarrow CS_{modeled}[id]$ / $CS_{modeled}[if]$ / $CS_{reset}(ex)$
7: $CS_{modeled}[id] \leftarrow CS_{modeled}[if]$
8:
9: ▷ *Evaluation*
➔ 10: $CS_{modeled}[if] \leftarrow$ Function(instruction)
➔ 11: $CS_{modeled}[if] \leftarrow$ LookUp Table(instruction)
➔ 12: $CS_{modeled}[id, ex, wb] \leftarrow$ Function($CS_{modeled}$)
13:
14: ▷ *Encryption and absorption of signals*
15: $CS \leftarrow selection(CS_{modeled})$
16: state ← state $\oplus$ $0^* \,||\, CS \,||\, 0^*$
17: $\text{state}_{\text{rate}} \leftarrow \text{state}_{\text{rate}} \oplus$ instruction
18: instruction_cipher ← $\text{state}_{\text{rate}}$
19: state ← permutation(state, 6)

IF	ID	EX	WB
828	18	03	
82C	0C	18	
830	18	0C	
834			
@	alu_operator		

Absorption of Control Signals

Software model to infer control signal values

During sequential encryption of instructions

Algorithm 1 Sequential encryption of instructions and propagation of control signals.

1: state ← initialization(key, nonce)
2: $CS_{modeled}[if, id, ex, wb] \leftarrow CS_{reset}$
(830) 3: **for** each instruction in code **do**
4: ▷ *Transmission between stages*
5: $CS_{modeled}[wb] \leftarrow CS_{modeled}[ex]$ / $CS_{modeled}[id]$ / $CS_{reset}(wb)$
6: $CS_{modeled}[ex] \leftarrow CS_{modeled}[id]$ / $CS_{modeled}[if]$ / $CS_{reset}(ex)$
7: $CS_{modeled}[id] \leftarrow CS_{modeled}[if]$
8:
9: ▷ *Evaluation*
10: $CS_{modeled}[if] \leftarrow$ Function(instruction)
11: $CS_{modeled}[if] \leftarrow$ LookUp Table(instruction)
12: $CS_{modeled}[id, ex, wb] \leftarrow$ Function($CS_{modeled}$)
13:
14: ▷ *Encryption and absorption of signals*
➔ 15: $CS \leftarrow selection(CS_{modeled})$
➔ 16: state ← state $\oplus$ $0^* \,||\, CS \,||\, 0^*$
17: $state_{rate} \leftarrow state_{rate} \oplus$ instruction
18: instruction_cipher ← $state_{rate}$
19: state ← permutation(state, 6)

IF	ID	EX	WB
828	18	03	
82C	0C	18	
830	18	0C	
834			
@	alu_operator		

Control signal absorption

Absorption of Control Signals

Software model to infer control signal values

During sequential encryption of instructions

Algorithm 1 Sequential encryption of instructions and propagation of control signals.

1: state ← initialization(key, nonce)
2: $CS_{modeled}[if, id, ex, wb] \leftarrow CS_{reset}$
3: **for** each instruction in code **do**
4: ▷ *Transmission between stages*
5: $CS_{modeled}[wb] \leftarrow CS_{modeled}[ex]$ / $CS_{modeled}[id]$ / $CS_{reset}(wb)$ ▷ *stall? multi-cycle?*
6: $CS_{modeled}[ex] \leftarrow CS_{modeled}[id]$ / $CS_{modeled}[if]$ / $CS_{reset}(ex)$ ▷ *stall? multi-cycle?*
7: $CS_{modeled}[id] \leftarrow CS_{modeled}[if]$ ▷ *multi-cycle?*
8:
9: ▷ *Evaluation*
10: $CS_{modeled}[if] \leftarrow$ Function(instruction) ▷ *Extracted signals* ①
11: $CS_{modeled}[if] \leftarrow$ LookUp Table(instruction) ▷ *Decoded signals* ②
12: $CS_{modeled}[id, ex, wb] \leftarrow$ Function($CS_{modeled}$) ▷ *Combinational signals* ③
13:
14: ▷ *Encryption and absorption of signals*
15: $CS \leftarrow selection(CS_{modeled})$
16: state ← state $\oplus$ $0^* \,||\, CS \,||\, 0^*$
17: $\text{state}_{\text{rate}} \leftarrow \text{state}_{\text{rate}} \oplus$ instruction
18: instruction_cipher ← $\text{state}_{\text{rate}}$
19: state ← permutation(state, 6)

MINES Saint-Étienne
Une école de l'IMT

Absorption of Control Signals

Software model to infer control signal values

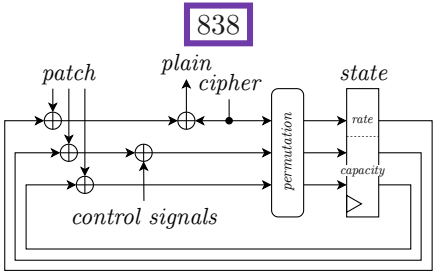
Control transfer: adapt patches

Absorption of Control Signals

Software model to infer control signal values

Control transfer: adapt patches

Hardware decryption



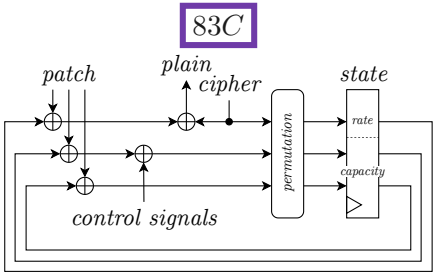
cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		*branch fetched*
5						
6						
7						
8						
	addresses		alu_operator			

Absorption of Control Signals

Software model to infer control signal values

Control transfer: adapt patches

Hardware decryption



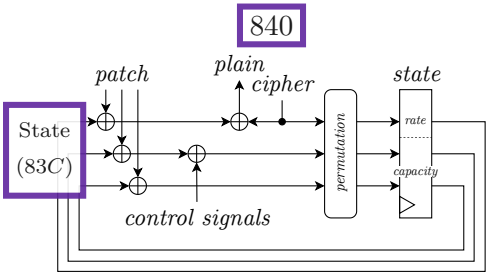
cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		*branch fetched*
5	840	83C	0D	18		*branch decoded*
6						
7						
8						
	addresses		alu_operator			

Absorption of Control Signals

Software model to infer control signal values

Control transfer: adapt patches

Hardware decryption



cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		*branch fetched*
5	840	83C	0D	18		*branch decoded*
6	82C	840	18	0D		*branch taken*
7						
8						
	addresses		alu_operator			

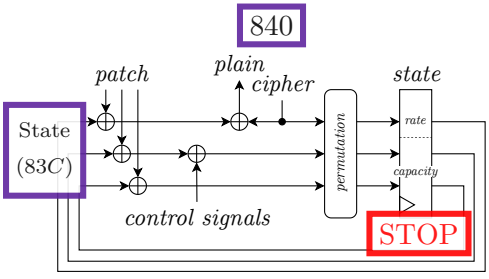
flush

Absorption of Control Signals

Software model to infer control signal values

Control transfer: adapt patches

Hardware decryption



cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		*branch fetched*
5	840	83C	0D	18		*branch decoded*
6	82C	840	18	0D		*branch taken*
7						
8						
	addresses		alu_operator			

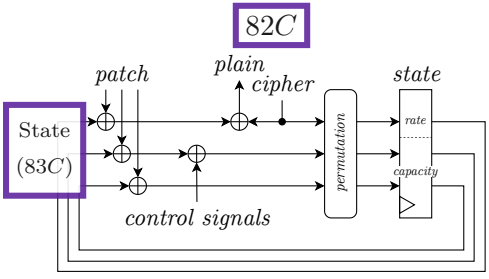
flush

Absorption of Control Signals

Software model to infer control signal values

Control transfer: adapt patches

Hardware decryption



cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		*branch fetched*
5	840	83C	0D	18		*branch decoded*
6	82C	840	18	0D		*branch taken*
7	830	82C	18	03		
8						
	addresses		alu_operator			

flush

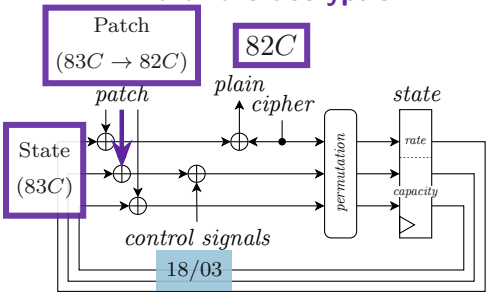
stall

Absorption of Control Signals

Software model to infer control signal values

Control transfer: adapt patches

Hardware decryption



➜ Adapt patch values

cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		*branch fetched*
5	840	83C	0D	18		*branch decoded*
6	82C	840	18	0D		*branch taken*
7	830	82C	18	03		
8						
	addresses		alu_operator			

flush

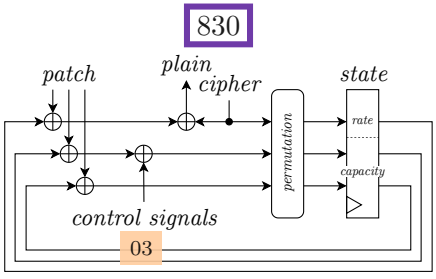
stall

Absorption of Control Signals

Software model to infer control signal values

Control transfer: adapt patches

Hardware decryption



cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		*branch fetched*
5	840	83C	0D	18		*branch decoded*
6	82C	840	18	0D		*branch taken*
7	830	82C	18	03		
8	834	830	18	03		
		addresses		alu_operator		

stall

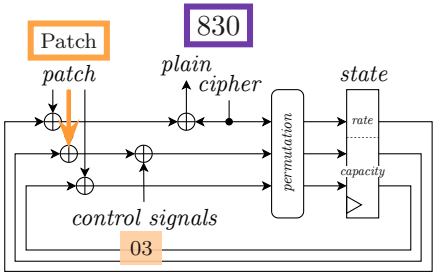
➜ Adapt patch values

Absorption of Control Signals

Software model to infer control signal values

Control transfer: adapt patches

Hardware decryption



- Adapt patch values
- Patches are extended

cyc	MEM	IF	ID	EX	WB
0	82C	828	18	03	
1	830	82C	0C	18	
2	834	830	18	0C	
3	838	834	18	18	
4	83C	838	18	18	
5	840	83C	0D	18	
6	82C	840	18	0D	
7	830	82C	18	03	
8	834	830	18	03	
		addresses		alu_operator	

branch fetched

branch decoded

branch taken

stall

Absorption of Control Signals

Validation and characterization

core-v-verif-fpga environnement

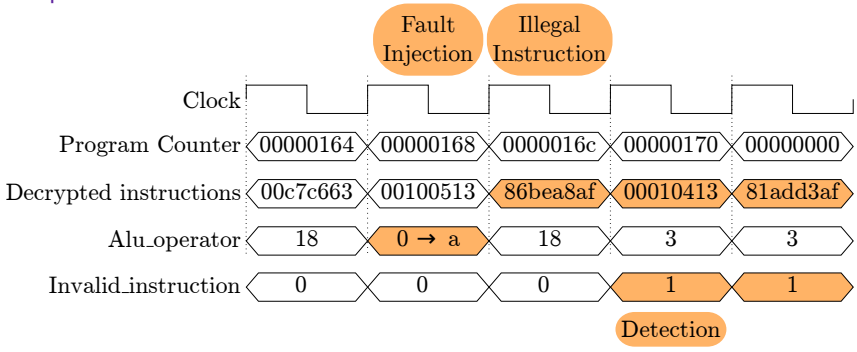
Makefile based:

- ➜ Compilation (*VerifyPin [18], Embench [19]* - 22 programs) and software encryption flow
- ✔ **Cycle-accurate** simulations with assertion of PC and instruction values (*Verilator*)
- ✔ Execution and validation on **FPGA** board (Nexys Video)
- ✔ **FIA**: emulated on FPGA (memory only) and on simulation (memory and control signals)

Absorption of Control Signals

Validation and characterization

Experimental validation

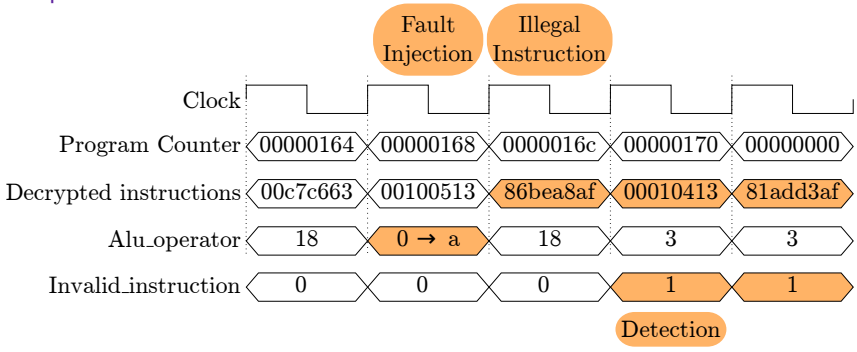


FIA ➜ Incorrect state ➜ Decryption = random instruction ➜ Illegal instruction

Absorption of Control Signals

Validation and characterization

Experimental validation



FIA ➜ Incorrect state ➜ Decryption = random instruction ➜ Illegal instruction

Original code is not executed anymore

Detection

Absorption of Control Signals

Validation and characterization

Memory

➜ Patches are extended:

$$|patch| = 2 \times |CS_{wb}| + |CS_{ex}| + |state|$$

Absorption of Control Signals

Validation and characterization

Memory

➜ Patches are extended:

$$|patch| = 2 \times |CS_{wb}| + |CS_{ex}| + |state|$$

- Example - Control Signals (21 bits):

```
' id ' :  [ ' imm_a_mux_sel ' , ' alu_op_a_mux_sel ' ] ,
' ex ' :  [ ' alu_operato r ' , ' branch_in_ex , ' reg file_a lu_wad dr ' ] ,
' wb ' :  [ ' regfile_mem_we ' , ' regf ile_m em_wad dr ' , ' data_type ' ]
```

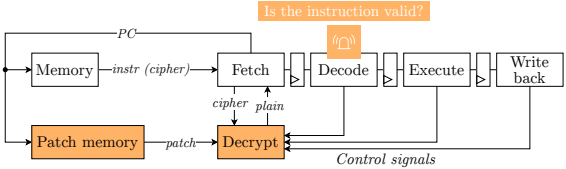
Name	Memory	Cycles	Freq.	Utilization
Ascon (320bits)	$\times 10$	0	-74.6	28.6 LUT
Ascon (320bits) + Control signals (21 bits)	$\times 11$	0	-74.6	29.8 LUT

MINES Saint-Étienne
Une école de l'IMT

Conclusion

Conclusion

Can these properties be provided together?



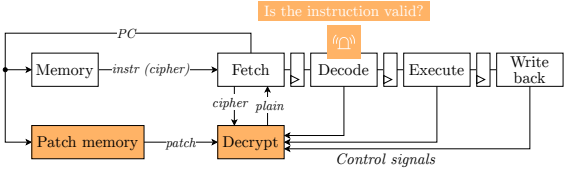
github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion

Can these properties be provided together?

Chained Instruction Encryption with Absorption of Control Signals:

- ✔ **Confidentiality** of Instructions
- ✔ **Integrity** of Control Flow
- ✔ **Integrity** of Instructions
- ✔ **Integrity** of (data-independent) Control Signals



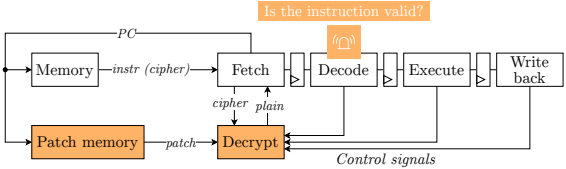
github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion

Can these properties be provided together?

Chained Instruction Encryption with Absorption of Control Signals:

- ✔ **Confidentiality** of Instructions
- ✔ **Integrity** of Control Flow
- ✔ **Integrity** of Instructions
- ✔ **Integrity** of (data-independent) Control Signals

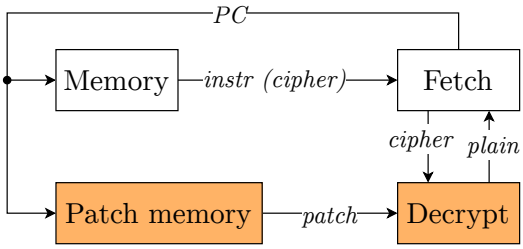


- ➜ **Open-source:** Github
- ➜ Implementation on CV32E40P
- ➜ Validation and Characterization on FPGA
  - ➜ FIA on VerifyPin execution
    - ➜ Memory: $\times$ 11
      - ➜ LUT: $+$ 29.8 % (1 PU)
    - ➜ Cycles: $+$ 0 %
  - ➜ Frequency: $-$ 74.6 % (1 PU)

github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Contributions:

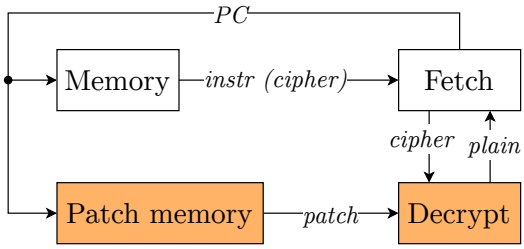


github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Contributions:

- ✔ **Patch policy**
  - ➜ Fine-grained CFI
  - ➜ 0 clock cycle overhead (COTS binaries)

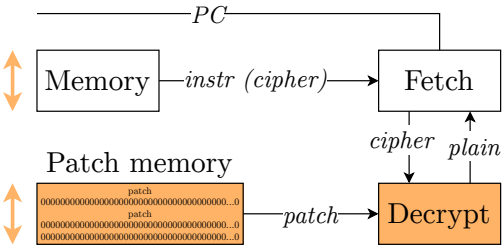


github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Contributions:

- ✔ **Patch policy**
  - ➜ Fine-grained CFI
  - ➜ 0 clock cycle overhead (COTS binaries)
  - ✖ But, patch memory as high as instr. mem.
  - ✖ But, empty at 75 %

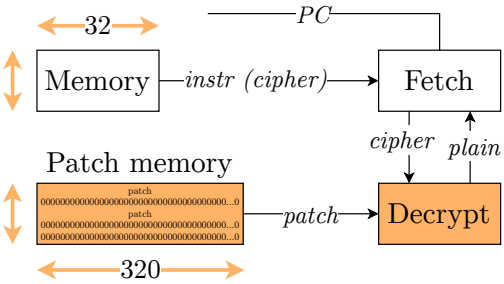


github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Contributions:

- ✔ **Patch policy**
  - ➜ Fine-grained CFI
  - ➜ 0 clock cycle overhead (COTS binaries)
  - ✖ But, patch memory as high as instr. mem.
  - ✖ But, empty at 75 %
  - ✖ **Ascon:** patches are 320 bits wide
  - ➜ Consider a 64 bits state (e.g., Prince)



github.com/theophile-gousselot/execution_integrity_down_to_control_signals

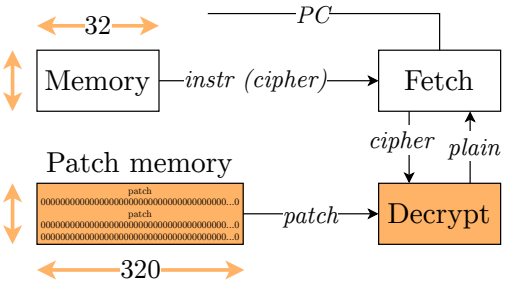
Conclusion
Limitations and perspectives

Contributions:

✔ Patch policy

- ➜ Fine-grained CFI
- ➜ 0 clock cycle overhead (COTS binaries)
- ✖ But, patch memory as high as instr. mem.
- ✖ But, empty at 75 %

- ✖ **Ascon:** patches are 320 bits wide
- ➜ Consider a 64 bits state (e.g., Prince)



✔ Infer control signal values

- ➜ By analyzing the microarchitecture and the program
- ✔ Control Signals can be absorbed in some CFI solutions
- ✔ CV32E40P, in-order, no cache

github.com/theophile-gousselot/execution_integrity_down_to_control_signals

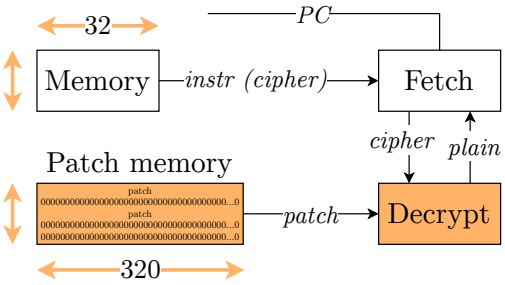
Conclusion
Limitations and perspectives

Contributions:

✔ Patch policy

- ➜ Fine-grained CFI
- ➜ 0 clock cycle overhead (COTS binaries)
- ✖ But, patch memory as high as instr. mem.
- ✖ But, empty at 75 %

- ✖ **Ascon:** patches are 320 bits wide
- ➜ Consider a 64 bits state (e.g., Prince)



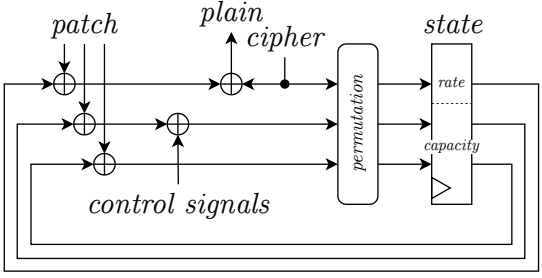
✔ Infer control signal values

- ➜ By analyzing the microarchitecture and the program
- ✔ Control Signals can be absorbed in some CFI solutions
- ✔ CV32E40P, in-order, no cache
- ? Cache, out-of-order execution

github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Security issues:

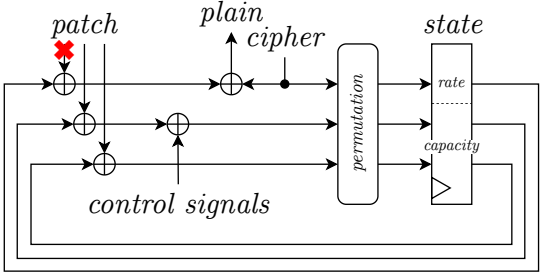


github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Security issues:

- ✖ Tamper with **patch**

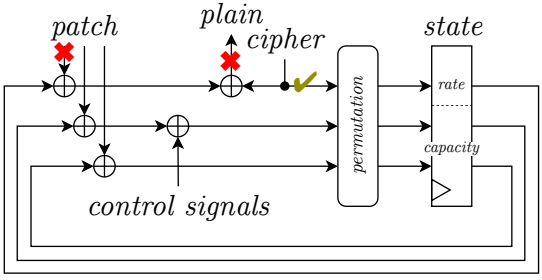


github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Security issues:

- ✖ Tamper with **patch**

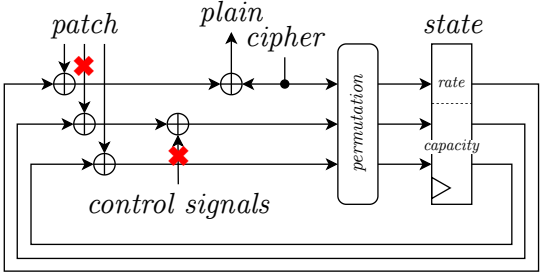


github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Security issues:

- ✖ Tamper with **patch**
- ✖ Tamper with **patch** and FIA on **control signals**

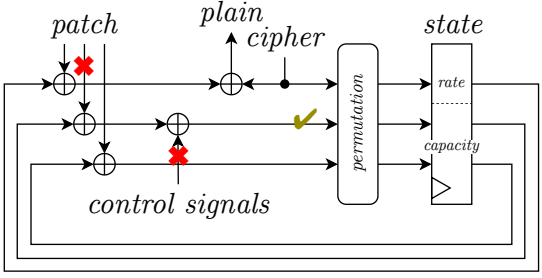


github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Security issues:

- ✖ Tamper with **patch**
- ✖ Tamper with **patch** and FIA on **control signals**

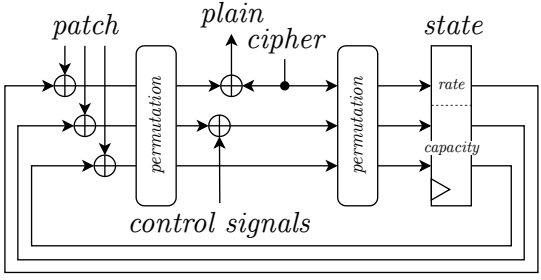


github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Security issues:

- ✖ Tamper with **patch**
- ✖ Tamper with **patch** and FIA on **control signals**
- ✔ Insert permutations after patch absorption

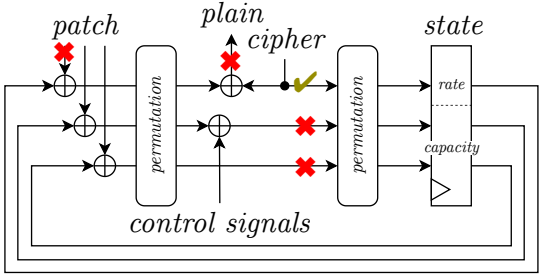


github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Security issues:

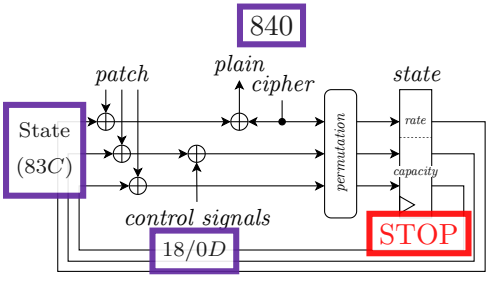
- ✖ Tamper with **patch**
- ✖ Tamper with **patch** and FIA on **control signals**
- ✔ Insert permutations after patch absorption



github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Hardware decryption

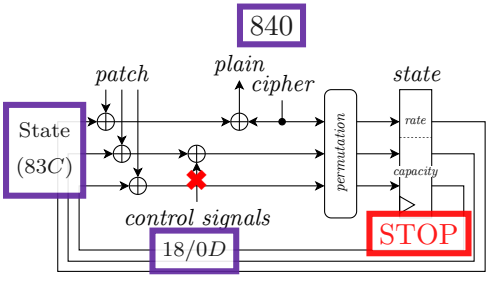


flush

cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		*branch fetched*
5	840	83C	0D	18		*branch decoded*
6	82C	840	18	0D		*branch taken*
7						
8						
	addresses		alu_operator			

Conclusion
Limitations and perspectives

Hardware decryption

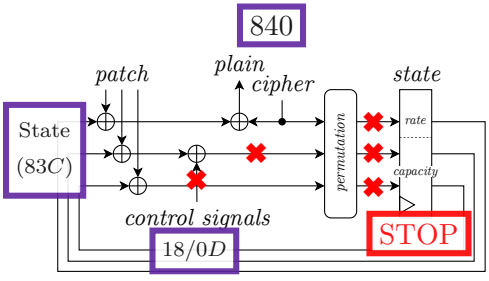


flush

cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		*branch fetched*
5	840	83C	0D	18		*branch decoded*
6	82C	840	18	0D		*branch taken*
7						
8						
	addresses		alu_operator			

Conclusion
Limitations and perspectives

Hardware decryption

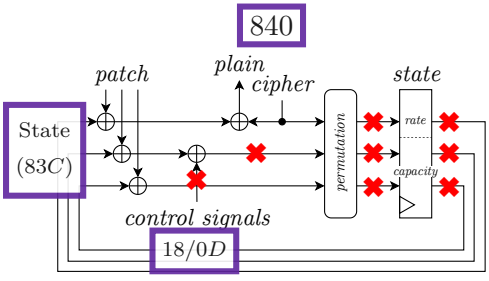


flush

cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		*branch fetched*
5	840	83C	0D	18		*branch decoded*
6	82C	840	18	0D		*branch taken*
7						
8						
	addresses		alu_operator			

Conclusion
Limitations and perspectives

Hardware decryption



flush

cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		*branch fetched*
5	840	83C	0D	18		*branch decoded*
6	82C	840	18	0D		*branch taken*
7						
8						
	addresses		alu_operator			

Conclusion
Limitations and perspectives

Security issues:

- ✖ Tamper with **patch**
- ✖ Tamper with **patch** and FIA on **control signals**
- ✔ Insert permutations after patch absorption

- ✖ FIA on **Control Signals** when **fetch is stalled**
- ✔ Do not STOP the hardware module

github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Conclusion
Limitations and perspectives

Security issues:

- ✖ Tamper with **patch**
- ✖ Tamper with **patch** and FIA on **control signals**
- ✔ Insert permutations after patch absorption

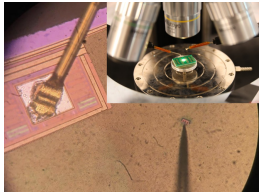
- ✖ FIA on **Control Signals** when **fetch is stalled**
- ✔ Do not STOP the hardware module

- ➜ Formal verification [20]

github.com/theophile-gousselot/execution_integrity_down_to_control_signals

Linear Code Extraction

(manuscript, part 3)



HOST23

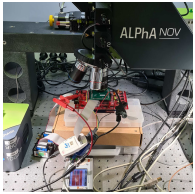
Third Best Hardware Demo*

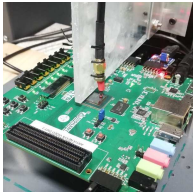
Patent: FR2209624

*among 24, at HOST23

Execution integrity

(manuscript, part 2)





HOST25

SemSecuElec

STMicroelectronics

Workshop ARSENE

MINES Saint-Étienne
Une école de l'IMT

Bibliography

[1] Chris Gerlinsky.
Breaking code read protection on the nxp lpc-family microcontrollers.
RECON, Brussels, Belgium, 2017.

[2] Johannes Obermaier and Stefan Tatschner.
Shedding too much light on a microcontroller's firmware protection.
In 11th USENIX Workshop on Offensive Technologies (WOOT 17), 2017.

[3] Bilgiday Yuce, Patrick Schaumont, and Marc Witteman.
Fault attacks on secure embedded software: Threats, design, and evaluation.
Journal of Hardware and Systems Security, 2:111–130, 2018.

[4] Johan Laurent, Vincent Beroulle, Christophe Deleuze, Florian Pebay-Peyroula, and Athanasios Papadimitriou.
Cross-layer analysis of software fault models and countermeasures against hardware fault attacks in a risc-v processor.
Microprocessors and Microsystems, 71:102862, 2019.

[5] Sergei P Skorobogatov and Ross J Anderson.
Optical fault induction attacks.
In Cryptographic Hardware and Embedded Systems-CHES 2002: 4th International Workshop Redwood Shores, CA, USA, August 13–15, 2002 Revised Papers 4, pages 2–12. Springer, 2003.

[6] Vanthanh Khuat, Jean-Luc Danger, and Jean-Max Dutertre.
Laser fault injection in a 32-bit microcontroller: from the flash interface to the execution pipeline.
In 2021 Workshop on Fault Detection and Tolerance in Cryptography (FDTC), pages 74–85. IEEE, 2021.

Bibliography

[7] Mario Werner, Erich Wenger, and Stefan Mangard.
Protecting the control flow of embedded processors against fault attacks.
In Smart Card Research and Advanced Applications: 14th International Conference, CARDIS 2015, Bochum, Germany, November 4-6, 2015. Revised Selected Papers 14, pages 161–176. Springer, 2016.

[8] Ruan De Clercq, Johannes Götzfried, David Übler, Pieter Maene, and Ingrid Verbauwhede.
Sofia: Software and control flow integrity architecture.
Computers & Security, 68:16–35, 2017.

[9] Jean-Luc Danger, Adrien Facon, Sylvain Guilley, Karine Heydemann, Ulrich Kühne, Abdelmalek Si Merabet, and Michaël Timbert.
Ccfi-cache: A transparent and flexible hardware protection for code and control-flow integrity.
In 2018 21st Euromicro Conference on Digital System Design (DSD), pages 529–536. IEEE, 2018.

[10] Mario Werner, Thomas Unterluggauer, David Schaffenrath, and Stefan Mangard.
Sponge-based control-flow protection for iot devices.
In 2018 IEEE European Symposium on Security and Privacy (EuroS&P), pages 214–226. IEEE, 2018.

[11] Olivier Savry, Mustapha El-Majihi, and Thomas Hiscock.
Confidaent: Control flow protection with instruction and data authenticated encryption.
In 2020 23rd Euromicro Conference on Digital System Design (DSD), pages 246–253. IEEE, 2020.

[12] Ronan Lashermes, Hélène Le Bouder, and Gaël Thomas.
Hardware-assisted program execution integrity: Hapei.
In Secure IT Systems: 23rd Nordic Conference, NordSec 2018, Oslo, Norway, November 28-30, 2018, Proceedings 23, pages 405–420. Springer, 2018.

Bibliography

[13] Anthony Zgheib, Olivier Potin, Jean-Baptiste Rigaud, and Jean-Max Dutertre.
Cifer: Code integrity and control flow verification for programs executed on a risc-v core.
In 2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), pages 100–110. IEEE, 2023.

[14] Thomas Chamelot, Damien Couroussé, and Karine Heydemann.
Mafia: Protecting the microarchitecture of embedded systems against fault injection attacks.
IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2023.

[15] Thomas Hiscock, Olivier Savry, and Louis Goubin.
Lightweight instruction-level encryption for embedded processors using stream ciphers.
Microprocessors and Microsystems, 64:43–52, 2019.

[16] François Bonnal, Vincent Dupaquis, Olivier Potin, and Jean-Max Dutertre.
Software-only control-flow integrity against fault injection attacks.
In 2023 26th Euromicro Conference on Digital System Design (DSD), pages 269–277. IEEE, 2023.

[17] Gaëtan Leplus, Olivier Savry, and Lilian Bossuet.
Secdec: Secure decode stage thanks to masking of instructions with the generated signals.
In 2022 25th Euromicro Conference on Digital System Design (DSD), pages 556–563. IEEE, 2022.

[18] Louis Dureuil, Guillaume Petiot, Marie-Laure Potet, Thanh-Ha Le, Aude Crohen, and Philippe de Choudens.
Fissc: A fault injection and simulation secure collection.
In Computer Safety, Reliability, and Security: 35th International Conference, SAFECOMP 2016, Trondheim, Norway, September 21-23, 2016, Proceedings 35, pages 3–11. Springer, 2016.
Les programmes sont disponibles à https://lazart.gricad-pages.univ-grenoble-alpes.fr/fissc/.

[19] Embench.
Embench: Open benchmarks for embedded platforms https://github.com/embench/embench-iot, 2022.

MINES Saint-Étienne
Une école de l'IMT

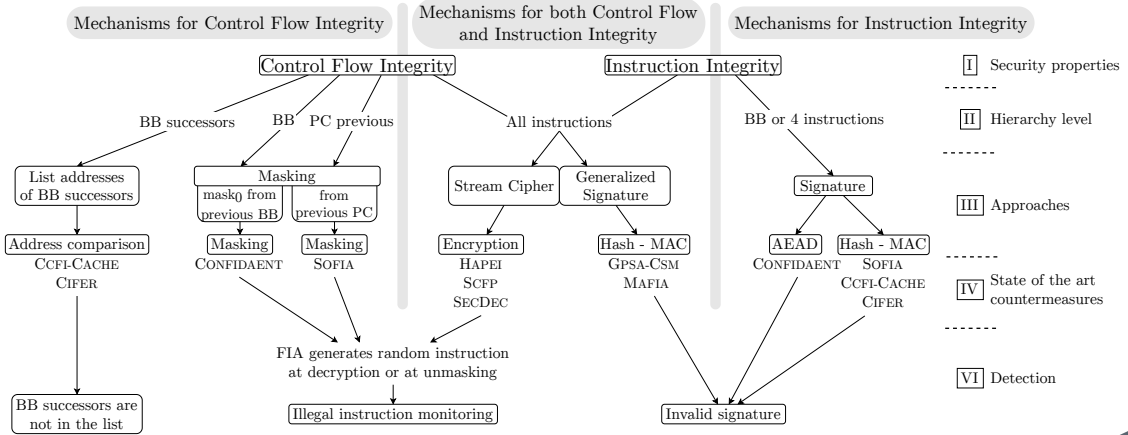
Bibliography

[20] Simon Tollec, Mihail Asavoae, Damien Couroussé, Karine Heydemann, and Mathieu Jan.
Exploration of fault effects on formal risc-v microarchitecture models.
In 2022 Workshop on Fault Detection and Tolerance in Cryptography (FDTC), pages 73–83. IEEE, 2022.

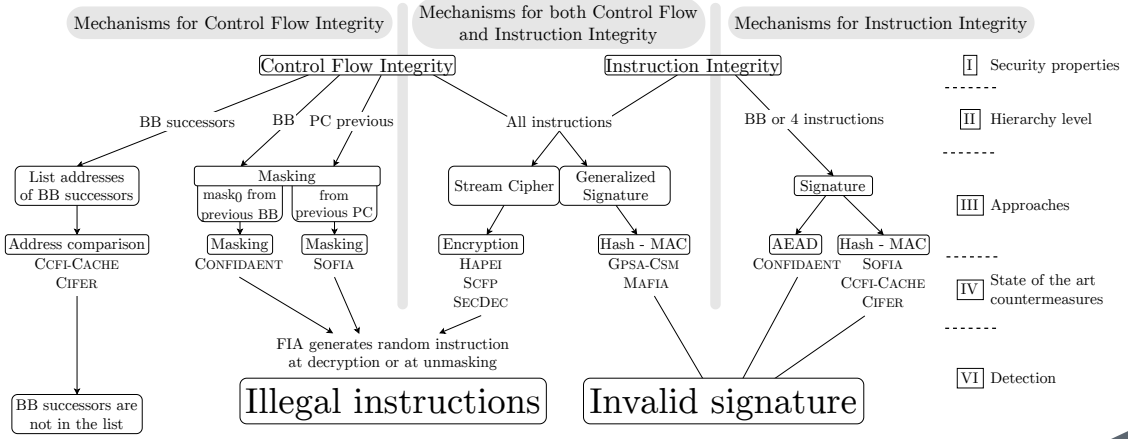
MINES Saint-Étienne
Une école de l'IMT

Table of content

Bibliography



Bibliography



Bibliography
Comparison
Mechanisms

Solution	Primitive	État	Signature	Taille mémoire	Cycles d'horloge
Confidaent [Savry 2020]	Ascon (Chif rement authentif é) + Masquage	-	128 bits	vecteurs d'initialisation tags de référence masques de référence instructions `load` masque	exécution des instructions insérées initialisation d'Ascon
Sofia [De Clercq 2017a]	AES (Chif rement) AES (CBC-MAC)	-	64 bits	MAC de référence CFG → arbre binaire	exécution des instructions insérées *fetch* des MAC
Ccfi-Cache [Danger 2018]	*non précisé* (*hash*)	-	32 bits	métadonnées instructions `nop`	exécution des instructions insérées
Cifer [Zgheib 2023b]	MISR (Code détecteur)	-	47 bits	métadonnées	0
SecDec [Leplus 2022]	Masquage	-	-	instructions `jalr`	exécution des instructions insérées
Hapei [Lashermes 2018]	SHA-256 (SHA2) (Chif rement via Sponge-duplex)	256 bits	-	patchs	*non implémenté*
Scfp [Werner 2018]	Prince (Chif rement via Sponge-duplex)	64 bits	-	patchs	*fetch* des patchs
Gpsa-Csm [Werner 2016]	CRC (Code détecteur)	32 bits	32 bits	signatures de référence patchs instructions `load` patch instructions de vérif cation	exécution des instructions insérées
Mafia [Chamelot 2023]	CRC (code détecteur) ou Prince (CBC-MAC)	32 bits	32 bits	signatures de référence instructions `load` patch CFG → arbre binaire instructions `nop`*	exécution des instructions insérées *fetch* des signatures de référence

* : lié à la protection des signaux de contrôle.

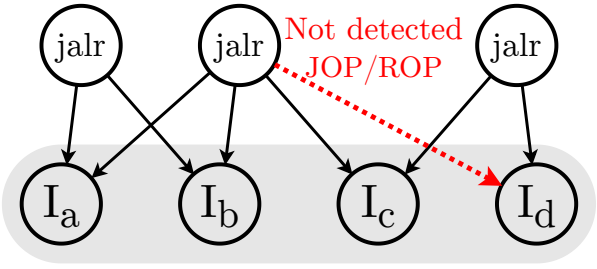
Bibliography

Comparison

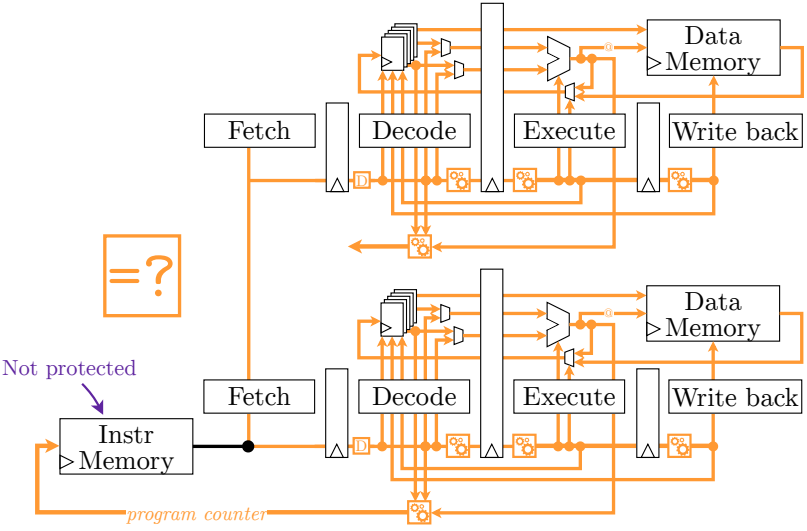
Validation and characterization

Solution	Disponible	RTL	Synthèse	Validation	Injection de fautes
Soft-Only [Bonnal 2023]	✓			*logique*	✓
Confidaent [Savry 2020]		✓		*logique*	
Sofia [De Clercq 2017a]		✓	FPGA	FPGA	
Ccfi-Cache [Danger 2018]		✓	FPGA	*logique*	✓
Cifer [Zgheib 2023b]		✓	FPGA	FPGA	✓
SecDec [Leplus 2022]		✓	ASIC	*logique*	✓
Hapei [Lashermes 2018]	✓			*VM*	✓
Scfp [Werner 2018]		✓	ASIC	*logique*	
Gpsa-Csm [Werner 2016]		✓	ASIC	*logique*	
Mafia [Chamelot 2023]		✓	ASIC	*logique*	
Ce travail [Gousselot 2025]	✓	✓	FPGA	FPGA	✓

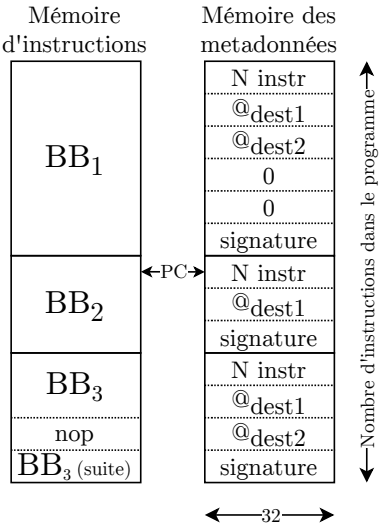
Bibliography
Equivalence class



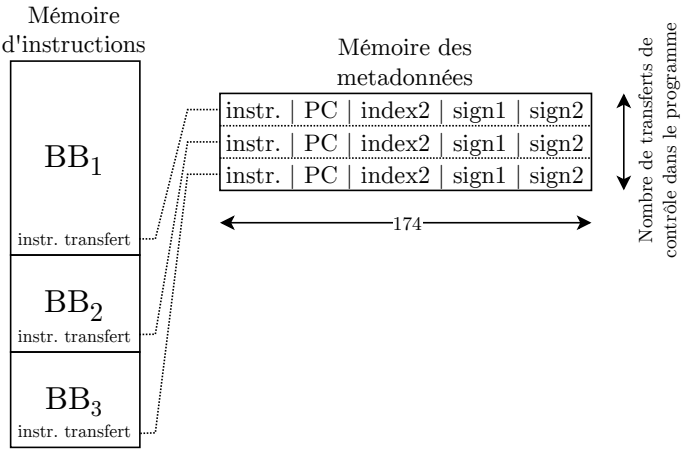
Bibliography
Lock Step



Bibliography
CCFI-Cache



Bibliography
Cifer



Chained Instruction Encryption

Algo CFG

Algorithme 1 Le CFG est construit itérativement en fonction du type d'instruction.

```
1:  for instruction in code do
2:      characterize instruction          ▷ Adresse, type, operandes, valeurs immédiates
3:      if instruction is not transfer control then
4:          successor ← addr + 4
5:      else if instruction == jal then
6:          successor ← addr + imm
7:      else if instruction == branch then
8:          successor ← [addr + imm, addr + 4]
9:  for each instruction in code do
10:     if instruction == jalr and rs1 != ra then      ▷ Sauts indirects sauf retour de fonction
11:         successor ← get_jalr_successors_from_nominal_exec()
12:     else if instruction == jalr and rs1 == ra then      ▷ Retours de fonction
13:         successor ← get_return_successors_rec()
```

Chained Instruction Encryption

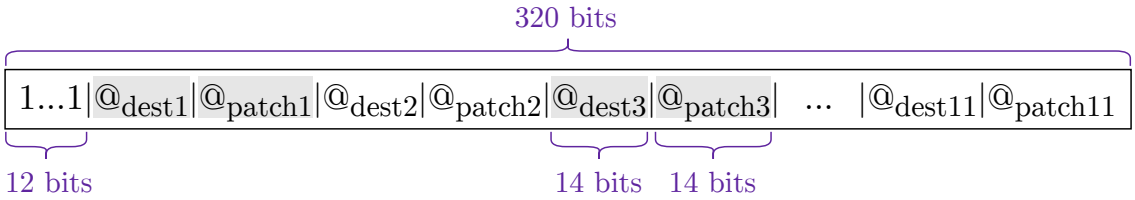
Algo Adapt patch

Algorithme 3 Génération des patchs.

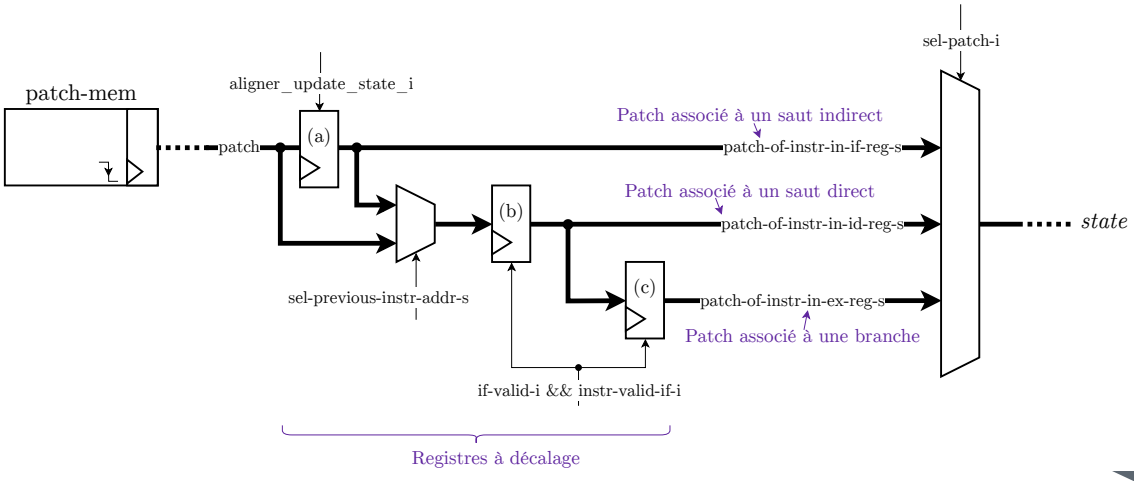
```
1:  patchs ← [0, 0, ..., 0]
2:
3:  ▷ (1) Traitement des sauts directs et branchements
4:  for each instruction in code do
5:      if instruction == branch then
6:          patchs[src] ← state_src+8 xor state_dest
7:      else if instruction == jal then
8:          patchs[src] ← state_src+4 xor state_dest
9:
10: ▷ (2) Traitement des sauts indirects sans conf it
11: for each instruction in code do
12:     if instruction == jalr then
13:         instruction.reallocate ← false
14:         for dest in jalr destinations do
15:             if patchs[dest] is not free then                ▷ Détection des conf its
16:                 instruction.reallocate ← true
17:         if not instruction.realocate then
18:             for dest in jalr destinations do
19:                 patchs[dest] ← state_src+4 xor state_dest
20:
21: ▷ (3) Traitement des sauts indirects en conf it
22: for each instruction in code do
23:     if instruction.reallocate then
24:         for dest in jalr destinations do
25:             free_address ← f nd_free_address()
26:             patchs[free_address] ← state_src+4 xor state_dest
```

Chained Instruction Encryption

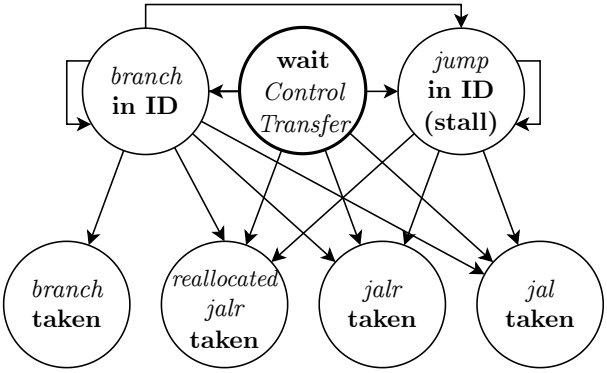
Reallocation information



Chained Instruction Encryption
Shift registers



Chained Instruction Encryption
FSM



Every state can reach "*wait* **Control Transfer**" except "*jump* **in ID (stall)**".

Chained Instruction Encryption

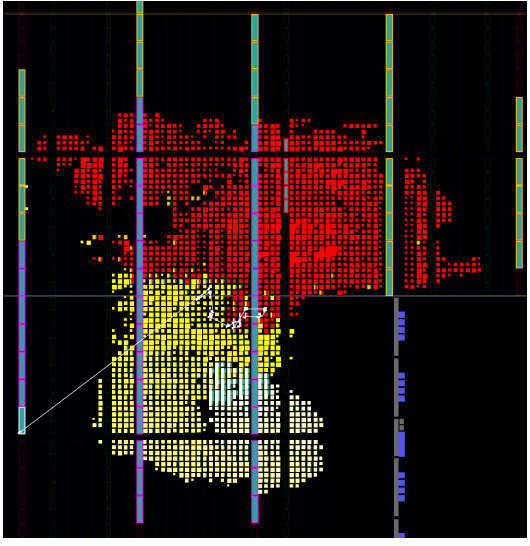
Embench statistics

Programmes (-Os)	#instructions	#br	#jal	#jalr	#patchs	#réallocations
aha-mont64	2354	247	108	249	603 (25,6 %)	23 (1,0 %)
crc32	3359	340	148	390	885 (26,3 %)	30 (0,9 %)
cubic	12124	838	186	1342	2564 (21,1 %)	46 (0,4 %)
dhrystone	3689	404	142	405	1042 (28,2 %)	32 (0,9 %)
edn	2486	245	115	248	597 (24,0 %)	25 (1,0 %)
fibonacci	3210	328	128	391	876 (27,3 %)	30 (0,9 %)
huffbench	3958	368	151	427	966 (24,4 %)	32 (0,8 %)
matmult-int	2223	240	112	243	585 (26,3 %)	25 (1,1 %)
md5sum	3611	358	149	400	950 (26,3 %)	35 (1,0 %)
minver	4483	421	147	641	1283 (28,6 %)	49 (1,1 %)
nbody	4449	404	132	549	1140 (25,6 %)	31 (0,7 %)
nettle-aes	2905	252	109	255	608 (20,9 %)	22 (0,8 %)
nettle-sha256	3640	250	109	246	602 (16,5 %)	23 (0,6 %)
nsichneu	6464	228	104	864	1181 (18,3 %)	22 (0,3 %)
picojpeg	6132	602	175	581	1490 (24,3 %)	43 (0,7 %)
primecount	3371	340	148	395	887 (26,3 %)	29 (0,9 %)
qrduino	5346	489	162	540	1261 (23,6 %)	38 (0,7 %)
sglib-combined	5274	513	232	679	1438 (27,3 %)	50 (0,9 %)
slre	4237	406	159	522	1114 (26,3 %)	36 (0,8 %)
st	4519	409	137	545	1143 (25,3 %)	33 (0,7 %)
statemate	3045	286	130	414	837 (27,5 %)	43 (1,4 %)
tarfind	3444	345	147	397	899 (26,1 %)	29 (0,8 %)
ud	2852	284	115	331	729 (25,6 %)	29 (1,0 %)
verifypin-0	3105	313	132	370	841 (27,1 %)	31 (1,0 %)
wikisort	7147	594	230	816	1737 (24,3 %)	62 (0,9 %)
Moyenne (-Os)	4297	380	144	490	1050 (24,4 %)	34 (0,8 %)
Moyenne (-O2)	4495	388	149	518	1088 (24,2 %)	37 (0,8 %)
Moyenne (-O3)	5060	422	151	562	1189 (23,5 %)	35 (0,7 %)

Chained Instruction Encryption

Embench statistics

FPGA device



Chained Instruction Encryption

Embench statistics

FPGA device

Chained Instruction Encryption

Embench statistics

Probability of no-detection

FIA ➜ Incorrect state ➜ Decryption = random instruction ➜ Illegal instruction

Chained Instruction Encryption

Embench statistics

Probability of no-detection

FIA ➜ Incorrect state ➜ Decryption = random instruction ➜ Illegal instruction

Original code is not executed anymore

Detection

Chained Instruction Encryption

Embench statistics

Probability of no-detection

How many random instructions before detection?

Chained Instruction Encryption

Embench statistics

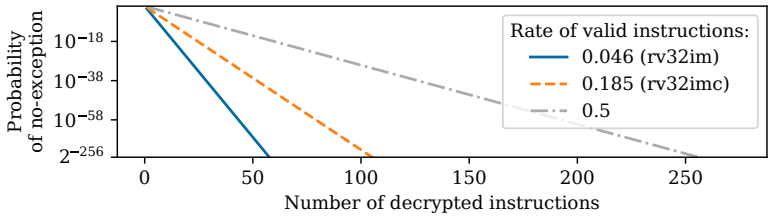
Probability of no-detection

FIA ➜ Incorrect state ➜ Decryption = random instruction ➜ Illegal instruction

Original code is not executed anymore

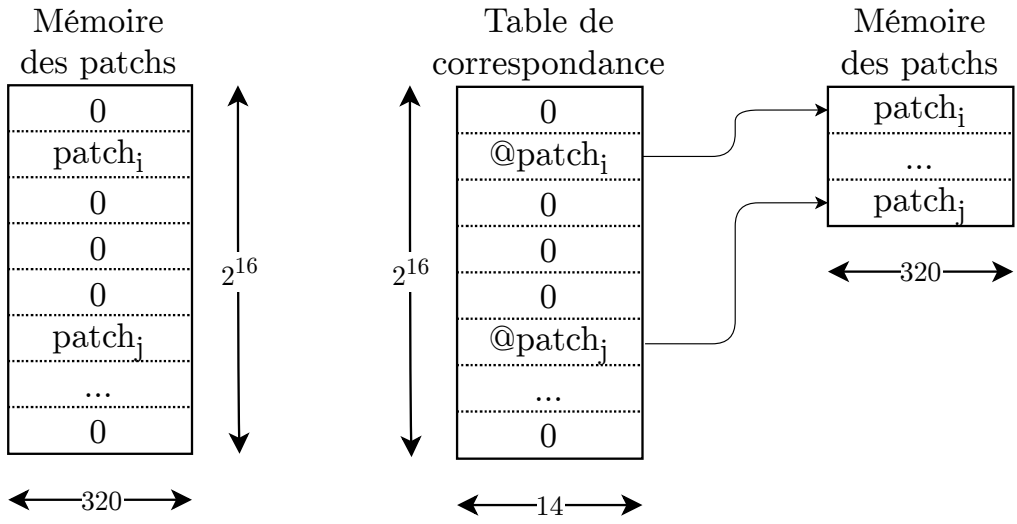
Detection

How many random instructions before detection?



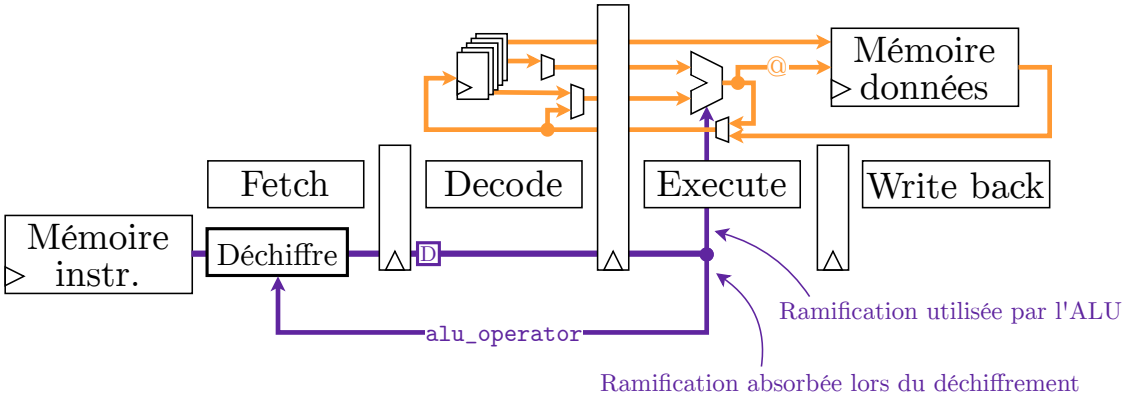
Chained Instruction Encryption

Reduce patch memory



Chained Instruction Encryption

Are you checking the signal used?



Absorption of Control Signals

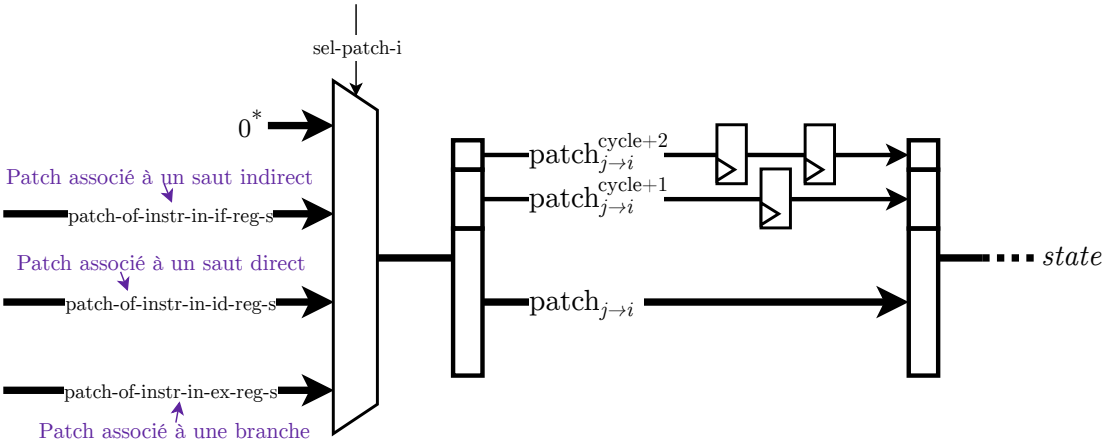
Taxonomy of control signals: examples

Solution	Primitive	État	Signature	Taille mémoire	Cycles d'horloge
CONFIDAENT [Savry 2020]	Ascon (Chif rement authentif é) + Masquage	-	128 bits	vecteurs d'initialisation tags de référence masques de référence instructions `load` masque	exécution des instructions insérées initialisation d'Ascon
SOFIA [De Clercq 2017a]	AES (Chif rement) AES (CBC-MAC)	-	64 bits	MAC de référence CFG → arbre binaire	exécution des instructions insérées *fetch* des MAC
CCFI-CACHE [Danger 2018]	*non précisé* (*hash*)	-	32 bits	métadonnées instructions `nop`	exécution des instructions insérées
CIFER [Zgheib 2023b]	MISR (Code détecteur)	-	47 bits	métadonnées	0
SECDEC [Leplus 2022]	Masquage	-	-	instructions `jalr`	exécution des instructions insérées
HAPEI [Lashermes 2018]	SHA-256 (SHA2) (Chif rement via Sponge-duplex)	256 bits	-	patchs	*non implémenté*
SCFP [Werner 2018]	Prince (Chif rement via Sponge-duplex)	64 bits	-	patchs	*fetch* des patchs
GPSA-CSM [Werner 2016]	CRC (Code détecteur)	32 bits	32 bits	signatures de référence patchs instructions `load` patch instructions de vérif cation	exécution des instructions insérées
MAFIA [Chamelot 2023]	CRC (code détecteur) ou Prince (CBC-MAC)	32 bits	32 bits	signatures de référence instructions `load` patch CFG → arbre binaire instructions `nop`*	exécution des instructions insérées *fetch* des signatures de référence

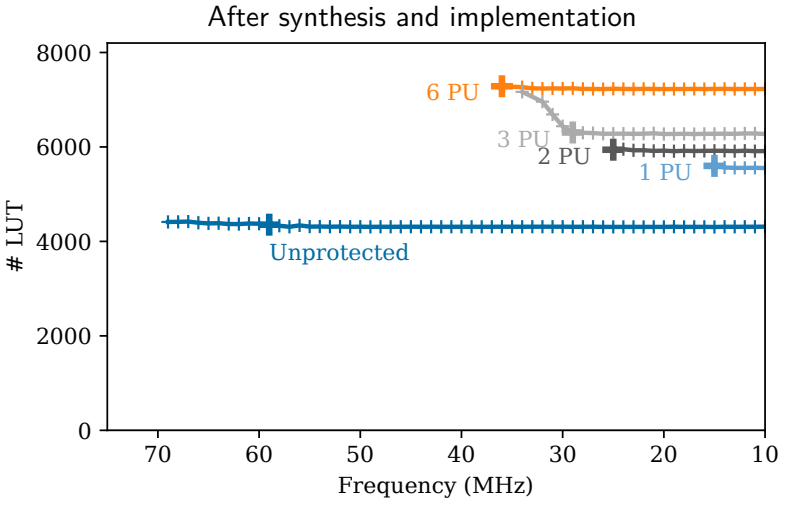
* : lié à la protection des signaux de contrôle.

Absorption of Control Signals

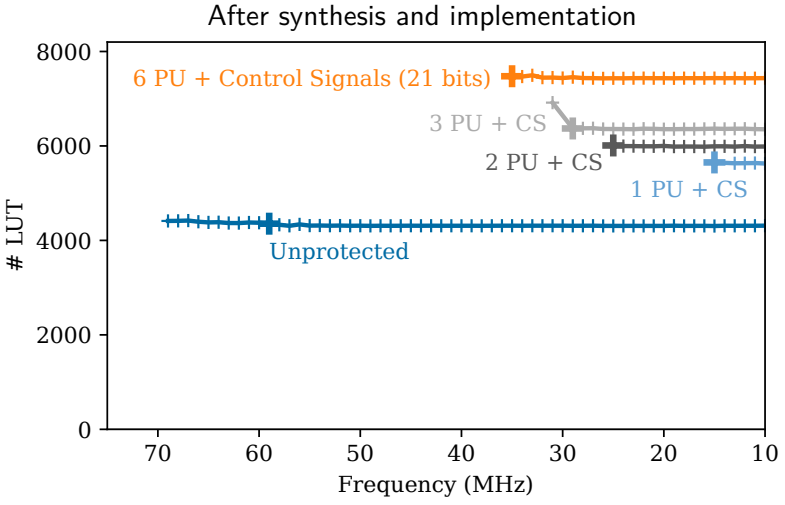
Shift registers: patch extented



Absorption of Control Signals
LUT and Frequency

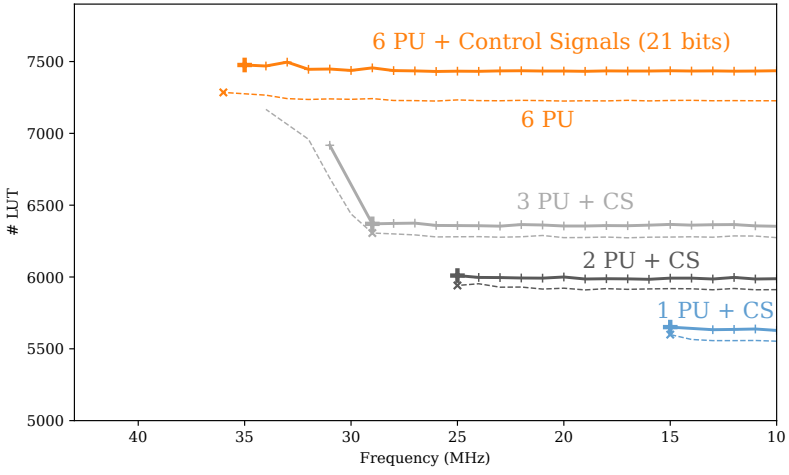


Absorption of Control Signals
LUT and Frequency

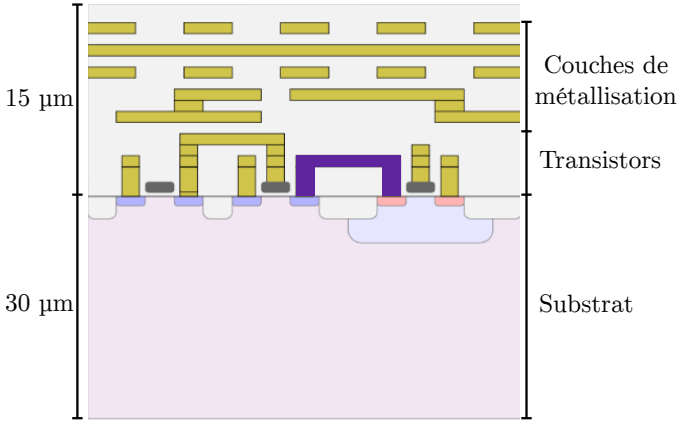


Absorption of Control Signals
LUT and Frequency

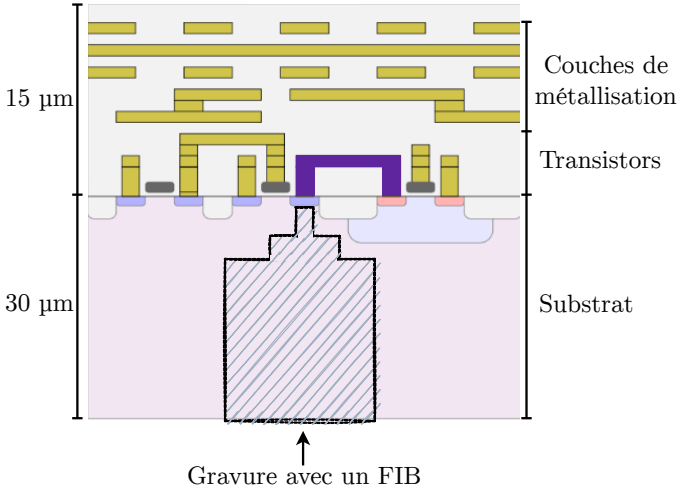
After synthesis and implementation



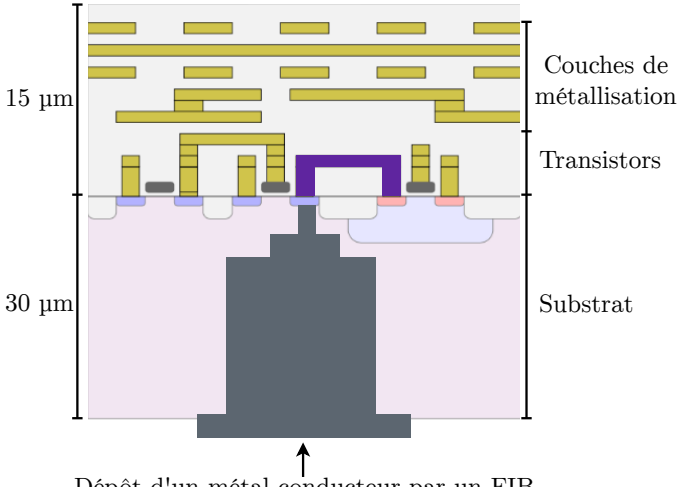
Linear Code Extraction
Microprobing



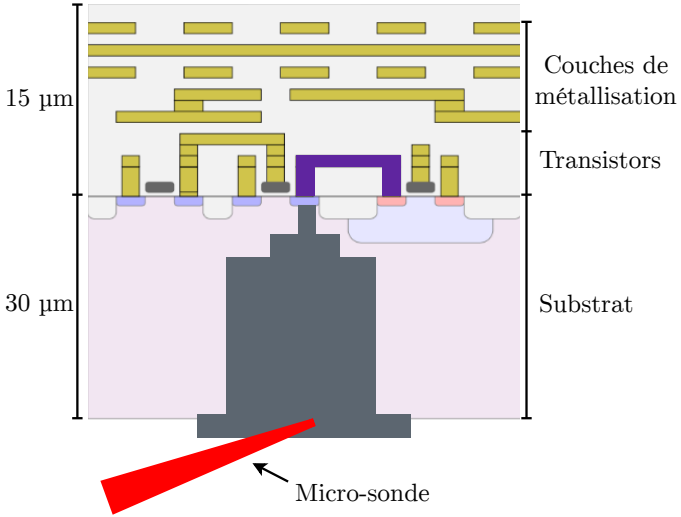
Linear Code Extraction
Microprobing



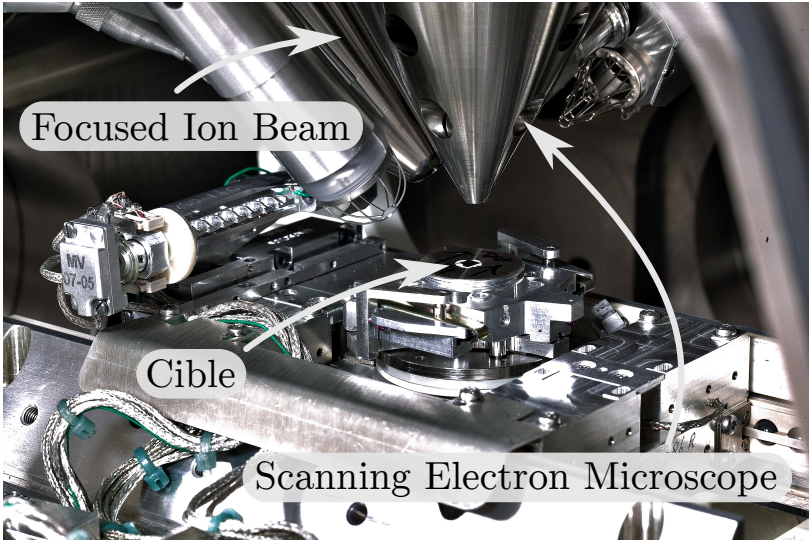
Linear Code Extraction
Microprobing



Linear Code Extraction
Microprobing

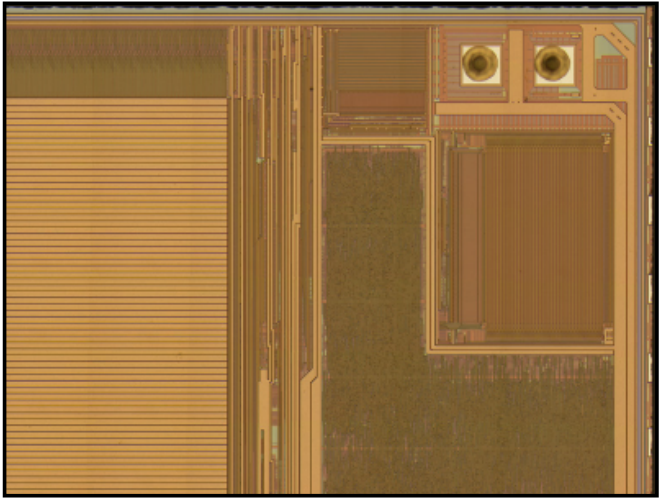


Linear Code Extraction
Dual Beam chamber

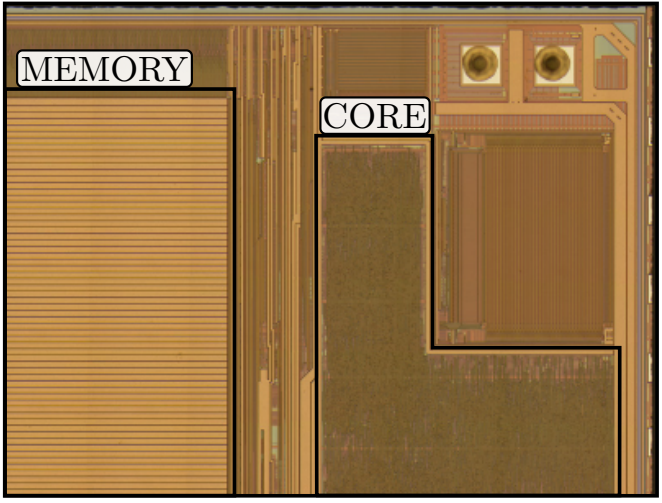


Linear Code Extraction

Step by step methodology

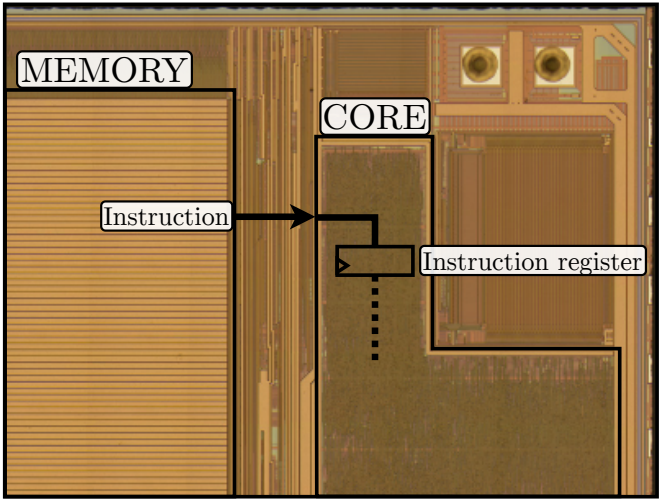


Linear Code Extraction
Step by step methodology



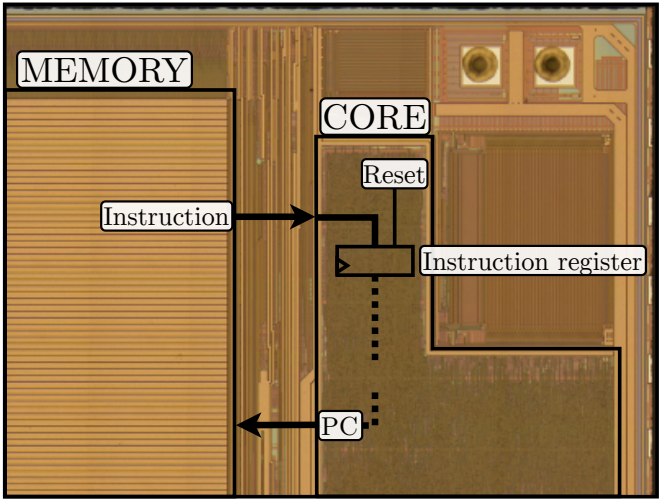
: Hardware components

Linear Code Extraction
Step by step methodology



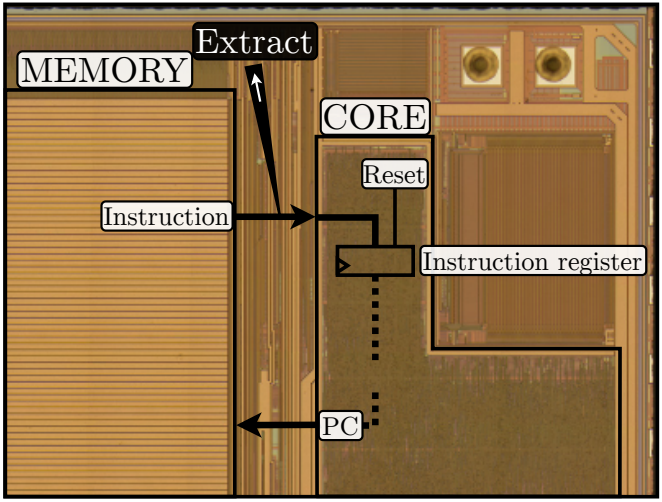
: Hardware components

Linear Code Extraction
Step by step methodology



: Hardware components

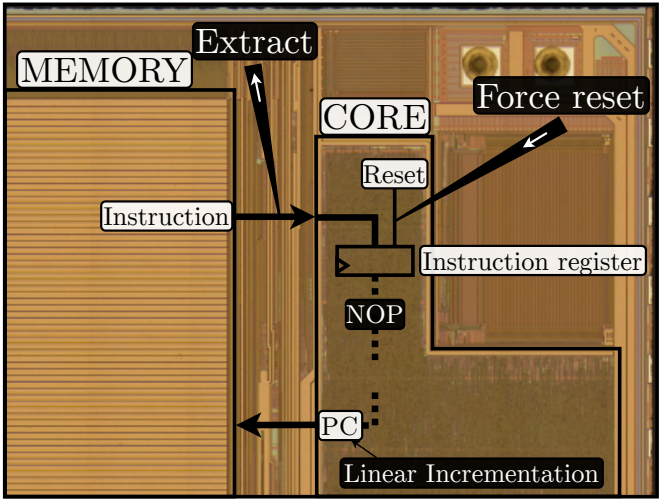
Linear Code Extraction
Step by step methodology



: Microprobes to eavesdrop

: Hardware components

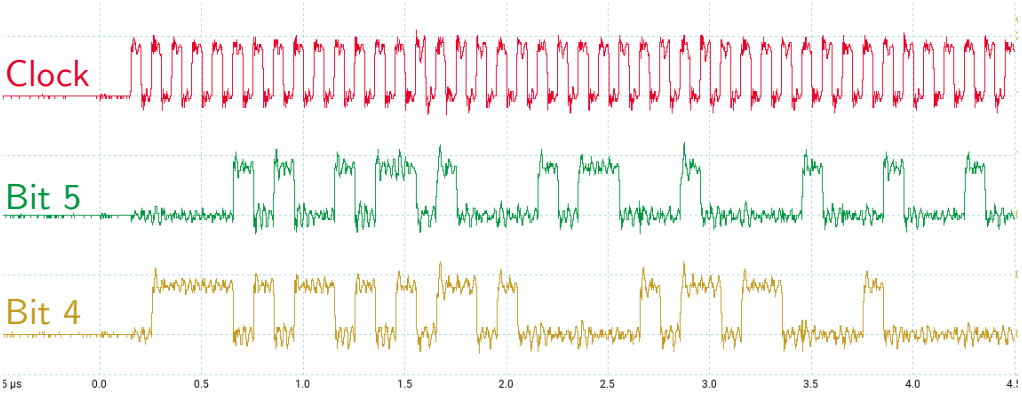
Linear Code Extraction
Step by step methodology



: Microprobes to eavesdrop
: Microprobe to induce a linear execution
: LCE attack effects
: Hardware components

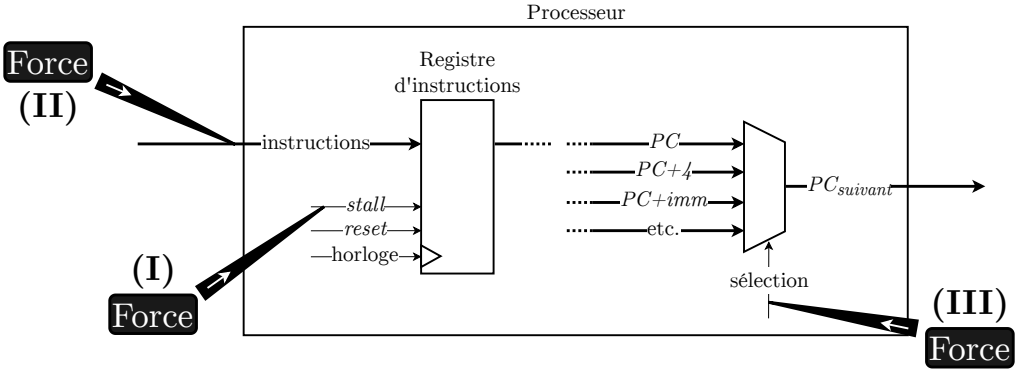
Linear Code Extraction

Practical Attack (Experimental Setup)



Extracted signals by the eavesdropping microprobes

Linear Code Extraction
Strategies to force a linear execution



Linear Code Extraction

Freezing an instruction

Memory	Executed	Extracted

```
0x80: auipc  t0,34062336
0x84: addi   t0,t0,-128
0x88: auipc  t1,34070528
0x8c: addi   t1,t1,-260
0x90: sw     zero,0(t0)
0x94: addi   t0,t0,4
0x98: bgtu   t1,t0,-8
0x9c: auipc  sp,34070528
```

Linear Code Extraction

Freezing an instruction

Memory	Executed	Extracted
0x80: auipc t0,34062336	auipc t0,34062336	auipc t0,34062336
0x84: addi t0,t0,-128	**addi t0,t0,-128**	addi t0,t0,-128
0x88: auipc t1,34070528		
0x8c: addi t1,t1,-260		
0x90: sw zero,0(t0)		
0x94: addi t0,t0,4		
0x98: bgtu t1,t0,-8		
0x9c: auipc sp,34070528		

Frozen

Linear Code Extraction

Freezing an instruction

Memory	Executed	Extracted
0x80: auipc t0,34062336	auipc t0,34062336	auipc t0,34062336
0x84: addi t0,t0,-128	**addi t0,t0,-128**	addi t0,t0,-128
0x88: auipc t1,34070528	**addi t0,t0,-128**	auipc t1,34070528
0x8c: addi t1,t1,-260		
0x90: sw zero,0(t0)		
0x94: addi t0,t0,4		
0x98: bgtu t1,t0,-8		
0x9c: auipc sp,34070528		

Frozen

Linear Code Extraction

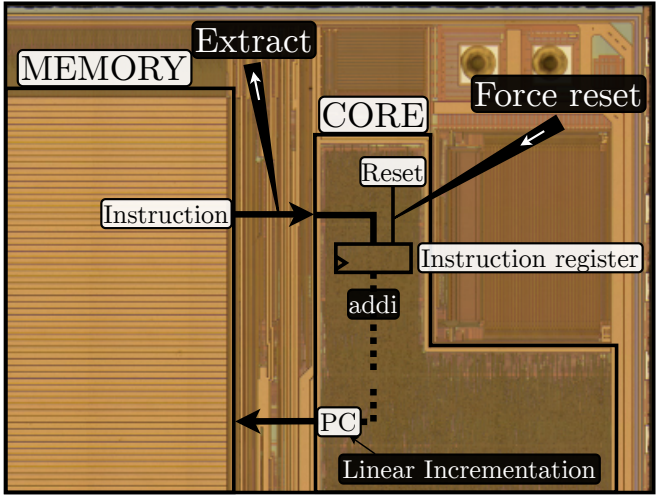
Freezing an instruction

Memory		Executed		Extracted	
0x80: auipc	t0,34062336	auipc	t0,34062336	auipc	t0,34062336
0x84: addi	t0,t0,-128	**addi**	**t0,t0,-128**	addi	t0,t0,-128
0x88: auipc	t1,34070528	**addi**	**t0,t0,-128**	auipc	t1,34070528
0x8c: addi	t1,t1,-260	**addi**	**t0,t0,-128**	addi	t1,t1,-260
0x90: sw	zero,0(t0)	**addi**	**t0,t0,-128**	sw	zero,0(t0)
0x94: addi	t0,t0,4	**addi**	**t0,t0,-128**	addi	t0,t0,4
0x98: bgtu	t1,t0,-8	**addi**	**t0,t0,-128**	bgtu	t1,t0,-8
0x9c: auipc	sp,34070528	**addi**	**t0,t0,-128**	auipc	sp,34070528

Linear Code Extraction

Lightweight countermeasures

Detecting legal instructions

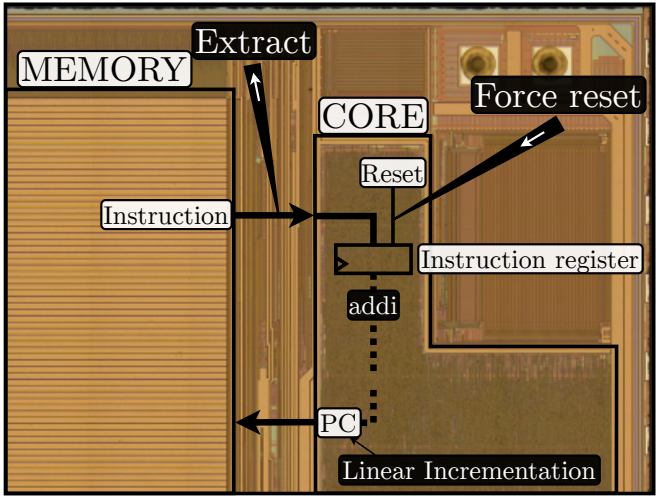


: Microprobes to eavesdrop
: Microprobe to induce a linear execution
: LCE attack effects
: Hardware components

Linear Code Extraction

Lightweight countermeasures

Detecting legal instructions



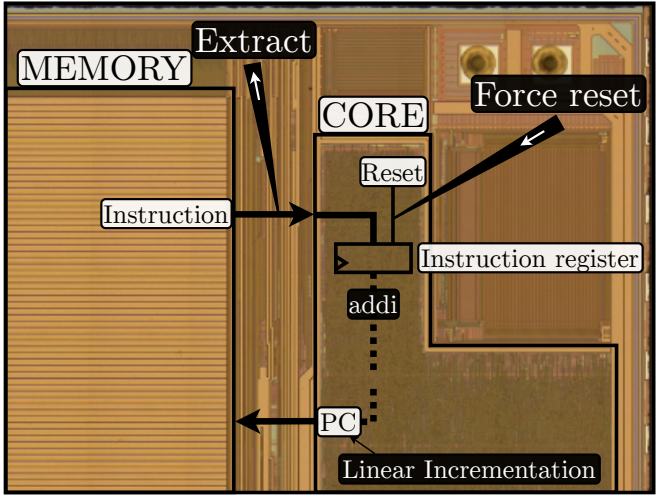
: Microprobes to eavesdrop
: Microprobe to induce a linear execution
: LCE attack effects
: Hardware components

- ✖ Software
- ➜ Hardware assisted

Linear Code Extraction

Lightweight countermeasures

Detecting legal instructions



Monitor the unexecuted instructions

: Microprobes to eavesdrop
: Microprobe to induce a linear execution
: LCE attack effects
: Hardware components

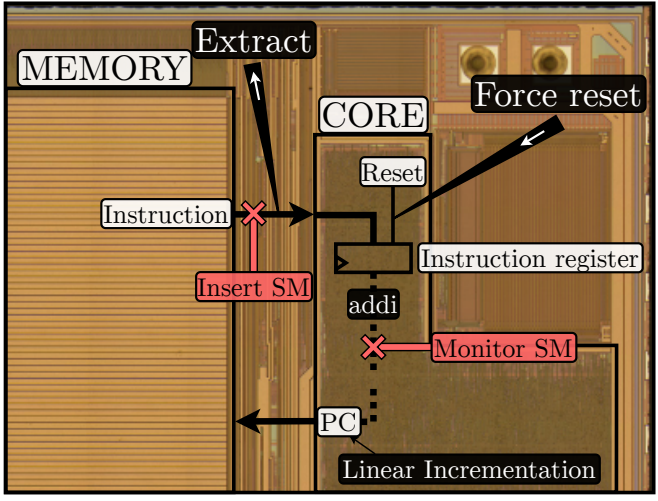
✖ Software

➜ Hardware assisted

Linear Code Extraction

Lightweight countermeasures

Detecting legal instructions



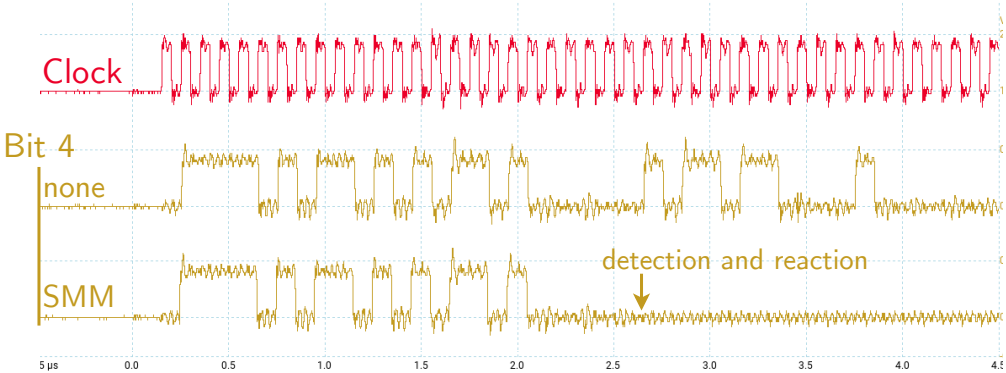
Monitor the unexecuted instructions

: Microprobes to eavesdrop
: Microprobe to induce a linear execution
: LCE attack effects
: Hardware components
SM : Security Marker (custom instruction)

Linear Code Extraction

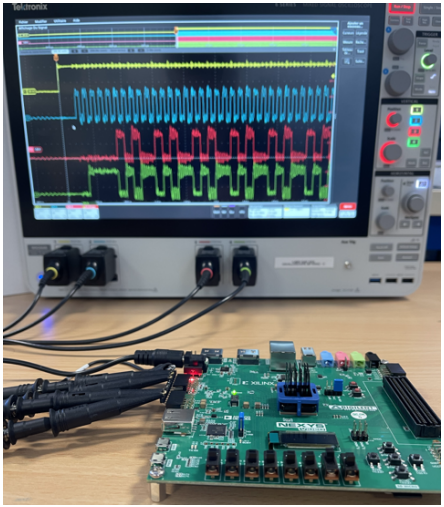
Lightweight countermeasures

Security Marker Monitoring: detection



Linear Code Extraction without/with countermeasure

Linear Code Extraction
Experimental Setup



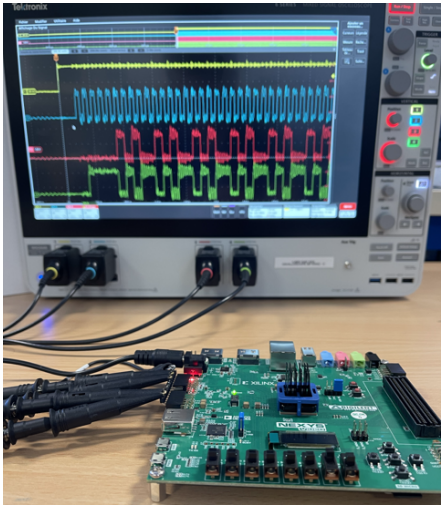
Simulation

- Embench codes
- cv32e40p

FPGA-Based demonstration

- Real-time LCE (Emulated microprobing)
- Quickly assess and observe core behavior
- Nexys Video, Artix 7

Linear Code Extraction
Experimental Setup



Simulation

- Embench codes
- cv32e40p

FPGA-Based demonstration

- Real-time LCE (Emulated microprobing)
- Quickly assess and observe core behavior
- Nexys Video, Artix 7

➜ **Area overhead: $\leq 1\%$ of cv32e40p**

MINES Saint-Étienne
Une école de l'IMT

ARSENE

FRANCE 2030

Mécanismes de sécurité d'un processeur RISC-V face aux attaques par injection de fautes

Théophile Gousselot

18 juin 2025

Mécanismes de sécurité d'un processeur RISC-V face aux attaques par injection de fautes

Théophile Gousselot

18 juin 2025

Mécanismes de sécurité d'un processeur RISC-V face aux attaques par injection de fautes

PhD Defense
Théophile Gousselot
Mines Saint-Étienne
June 18, 2025

INSPIRING INNOVATION SINCE 1816

Appendix - Table of content